

2025 - 2026

PROJET DE FIN D'ÉTUDES

DIPLÔME NATIONAL D'INGÉNIEUR

SPÉCIALITÉ : INFORMATIQUE

**Conception et réalisation d'une
architecture cloud sécurisée
Zero Trust et DevSecOps**

Réalisé par: Habiboullah Mansour

Encadré par:

Encadrant ESPRIT: Rihem Matoussi

Encadrant Entreprise: Houssein Ezzedine



CONFIDENTIALITÉ

Certaines informations utilisées dans le cadre de ce stage sont de nature interne ou confidentielle.

Afin de préserver les intérêts de **MENAL - SARL**, les données sensibles présentées dans ce rapport ont été, si nécessaire, anonymisées, adaptées ou remplacées par des données illustratives.

Le présent document est destiné à un usage académique et ne doit pas permettre la divulgation d'informations confidentielles appartenant à l'entreprise.



Je valide le dépôt du rapport PFE relatif à l'étudiant nommé ci-dessous / I
validate the submission of the student's report:

- Nom & Prénom /Name & Surname :*Hakiboullah Mansour*.....

Encadrant Entreprise/ Business site Supervisor

- Nom & Prénom /Name & Surname : ...*Houssein Fzedine*.....

Cachet & Signature / Stamp & Signature



Encadrant Académique/Academic Supervisor

- Nom & Prénom /Name & Surname :

Signature / Signature

Ce formulaire doit être rempli, signé et scanné/This form must be completed, signed and
scanned.

Ce formulaire doit être introduit après la page de garde/ This form must be inserted after the cover
page.

Dédicace

*À mes parents,
pour la confiance qu'ils m'ont accordée bien avant que je la mérite.*

*À ma famille et à mes proches,
qui ont accepté mes absences sans jamais me les reprocher.*

*À toutes celles et à tous ceux qui, à Nouakchott comme à Tunis,
m'ont ouvert une porte au bon moment.*

Je dédie ce travail.

Remerciements

Au terme de ce projet de fin d'études, je tiens à exprimer ma reconnaissance à toutes les personnes qui ont contribué à son aboutissement.

Je remercie d'abord M. Houssein Ezzedine, mon encadrant au sein de MENAL-SARL, pour la confiance qu'il m'a témoignée en me confiant la chaîne complète de ce projet, de l'audit à la mise en service. Sa disponibilité lors des points hebdomadaires et sa franchise devant chaque écart entre ce qui était prévu et ce qui était construit ont façonné la méthode de ce mémoire autant que son résultat.

Je remercie Mme Rihem Matoussi, mon encadrante pédagogique à ESPRIT, pour ses conseils, son exigence sur la structure et la rigueur du mémoire, et son suivi attentif tout au long du stage. Ses remarques ont conduit à resserrer la démarche autour d'une chaîne unique, du constat initial jusqu'à la preuve mesurée.

Je remercie l'équipe de MENAL-SARL pour son accueil et pour m'avoir donné accès à une application réelle et à des données réelles, sans lesquelles ce projet serait resté une maquette.

Je remercie enfin les membres du jury pour l'honneur qu'ils me font en acceptant d'évaluer ce travail, le corps enseignant d'ESPRIT pour la formation reçue, et ma famille pour son soutien constant.

Résumé

Ce mémoire présente la conception et la réalisation du **socle MENAL**, une plateforme d'hébergement et de supervision fondée sur les principes **Zero Trust** et **DevSecOps**, destinée à une entreprise émergente exploitable par une seule personne avec un budget de quelques dizaines d'euros par mois. L'audit de l'hébergement de l'application pilote ELSON, qui collecte des contributions vocales pour un corpus linguistique, révèle quatre constats : confiance implicite au réseau, infrastructure non reproductible, livraison non contrôlée et supervision absente.

La démarche va de la valeur à la preuve : une analyse de risque (EBIOS Risk Manager et STRIDE) sur quatre flux produit douze menaces cotées et douze exigences vérifiables, chacune reliée à un contrôle et à un test. L'architecture, déployée sur Google Cloud en trois environnements (développement, recette, production), repose sur un point d'entrée unique filtré par Cloud Armor, des conteneurs Cloud Run, une base PostgreSQL sans adresse publique, un plan d'identités sans aucune clé stockée, une authentification administrateur à deux facteurs (TOTP), une infrastructure décrite en code (Terraform), une chaîne de livraison à quatre portes bloquantes, et une supervision sur BigQuery : sept règles de détection versionnées, qualification assistée des alertes par le modèle ATT&CK-BERT, corrélation par entité et tableau de bord Next.js alimenté par une API FastAPI. Le passage des requêtes planifiées à une chaîne en flux (puits de journaux, job de détection à la minute) ramène le délai entre l'attaque et l'incident affiché de quinze minutes à moins de trois.

Une campagne de douze tests, complétée le 7 septembre 2026 par cinq refus provoqués de la chaîne de livraison et un scénario d'attaque de bout en bout, mesure le résultat : dix tests conformes et deux partiellement conformes, conservés et expliqués.

Mots-clés : Zero Trust · DevSecOps · Google Cloud · infrastructure en code · MFA · supervision de sécurité · détection comme code · MITRE ATT&CK · FastAPI · Next.js.

Abstract

This report presents the design and implementation of the **MENAL platform**, a hosting and security-monitoring platform built on **Zero Trust** and **DevSecOps** principles, intended for an emerging company and operable by a single person within a budget of a few tens of euros per month. An audit of the hosting of the pilot application ELSON, which collects voice contributions for a linguistic corpus, reveals four findings: implicit trust in the network, non-reproducible infrastructure, uncontrolled delivery and no monitoring.

The approach goes from value to proof: a risk analysis (EBIOS Risk Manager and STRIDE) over four data flows yields twelve rated threats and twelve verifiable requirements, each linked to a control and a test. The architecture, deployed on Google Cloud across three environments (dev, staging, production), relies on a single entry point filtered by Cloud Armor, Cloud Run containers, a PostgreSQL database with no public address, a keyless identity plan, two-factor administrator authentication (TOTP), infrastructure as code (Terraform), a four-gate delivery pipeline, and BigQuery-based monitoring: seven versioned detection rules, assisted alert qualification with the

ATT&CK-BERT model, per-entity correlation and a Next.js dashboard fed by a FastAPI service. Moving from scheduled queries to a streaming chain (log sink, one-minute detection job) brings the delay between an attack and its displayed incident from fifteen minutes down to under three.

A campaign of twelve tests, completed on 7 September 2026 with five provoked pipeline refusals and an end-to-end attack scenario, measures the outcome: ten tests pass and two partially pass, kept and explained.

Keywords: Zero Trust · DevSecOps · Google Cloud · infrastructure as code · MFA · security monitoring · detection as code · MITRE ATT&CK · FastAPI · Next.js.

Sommaire

Introduction générale	1
1 Contexte général du projet	2
1.1 Cadre du projet et organisme d'accueil	2
1.2 L'application pilote ELSON et l'actif à protéger	3
1.3 Étude de l'existant	3
1.4 Problématique	4
1.5 Solution proposée : le socle MENAL	5
1.6 Méthodologie de travail	6
2 État de l'art et choix technologiques	7
2.1 L'architecture Zero Trust	7
2.2 L'approche DevSecOps	8
2.3 Supervision et détection de sécurité	9
2.3.1 Du SIEM à la supervision sur entrepôt	9
2.3.2 MITRE ATT&CK et qualification assistée des alertes	9
2.4 Choix de la plateforme d'hébergement : Google Cloud	10
2.5 Technologies retenues, une par une	10
2.5.1 Terraform — l'infrastructure en code	10
2.5.2 GitHub Actions et fédération OIDC — la chaîne de livraison	11
2.5.3 Gitleaks, Semgrep, pytest/jest, Trivy — les quatre portes	11
2.5.4 Docker multi-étape et Cloud Run — l'exécution	11
2.5.5 PostgreSQL sur Cloud SQL — les données transactionnelles	11
2.5.6 FastAPI — l'API de supervision	11
2.5.7 Next.js — le tableau de bord	11
2.5.8 Authentification à deux facteurs — TOTP	11
2.5.9 BigQuery, Log Router et Cloud Scheduler — la chaîne de détection	12
2.5.10 ATT&CK-BERT et ONNX — la qualification assistée	12
2.6 Référentiels mobilisés et positionnement	12
3 Analyse des besoins et modélisation des menaces	13
3.1 Acteurs et cas d'utilisation	13
3.2 Spécification des besoins	14
3.3 Flux de données et frontières de confiance	15
3.4 Analyse du risque	15
3.5 Exigences de sécurité et traçabilité	16
4 Conception de l'architecture	18
4.1 Les quatre principes directeurs	18
4.2 Vue d'ensemble : trois environnements, un déploiement	18
4.2.1 Trois environnements décrits par le même code	18
4.2.2 Le diagramme de déploiement	19

4.3	Le modèle en cinq couches	19
4.4	Le point d'entrée et le réseau	21
4.5	Les identités et les secrets	22
4.6	La session administrateur à deux facteurs (MFA)	22
4.7	Les données	24
4.7.1	Données applicatives : une base par application, une instance privée	24
4.7.2	L'entrepôt de sécurité	24
4.8	La chaîne de détection en flux : de l'événement à l'incident	25
4.9	L'automatisation	26
4.10	Registre des décisions et composants écartés	26
5	Réalisation	28
5.1	Environnement de travail et organisation du dépôt	28
5.2	Chaîne de provisionnement de l'infrastructure (F2)	29
5.3	Chaîne de livraison applicative (F3)	29
5.3.1	Huit étapes, quatre portes	29
5.3.2	Les refus vérifiés	30
5.4	Chaîne de supervision et de détection (F4)	31
5.4.1	Collecte en flux et normalisation	31
5.4.2	Les sept règles de détection	31
5.4.3	La qualification assistée	31
5.4.4	L'API de supervision	32
5.4.5	Le tableau de bord MENAL Sentinel et l'accès à deux facteurs	32
5.5	Mise en service d'ELSON sur le socle	34
5.5.1	Prérequis et procédure d'accueil	34
5.5.2	Ce qui a été mis en service et pourquoi	34
5.5.3	Portée de l'isolation et frontière de responsabilité	36
5.6	Conduite du projet	36
5.6.1	Tâches quotidiennes	36
5.6.2	Difficultés rencontrées et corrections	36
5.6.3	Écarts entre conception et réalisation	36
5.6.4	Planning du stage	36
6	Validation, résultats et limites	39
6.1	Stratégie de validation	39
6.2	Résultats de la campagne de tests	39
6.3	Démonstration de bout en bout	41
6.4	Évaluation de la détection et de la qualification assistée	41
6.5	Performance, résilience et coût	42
6.6	Confrontation aux principes Zero Trust et maturité	42
6.7	Limites et perspectives	43
	Conclusion générale	45
	Bibliographie / Nétographie	45

A	Dépôt Git, environnements et extraits de code	48
A.1	Structure du dépôt	48
A.2	Variables propres à chaque environnement	48
A.3	Module <code>run-service</code> (extrait)	49
A.4	Chaîne de livraison applicative : les portes (extrait de <code>app-delivery.yml</code>)	49
A.5	Fédération d'identité de la chaîne (extrait)	50
A.6	Pare-feu de sortie (extrait)	50
A.7	Règle de détection R6 (extrait simplifié)	50
A.8	Test automatisé T12 (extrait)	51
B	Diagrammes et écrans complémentaires	52
B.1	Matrice de risque	52
B.2	Principe de la qualification assistée	53
B.3	Diagrammes de classes de l'API de supervision	53
B.4	Écrans secondaires du tableau de bord	54
B.5	Séquences de la livraison (UC2)	55
C	Relevés de la chaîne de livraison (07/09/2026)	56
D	Relevés de configuration et de tests	58
D.1	Point d'entrée et réseau (EX1, EX3)	58
D.2	Base de données privée et isolée (T3, T4)	59
D.3	Identités, secrets et fédération (T4, T10)	60
D.4	Entrepôt : droits par table et volumes (T11, T12)	62
D.5	Chaîne de livraison (T8, T9)	62
D.6	Protocole d'évaluation de la qualification assistée	63
E	Correspondance MITRE ATT&CK	64

Table des figures

1.1	Organigramme fonctionnel de MENAL-SARL et positionnement du projet. Le périmètre d'accueil réunit les fonctions « Cloud et DevOps » et « Cyber et SOC » de la direction technique.	2
1.2	Architecture de l'existant : un serveur unique protégé par un pare-feu réseau. Le déploiement et l'administration (traits rouges) contournent le pare-feu. Les repères C1 à C4 renvoient au tableau 1.1.	3
1.3	Démarche du projet et correspondance avec les chapitres. La flèche rouge est la boucle de correction : un test qui échoue au chapitre 6 renvoie à la conception.	6
2.1	Du modèle périmétrique (A) au Zero Trust (B) : un point de décision vérifie chaque accès selon l'identité, les droits et le contexte ; le réseau privé reste une protection complémentaire.	7
3.1	Diagramme des cas d'utilisation du socle. A4 exécute la livraison déclenchée par A3 ; les acteurs hostiles appartiennent au modèle de menace, non aux cas d'utilisation.	14
3.2	Flux de données et frontières de confiance. TB1 et TB2 sont franchies par un chemin réseau ; TB3 par des appels aux services managés, où l'autorisation dépend d'abord de l'identité présentée.	15
4.1	Les trois environnements du socle et leur chemin de promotion. Un projet Google Cloud, un répertoire et un état Terraform par environnement ; le code est identique.	19
4.2	Diagramme de déploiement du socle (environnement de recette, relevé du 08/09/2026). Un seul chemin entrant (443 vers le répartiteur), un seul chemin vers la base (connecteur d'accès VPC puis adresse privée 10.20.0.3 sur le port 5432), une journalisation en flux (journaux du WAF et des services, repère J, acheminés par le puits vers BigQuery) et des liaisons vers les services managés dont l'autorisation dépend de l'identité, non du réseau. Les flèches horizontales de la zone Cloud Run représentent : tableau de bord → API (jeton), job de détection → enrichissement (déclenchement), enrichissement → encodeur (encodage).	20
4.3	Séquence de connexion de l'administrateur : le jeton intermédiaire (étape 2) n'ouvre aucun accès ; seul le jeton d'accès délivré après le code TOTP (étape 6) autorise les appels. Le tableau de bord, simple relais vers l'API, n'est pas représenté.	23
4.4	La chaîne de détection en flux et le budget de temps de chaque maillon. Le délai n'est pas dans le calcul mais dans l'acheminement et la cadence ; chaque maillon est borné et mesurable sur les horodatages des tables.	25
5.1	Services et jobs Cloud Run de la recette (relevé du 08/09/2026) : cinq services et trois jobs, chacun sous sa propre identité.	28
5.2	La chaîne de livraison applicative : huit étapes, dont quatre portes bloquantes (rouge). Une porte rouge ne fait pas qu'échouer : tous les jobs suivants passent en <i>skipped</i> , aucune image n'est publiée, et la chaîne ne s'authentifie même pas au cloud.	30
5.3	Porte 6 fermée : Trivy identifie la CVE critique corrigable injectée (run 34165859372)	31
5.4	Accès administrateur à deux facteurs : mot de passe (gauche), puis code de vérification TOTP (centre) généré par l'application d'authentification du compte admin@menal-sarl.mr (droite). Entre les deux écrans, la session n'a aucun droit.	33

5.5	Vue d'ensemble de MENAL Sentinel en recette (07/09/2026, 23 h 27, deux minutes après l'attaque du scénario du chapitre 6) : 47 détections, 1 868 blocages WAF, modèle <code>attack-bert-onnx-fp32@v1.0</code> , pic de trafic visible à 23 h 26.	33
5.6	Écran des détections (recette, 07/09/2026) : les règles R6 (injection, critique), R3 (traversée de chemin, haute) et R2 (pic WAF, moyenne) déclenchées par une même adresse source (masquée, RFC 5737), chacune portant sa tactique et sa technique MITRE.	34
5.7	Écran des incidents : les 47 détections de l'adresse source sont corrélées en un incident unique de score 100, sévérité critique, deux tactiques distinctes (chaîne d'attaque probable, T1190 et T1498), verdict « Confirmé » par l'administrateur à 23 h 28.	34
5.8	Page d'accueil d'ELSON servie par le socle à <code>elson.menal-sarl.com</code> (HTTPS par le répartiteur, WAF actif) : plateforme trilingue de constitution de corpus pour les langues nationales.	35
5.9	Parcours d'un contributeur ELSON derrière le socle : connexion (identifiant masqué), validation d'une contribution (phrase source, traduction hassaniya, audio à évaluer), espace contributeur (points de la session de compétition) et classement. Ces écrans concentrent la valeur à protéger ; identités des utilisateurs et logique métier restent la responsabilité de l'application.	35
5.10	Planning du stage du 03/03/2026 au 07/09/2026 : neuf phases et sept jalons. Les phases se chevauchent volontairement ; les itérations sont de deux semaines.	38
6.1	Scénario du 07/09/2026, 23 h 25 min 38 s : treize charges d'attaque envoyées vers <code>elson.menal-sarl.com/api/search</code> depuis un poste Kali ; treize refus 403 rendus par Cloud Armor avant l'application (T1, rang 3).	41
6.2	Chronologie du scénario du 07/09/2026, relevée sur la capture Kali et les captures horodatées du tableau de bord (figures 5.5 à 5.7). Le verdict humain intervient 2 min 23 s après l'envoi de l'attaque.	41
B.1	Matrice de risque : position des douze menaces selon leur gravité et leur vraisemblance (tableau 3.3).	52
B.2	Chaîne de traçabilité, de la valeur à la preuve, et son instance sur le cas M3 → EX3 → T3.	52
B.3	Principe de la qualification assistée : l'alerte et les descriptions des techniques ATT&CK sont encodées en vecteurs ; les trois techniques les plus proches sont proposées à l'administrateur, qui décide.	53
B.4	Couche domaine : <code>Detection</code> , <code>Enrichment</code> et <code>Verdict</code> n'exposent aucune opération de modification ni de suppression ; l'immutabilité des preuves est portée par le modèle objet et par les droits IAM (T12).	53
B.5	Couche service : dépôts en lecture seule sur l'entrepôt, service d'authentification à deux jetons, service de corrélation par entité.	54
B.6	Santé des règles (gauche) : les sept règles, leur sévérité, leur technique et leurs déclenchements sur trente jours — R6, R3 et R2 ont réagi au scénario du 07/09, les quatre autres sont silencieuses. Vulnérabilités priorisées (droite) : CVE de l'image livrée, alimentées par le rapport Trivy de la chaîne, avec statut KEV, score EPSS et version corrective.	54
B.7	Phase 1 — demande de fusion : les portes s'exécutent, l'image est construite mais rien n'est publié ni déployé ; aucun job ne détient de permission d'identité cloud.	55
B.8	Phase 2 — fusion sur la branche principale : authentification fédérée, publication par empreinte, déploiement, sonde par le point d'entrée ; le retour arrière est déclenché par l'administrateur.	55

C.1	Run vert de référence (34165407841, fusion sur <code>main</code>) : les huit étapes dans l'ordre du diagramme, dédoublées <code>api/dashboard</code> à partir de l'étape 4 ; total 5 min 37.	56
C.2	Porte 2 fermée : Gitleaks détecte la clé injectée (run 34165824113)	56
C.3	Porte 3 fermée (run 34165832135) : Semgrep signale les deux constats injectés (<code>eval</code> sur entrée utilisateur, <code>subprocess</code> avec <code>shell=True</code>) sur 290 règles exécutées ; les étapes 4 à 8 sont ignorées.	56
C.4	Porte 4 fermée côté API (run 34165841139) : un test en échec sur trente ; construction, analyse d'image, publication et déploiement ignorés.	57
C.5	Porte 4 fermée côté tableau de bord (run 34165850179) : un test en échec sur vingt-trois ; la porte réagit à la régression précise sans casser en bloc.	57
D.1	Services Cloud Run (console) : les quatre services publics n'acceptent que le trafic interne et celui du répartiteur (<i>Internal + Load Balancing</i>) ; l'encodeur est interne et exige une authentification.	58
D.2	Connecteur d'accès VPC <code>menal-vpc-connector-stg</code> (10.0.3.0/28, état <code>READY</code>) et plage d'appairage de services privés <code>google-services-staging</code> (10.20.0.0/16) qui porte l'adresse de Cloud SQL.	58
D.3	T3 (rang 3) : instance <code>menal-db-staging</code> , PostgreSQL 15, adresse 10.20.0.3 de type <code>PRIVATE</code> ; trois bases (<code>postgres</code> , <code>menal_db</code> , <code>elson_db</code>) ; <code>ipv4Enabled = False</code> , <code>sslMode = ENCRYPTED_ONLY</code>	59
D.4	T4 (rang 1) : cinq exécutions quotidiennes consécutives du job <code>elson-sql-isolation-check-staging</code> (du 04 au 08/09/2026) et le journal de la dernière : aucun utilisateur super-utilisateur, connexions croisées refusées dans les deux sens, accès propres préservés, <code>isolated = True</code>	59
D.5	Rôles IAM du projet (console) : un compte de service par composant avec ses seuls rôles techniques ; le compte Compute par défaut conserve le rôle Éditeur hérité de la création du projet (plan d'amélioration).	60
D.6	Secret Manager : neuf secrets, un par usage ; huit chiffrés par la clé Cloud KMS du socle.	60
D.7	T4 (rang 3) : pour chaque secret, l'unique identité autorisée à le lire (<code>sa-elson</code> pour les cinq secrets d'ELSON, <code>sa-api</code> pour la base, la signature des jetons et la clé MFA, <code>sa-dashboard</code> pour son secret de session).	61
D.8	T10 (rang 3 et 4) : condition de la fédération d'identité (dépôt <code>Mansour37/menal-zero-trust</code> , branche <code>main</code>) et inventaire des clés : zéro clé sur les sept comptes de service dédiés.	61
D.9	T12 (rang 3) : droits du jeu de données <code>menal_security_staging</code> — écriture pour <code>sa-cicd</code> , <code>sa-pipeline</code> et l'agent du puits de journaux ; lecture pour <code>sa-api</code> , <code>sa-dashboard</code> et <code>sa-enrich-job</code> ; chiffrement par la clé KMS <code>menal-api-key-staging</code>	62
D.10	T11 : volumes de l'entrepôt au 08/09/2026 (232 254 journaux bruts) et répartition des requêtes normalisées par service émetteur.	62
D.11	Précision des quatre méthodes de qualification sur 40 alertes, au rang 1 puis aux rangs 1–3 : dictionnaire 0,45/0,60 ; TF-IDF 0,53/0,70 ; ATT&CK-BERT float32 (retenu) 0,68/0,88 ; ATT&CK-BERT int8 (rejeté) 0,05/0,33. Cible : 0,80 aux rangs 1–3.	63

Liste des tableaux

1.1	Les quatre constats de l'audit de l'existant.	4
1.2	Les quatre objectifs du projet et leurs indicateurs de réussite.	5
1.3	Identifiants utilisés dans le mémoire.	6
2.1	Les sept principes du NIST SP 800-207 et leur traduction pour le socle.	8
2.2	Les quatre familles de contrôles automatiques d'une chaîne DevSecOps et les outils retenus.	8
2.3	Comparaison qualitative des options de supervision (+ favorable, = intermédiaire, – défavorable).	9
2.4	Services Google Cloud mobilisés, rôle dans le socle et bonne pratique appliquée.	10
3.1	Les six acteurs du modèle.	13
3.2	Besoins fonctionnels (BF) et non fonctionnels (BNF) du socle.	14
3.3	Les douze menaces retenues, par flux et catégorie STRIDE, avec leur cotation (gravité × vraisemblance).	16
3.4	Matrice de traçabilité : exigence, critère de vérification, contrôle prévu, test et référentiel.	17
4.1	Les cinq couches du socle et les exigences qu'elles portent.	21
4.2	Matrice des identités : les interdictions sont la segmentation (inventaire IAM du 08/09/2026 : sept comptes de service dédiés sans aucune clé, plus le compte Compute par défaut).	23
4.3	Modèle de données de l'entrepôt de sécurité : un auteur par table (volumes relevés le 08/09/2026).	25
4.4	Registre des décisions d'architecture D1 à D9.	27
5.1	Les cinq refus provoqués du 07/09/2026 : une faille injectée, un type détecté, un verdict daté (identifiants de run GitHub Actions).	30
5.2	Les sept règles de détection R1 à R7 (job à la minute, fenêtre de cinq minutes).	32
5.3	Frontière de responsabilité entre le socle (hébergeur) et l'éditeur de l'application hébergée.	36
5.4	Les sept difficultés rencontrées, leur correction et la leçon retenue.	37
5.5	Les cinq écarts entre conception et réalisation.	37
6.1	Synthèse de la campagne de douze tests, à la date du 07/09/2026.	40
6.2	Mesures de performance, de résilience, d'exploitabilité et de coût.	43
6.3	Confrontation du socle aux sept principes du NIST SP 800-207 et réponse aux objectifs.	43
6.4	Plan d'amélioration issu de la revue finale du 08/09/2026.	44
A.1	Variables Terraform différant entre environnements.	48
E.1	Règles du socle, alertes de plateforme et techniques MITRE ATT&CK (Enterprise v17.1).	64

Liste des abréviations

Sigle	Signification
API	Interface de programmation (<i>Application Programming Interface</i>)
ATT&CK	<i>Adversarial Tactics, Techniques and Common Knowledge</i> — référentiel MITRE des techniques d'attaque
BERT / ONNX	Modèle de langue (<i>Bidirectional Encoder Representations from Transformers</i>) / format d'échange de modèles (<i>Open Neural Network Exchange</i>)
BF / BNF	Besoin fonctionnel / non fonctionnel
CI/CD	Intégration et livraison continues (<i>Continuous Integration / Delivery</i>)
CIS, CISA	<i>Center for Internet Security</i> ; <i>Cybersecurity and Infrastructure Security Agency</i>
CVE	<i>Common Vulnerabilities and Exposures</i> — identifiant public d'une vulnérabilité
DevSecOps	Intégration de la sécurité dans la chaîne de développement et d'exploitation
EBIOS RM	Expression des Besoins et Identification des Objectifs de Sécurité — Risk Manager (ANSSI)
GCP	<i>Google Cloud Platform</i>
IaC	Infrastructure en code (<i>Infrastructure as Code</i>)
IAM	Gestion des identités et des accès (<i>Identity and Access Management</i>)
JWT	Jeton web signé (<i>JSON Web Token</i>)
KMS	Service de gestion de clés (<i>Key Management Service</i>)
LB / WAF	Répartiteur de charge (<i>Load Balancer</i>) / pare-feu applicatif (<i>Web Application Firewall</i>)
MFA / TOTP	Authentification à plusieurs facteurs / code temporel à usage unique (<i>Time-based One-Time Password</i> , RFC 6238)
NIST	<i>National Institute of Standards and Technology</i>
OIDC / WIF	<i>OpenID Connect</i> / fédération d'identité des charges de travail (<i>Workload Identity Federation</i>)
OWASP	<i>Open Worldwide Application Security Project</i>
PFE	Projet de fin d'études
PITR	Restauration à un instant donné (<i>Point-In-Time Recovery</i>)
RGPD	Règlement général sur la protection des données (UE 2016/679)
RPO / RTO	Perte de données maximale admissible / délai de reprise maximal
SAST / SCA	Analyse statique du code / analyse de composition logicielle
SBOM	Nomenclature logicielle (<i>Software Bill of Materials</i>)
SIEM	Système de gestion des informations et événements de sécurité
SLO	Objectif de niveau de service (<i>Service Level Objective</i>)
SQL	Langage de requête structuré
STRIDE	<i>Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege</i>
TLS	Sécurité de la couche de transport (<i>Transport Layer Security</i>)
UC	Cas d'utilisation (<i>Use Case</i>)
VPC	Réseau privé virtuel (<i>Virtual Private Cloud</i>)

Sigle	Signification
XSS / SQLi / LFI	Script inter-sites / injection SQL / inclusion de fichier local

Introduction générale

Protéger une frontière ne suffit plus à sécuriser un système d'information. Les applications tournent dans le cloud, les accès arrivent de n'importe où, et les incidents les plus graves empruntent souvent des chemins que l'on croyait internes : une clé laissée dans un dépôt de code, un compte légitime volé, une modification manuelle passée inaperçue. Deux réponses se sont généralisées : le **Zero Trust**, pour lequel chaque autorisation se décide sur l'identité et non sur la position dans le réseau, et le **DevSecOps**, qui place les contrôles de sécurité dans la chaîne de développement plutôt qu'à sa sortie.

Ces deux approches sont bien documentées et outillées pour des équipes spécialisées aux budgets conséquents ; elles le sont beaucoup moins à l'échelle d'une entreprise émergente. MENAL-SARL, jeune entreprise mauritanienne positionnée sur le cloud et la cybersécurité, développe ELSON, une application qui collecte des contributions vocales et textuelles pour constituer un corpus dans des langues nationales peu outillées numériquement. Ce corpus est son premier actif et les données des participants sont des données personnelles. ELSON tournait pourtant sur un serveur unique derrière un seul pare-feu, était déployée par un script lancé depuis un poste personnel et n'était pas supervisée ; pour y remédier, l'entreprise ne pouvait compter que sur une personne technique et sur quelques dizaines d'euros par mois.

La question posée est donc pratique : *avec ces moyens, comment construire une plateforme d'hébergement qui applique réellement le Zero Trust et le DevSecOps, qui porte ELSON puis d'autres applications par simple paramétrage, et dont la sécurité se démontre au lieu de s'affirmer ?* Elle se décline en quatre sous-questions : faire de l'identité le critère principal d'accès ; décrire toute l'infrastructure en code ; ne déployer aucune version sans contrôles automatiques ni clé permanente ; détecter en quelques minutes et aider une personne seule à qualifier.

La réponse retenue est un **socle** : une base d'hébergement et de supervision partagée par toutes les applications de l'entreprise, à laquelle chaque application se raccorde en déclarant ses paramètres. La démarche va *de la valeur à la preuve* : identifier ce qui doit être protégé, en déduire des exigences vérifiables, concevoir et construire l'architecture qui y répond, puis la mesurer avec des tests rédigés avant la conception.

Six chapitres suivent cette démarche. Le **chapitre 1** situe le projet : l'entreprise, l'application pilote, l'audit de l'existant, la problématique, les objectifs et les contraintes. Le **chapitre 2** confronte ces objectifs à l'état de l'art et justifie chaque technologie retenue. Le **chapitre 3** transforme le cadre en modèle vérifiable : acteurs, flux, frontières de confiance, douze menaces et douze exigences reliées à douze tests. Le **chapitre 4** conçoit l'architecture : environnements, déploiement, réseau, identités, session à deux facteurs, données et chaîne de détection. Le **chapitre 5** rend compte de la réalisation : chaînes automatisées, supervision, mise en service d'ELSON, conduite du projet. Le **chapitre 6** mesure : campagne de tests, scénario de bout en bout, qualification assistée, performances, coût et limites.

Chapitre 1

Contexte général du projet

De l'application ELSON au besoin d'un socle d'hébergement sécurisé, reproductible et supervisé.

Introduction

Ce chapitre situe le projet : le cadre du stage, l'entreprise et le service d'accueil, l'application pilote ELSON et la valeur de ses données. L'étude de l'existant met en évidence quatre constats qui fondent la problématique ; le chapitre se termine par les objectifs, les contraintes, le périmètre et la démarche suivie. Il ne présente aucun résultat : il pose le problème.

1.1 Cadre du projet et organisme d'accueil

Ce projet de fin d'études s'inscrit dans le cycle d'ingénieur de l'École Supérieure Privée d'Ingénierie et de Technologies (ESPRIT). Il s'est déroulé chez MENAL-SARL, à Nouakchott (Mauritanie), du 3 mars au 7 septembre 2026, sous la direction de M. Houssein Ezzedine pour l'entreprise et de Mme Rihem Matoussi pour l'école.

MENAL-SARL est une jeune entreprise mauritanienne positionnée sur deux axes : des services d'ingénierie cloud et cybersécurité pour des organisations locales, et le développement de ses propres produits numériques, notamment dans le domaine des données linguistiques. Son effectif est réduit — une seule personne est référente sur l'infrastructure et la sécurité — et ses moyens sont ceux d'une structure émergente : toute solution doit pouvoir être exploitée par une personne et coûter quelques dizaines d'euros par mois.

Le service d'accueil est l'équipe technique, responsable de l'hébergement, de la sécurisation et de la supervision des applications (figure 1.1) ; ses pôles sont des fonctions, souvent tenues par la même personne. Ma mission couvrirait la chaîne complète — auditer l'existant, concevoir, construire, mettre en service pour ELSON, mesurer — dans les trois rôles d'architecte, de développeur et d'exploitant, ce qui structure les chapitres 4 à 6.

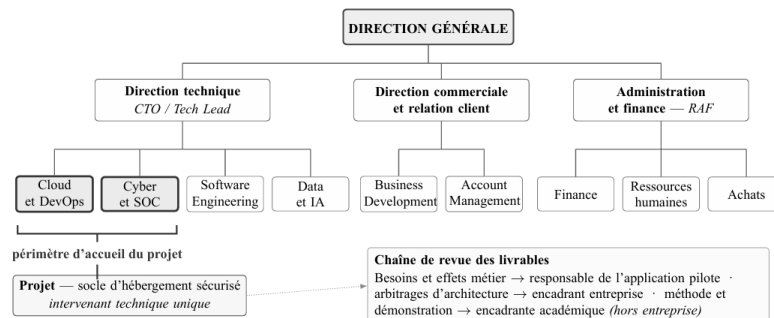


Fig. 1.1 – Organigramme fonctionnel de MENAL-SARL et positionnement du projet. Le périmètre d'accueil réunit les fonctions « Cloud et DevOps » et « Cyber et SOC » de la direction technique.

1.2 L'application pilote ELSON et l'actif à protéger

ELSON (`elson.menal-sarl.com`) est une plateforme web développée par MENAL-SARL avec ses partenaires. Des participants y traduisent des phrases et enregistrent leur lecture dans des langues nationales (hassaniya, pulaar, soninké, wolof) ; des relecteurs valident chaque contribution ; un classement et une compétition dotée de prix entretiennent la participation. Ces contributions constituent progressivement un *corpus linguistique* destiné à l'entraînement de systèmes de reconnaissance et de synthèse de la parole.

Définition — corpus linguistique

Ensemble structuré d'enregistrements (ici de la voix et du texte) associés à leur transcription et à des métadonnées. Sa valeur tient à sa taille, à sa qualité et à son exclusivité : un corpus que nul autre ne détient constitue un avantage économique durable.

ELSON manipule deux catégories de données sensibles : les **contributions**, actif principal de l'entreprise dont la copie ferait perdre l'avantage d'exclusivité, et les **informations d'identité** des participants (nom, téléphone, courriel), données personnelles au sens de la loi mauritanienne n° 2017-020 [1] et, par analogie, du RGPD [2]. La compétition ajoute un troisième enjeu : la fraude (contributions dupliquées ou générées) n'attaque pas l'infrastructure, elle utilise des fonctions autorisées ; le socle devra en observer les volumes, la décision restant à l'application.

1.3 Étude de l'existant

Avant le projet, ELSON fonctionnait sur un serveur unique loué chez un hébergeur européen (figure 1.2). L'application (site et API) et la base PostgreSQL partageaient la même machine. Un pare-feu réseau à l'entrée était la seule frontière entre un extérieur hostile et un intérieur réputé sûr : le modèle *périmétrique*. Trois chemins coexistaient : les participants en HTTPS à travers le pare-feu ; le développeur déployant par un script depuis son poste avec une clé de longue durée ; l'administrateur en SSH. Les journaux restaient sur le disque, sans centralisation ni analyse.

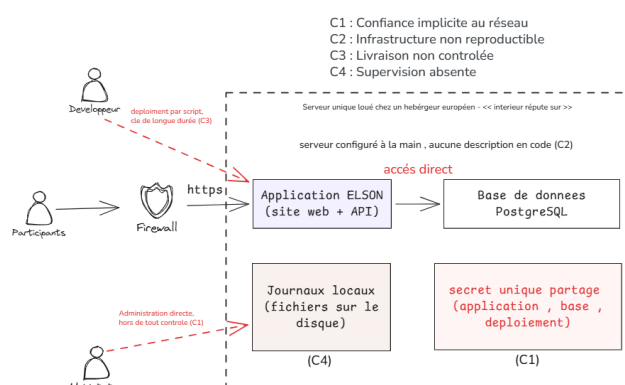


Fig. 1.2 – Architecture de l'existant : un serveur unique protégé par un pare-feu réseau. Le déploiement et l'administration (traits rouges) contournent le pare-feu. Les repères C1 à C4 renvoient au tableau 1.1.

L'audit mené en mars et avril 2026 reconnaît d'abord ce qui fonctionne : l'application est en production, HTTPS est en place et le choix de PostgreSQL est sain. Les faiblesses viennent de l'organisation qui entoure l'application (tableau 1.1). Ces constats ne sont pas indépendants : C1

explique pourquoi la fuite d'un seul secret serait grave, C3 comment il pourrait fuir, C4 pourquoi personne ne s'en apercevrait, C2 pourquoi la reconstruction serait longue.

Tab. 1.1 – Les quatre constats de l'audit de l'existant.

#	Constat	Ce qui a été observé	Risque principal
C1	Confiance implicite au réseau	Le pare-feu d'entrée est le seul contrôle ; à l'intérieur, tout processus joint la base ; un même secret sert à l'application, à la base et au déploiement.	Un attaquant qui franchit la frontière une seule fois accède à tout, y compris au corpus.
C2	Infrastructure non reproductible	Serveur configuré à la main ; aucune description en code ; une partie du schéma de base hors dépôt.	Personne ne peut reconstruire le service à l'identique ; la connaissance est dans une tête.
C3	Livraison non contrôlée	Déploiement par script depuis un poste personnel avec une clé qui n'expire jamais ; aucune analyse du code, des secrets, des dépendances.	Un secret ou une bibliothèque vulnérable arrive en production sans être vu ; le vol de la clé donne le contrôle du déploiement.
C4	Supervision absente	Journaux locaux, non centralisés ; aucune règle de détection, aucune alerte, aucun tableau de bord.	Une attaque réussie resterait invisible ; impossible de dire quand et comment elle a eu lieu.

1.4 Problématique

Deux approches répondent, chacune pour une partie, à ces constats. Le **Zero Trust** — aucun accès n'est accordé d'avance, même depuis l'intérieur ; chaque requête est vérifiée selon l'identité, les droits et le contexte (NIST SP 800-207 [3]) — répond à C1. Le **DevSecOps** — contrôles de sécurité automatiques intégrés à la chaîne de développement et de déploiement, infrastructure en code — répond à C2 et C3. La supervision, indissociable du Zero Trust, répond à C4. La difficulté n'est pas de connaître ces approches, mais de les appliquer avec les moyens d'une entreprise émergente.

Problématique

Comment une entreprise émergente, qui ne dispose que d'une personne technique et de quelques dizaines d'euros par mois, peut-elle concevoir et réaliser une plateforme d'hébergement cloud fidèle aux principes Zero Trust et DevSecOps, capable de porter ELSON puis d'autres applications par simple paramétrage, et dont la sécurité se démontre au lieu de s'affirmer ?

Elle se décline en quatre questions, une par constat : **Accès** (C1) — comment donner à l'identité, plutôt qu'à la position réseau, le rôle de critère principal d'autorisation ? **Infrastructure** (C2) — comment décrire l'infrastructure entière en code pour pouvoir la reconstruire et repérer toute modification faite en dehors ? **Livraison** (C3) — comment interdire tout déploiement sans contrôles automatiques et sans clé permanente ? **Supervision** (C4) — comment centraliser les journaux, détecter en quelques minutes, aider à qualifier et restituer dans un tableau de bord, le tout à faible coût ?

1.5 Solution proposée : le socle MENAL

Définition — socle

Ensemble formé par les services cloud, le code d'infrastructure, les chaînes automatisées et les procédures grâce auxquels une application est hébergée, sécurisée et supervisée. Une application s'y raccorde en déclarant ses paramètres (nom, image, secrets, base de données) sans modifier le socle lui-même.

Sans socle, chaque nouvelle application exigerait de refaire la sécurité, la supervision et le déploiement ; avec lui, la sécurité devient un service inclus dès le premier jour, et le socle peut devenir à terme une offre d'hébergement sécurisé (le chapitre 6 précise ce qui manque encore). Le projet poursuit quatre objectifs, un par constat, chacun associé à un indicateur vérifié au chapitre 6 (tableau 1.2), sous trois contraintes transversales :

- **K1 — humaine** : le socle est exploité par une seule personne ; on préfère les services managés et l'automatisation, et l'on écarte tout composant exigeant une exploitation permanente ;
- **K2 — économique** : coût mensuel cible ≤ 50 € en fonctionnement normal ; chaque coût fixe est identifié et justifié ;
- **K3 — données personnelles** : chiffrement, journaux d'accès, sauvegarde et restauration ; le socle fournit les moyens techniques, la conformité juridique reste de la responsabilité de l'entreprise.

Tab. 1.2 – Les quatre objectifs du projet et leurs indicateurs de réussite.

#	Objectif	Répond à	Indicateur de réussite (vérifié au chapitre 6)
O1	Contrôler chaque accès par l'identité (Zero Trust)	C1	Une identité par service ; aucune clé de longue durée ; base sans adresse publique ; second facteur pour l'administrateur ; accès non autorisés refusés aux tests
O2	Rendre l'infrastructure reproductible (IaC)	C2	Reconstruction d'un environnement depuis le seul dépôt ; toute modification hors code détectée
O3	Sécuriser et automatiser la livraison (DevSecOps)	C3	Quatre portes bloquantes (secrets, code, tests, image) éprouvées par des refus ; 100 % des déploiements par la chaîne ; identité sans clé
O4	Superviser et détecter	C4	Journaux centralisés ; règles actives ; incident affiché moins de trois minutes après l'attaque ; qualification assistée et tableau de bord utilisables par l'administrateur

L'objectif O4 contient l'apport le plus original du projet : la **qualification assistée des alertes** — pour chaque alerte brute, le socle propose les techniques d'attaque du référentiel MITRE ATT&CK [4] les plus proches ; l'administrateur gagne du temps et garde la décision.

Périmètre. Le périmètre est l'hébergement, la sécurisation et la supervision des applications MENAL sur le socle, ELSON étant le cas pilote. Sont hors périmètre : la logique métier des applications (dont la règle qui décide qu'une contribution est frauduleuse) ; la gouvernance juridique des données (base légale, durées de conservation) ; la haute disponibilité multi-région et la protection contre un adversaire étatique ; l'acquisition d'un SIEM commercial, incompatible avec K2.

1.6 Méthodologie de travail

Six étapes structurent la démarche et les six chapitres (figure 1.3) : des constats découlent les objectifs, confrontés ensuite à l'état de l'art ; l'analyse des menaces produit des exigences vérifiables ; la conception puis la réalisation les mettent en œuvre ; une campagne de tests mesure ce qui tient, et tout test en échec rouvre la conception. Le chapitre 3 s'appuie sur deux méthodes d'analyse de risque : EBIOS Risk Manager [5] pour partir des valeurs à protéger et STRIDE [6] pour passer chaque flux en revue ; toutes deux servent d'outils de raisonnement, non de démarche d'homologation.

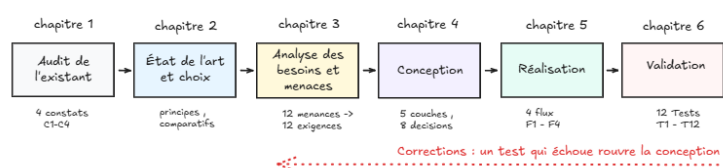


Fig. 1.3 – Démarche du projet et correspondance avec les chapitres. La flèche rouge est la boucle de correction : un test qui échoue au chapitre 6 renvoie à la conception.

Le stage a été conduit par itérations de deux semaines closes par une démonstration, avec un point hebdomadaire et un journal de décisions daté (chapitre 4) ; trois environnements Google Cloud — développement, recette (*staging*), production — sont décrits par le même code dans un dépôt unique. Le mémoire utilise des identifiants stables, rappelés au tableau 1.3 ; la règle T_n vérifie EX_n relie chaque exigence à son test.

Tab. 1.3 – Identifiants utilisés dans le mémoire.

Préfixe	Signification	Préfixe	Signification
C1–C4	Constats de l'audit de l'existant	M1–M12	Menaces retenues (STRIDE)
O1–O4	Objectifs du projet, un par constat	EX1–EX12	Exigences de sécurité vérifiables
K1–K3	Contraintes : humaine, économique, données personnelles	T1–T12	Tests ; T_n vérifie EX_n
A1–A6	Acteurs légitimes (A1–A4) et hostiles (A5, A6)	D1–D9	Décisions d'architecture datées
UC1–UC6	Cas d'utilisation du socle	É1–É5	Écarts entre conception et réalisation
BF, BNF	Besoins fonctionnels et non fonctionnels	R1–R7	Règles de détection applicatives
F1–F4	Flux : accès, provisionnement, livraison, supervision	A1–A3 (alertes)	Alertes de plateforme Cloud Monitoring
TB1–TB3	Frontières de confiance	P1–P7	Principes Zero Trust du NIST SP 800-207

Conclusion

L'audit révèle une application saine posée sur une base fragile (C1 à C4). La réponse est un socle construit sur le Zero Trust et le DevSecOps ; il vise quatre objectifs mesurables (O1 à O4) dans le respect de trois contraintes (K1 à K3), et chaque propriété qu'il annonce devra être établie par un test. Le chapitre suivant confronte ces objectifs à l'état de l'art et justifie les technologies retenues.

Chapitre 2

État de l'art et choix technologiques

Zero Trust, DevSecOps et supervision de sécurité : principes, comparaisons et technologies retenues.

Introduction

Pour chacun des quatre objectifs, des normes, des méthodes et des outils existent déjà. Ce chapitre les présente, compare les options et justifie les choix du projet selon une règle constante : chaque choix est expliqué par un *critère décisif* et par un *prix payé*, jamais par une préférence. Il traite le Zero Trust (O1), le DevSecOps (O2, O3), la supervision et la qualification des alertes (O4), puis la plateforme cloud et, technologie par technologie, les outils retenus.

2.1 L'architecture Zero Trust

Dans le modèle périmétrique (figure 2.1-A), un pare-feu sépare un extérieur hostile d'un intérieur réputé sûr, et plus rien n'est vérifié une fois la frontière franchie ; il s'affaiblit dès que les services sont dispersés dans le cloud et qu'un compte légitime peut être volé (C1). Le Zero Trust (figure 2.1-B) renverse le raisonnement : la position réseau ne prouve rien ; chaque requête passe par un point de décision qui vérifie *qui demande, ce qu'il a le droit de faire et dans quel contexte*. Formulée par Forrester [7], appliquée par Google avec BeyondCorp [8], l'idée est formalisée par le NIST SP 800-207 [3], dont les sept principes servent de grille au chapitre 6 (tableau 2.1). Le Zero Trust ne supprime ni le réseau privé ni les pare-feu ; il leur retire le rôle de *preuve suffisante*.

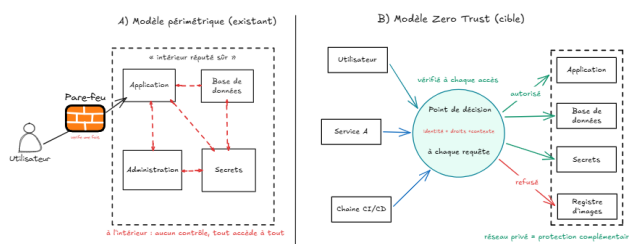


Fig. 2.1 – Du modèle périmétrique (A) au Zero Trust (B) : un point de décision vérifie chaque accès selon l'identité, les droits et le contexte ; le réseau privé reste une protection complémentaire.

Le *Zero Trust Maturity Model* de la CISA [9] complète ces principes par une grille en cinq piliers sur quatre niveaux, utilisée au chapitre 6. Trois limites sont retenues : le Zero Trust déplace la confiance vers les services d'identité et de journalisation, qui doivent eux-mêmes être protégés ; chaque contrôle coûte en complexité ; une propriété n'est acquise que si elle est vérifiée, d'où la campagne de tests du chapitre 6.

Tab. 2.1 – Les sept principes du NIST SP 800-207 et leur traduction pour le socle.

#	Principe (reformulé)	Traduction pour le socle	Obj.
P1	Toute donnée et tout service sont des ressources à protéger	Base, journaux, images, secrets et état d'infrastructure sont tous sous contrôle d'accès	O1
P2	Toute communication est sécurisée	Chiffrement et authentification ne dépendent pas d'être « à l'intérieur »	O1
P3	L'accès est accordé par session	Jetons de courte durée ; aucune clé permanente	O1, O3
P4	Politique fondée sur l'identité et le contexte	Une identité par service, droits minimaux par ressource	O1
P5	La posture est surveillée en continu	Journalisation centralisée, détection à la minute, détection de dérive	O2, O4
P6	Authentification et autorisation précèdent chaque accès	Vérification à chaque requête, pas seulement à l'entrée	O1
P7	Les observations améliorent la politique	Détection, qualification, verdicts, retour vers la conception	O4

2.2 L'approche DevSecOps

Définition — chaîne CI/CD et porte

Une chaîne d'intégration et de livraison continues est une suite d'étapes automatiques déclenchée à chaque modification du code : construction, tests, publication, déploiement. Une *porte* est une étape dont l'échec arrête la livraison.

Le DevSecOps place des contrôles de sécurité dans cette chaîne, comme le recommande le NIST SP 800-218 [10] : le contrôle devient systématique et intervient avant la production. Un contrôle bloque lorsque son critère est défini à l'avance et fiable (secret détecté, vulnérabilité critique corrigeable, test en échec) ; un contrôle bruyant reste informatif jusqu'à ce qu'un seuil sûr soit établi. Quatre familles structurent la chaîne du projet (tableau 2.2).

Tab. 2.2 – Les quatre familles de contrôles automatiques d'une chaîne DevSecOps et les outils retenus.

Famille	Question posée	Outils (retenu en gras)
Détection de secrets	Un mot de passe, une clé ou un jeton a-t-il été écrit dans le code ou son historique ?	Gitleaks [11], TruffleHog
Analyse statique (SAST)	Le code contient-il un motif vulnérable connu (injection, exécution de commande, désactivation d'une vérification) ?	Semgrep [12], CodeQL, SonarCloud
Tests automatisés	Le comportement et le contrat d'API sont-ils conformes ?	pytest , jest
Analyse des images et de l'IaC (SCA)	L'image ou la description Terraform embarque-t-elle une CVE ou une exposition au-dessus du seuil accepté ?	Trivy [13], Gripe, Checkov [14]

L'infrastructure en code (IaC) [15] décrit réseaux, bases, services et droits dans des fichiers versionnés qu'un outil applique ; elle répond au constat C2 par la reproductibilité, la revue (un changement d'infrastructure se lit comme un changement de code) et la détection de dérive. Le fichier d'état décrit toute la topologie : sa protection est une exigence à part entière (chapitre 3).

La fédération d'identité (*Workload Identity Federation*, WIF) supprime la clé de compte de service que l'on confie classiquement à la chaîne : celle-ci présente un jeton OpenID Connect signé par GitHub, que le cloud vérifie (dépôt, branche) et échange contre un accès de courte durée [16] ; il n'y a rien à voler. Nomenclature logicielle (SBOM), signature d'image et provenance SLSA [17] vont plus loin ; le projet en retient une version légère — SBOM produit par Trivy, déploiement par empreinte `sha256` — la signature restant une perspective.

2.3 Supervision et détection de sécurité

2.3.1 Du SIEM à la supervision sur entrepôt

Un SIEM (*Security Information and Event Management*) collecte et normalise des journaux, applique des règles, corrèle les signaux et offre une interface d’investigation ; les produits du marché sont facturés au volume et exigeants en exploitation, disproportionnés pour une personne et quelques dizaines d’euros (K1, K2). Le constat C4 n’exige pas un SIEM, mais la centralisation, des règles et des alertes : une alternative répandue consiste à stocker les journaux dans un **entrepôt de données** interrogeable en SQL et à écrire les règles *comme du code*, versionnées dans le même dépôt que l’infrastructure (*detection as code*, format Sigma [18]). Le tableau 2.3 compare les familles ; « entrepôt + règles en code » est retenue : pas la plus complète, mais elle satisfait K1 et K2 et garde événements, détections et vecteurs au même endroit ; ses faiblesses — ni règles fournies ni interface — sont compensées par sept règles et un tableau de bord légers (chapitre 5).

Tab. 2.3 – Comparaison qualitative des options de supervision (+ favorable, = intermédiaire, – défavorable).

Critère	SIEM commercial	SIEM cloud du fournisseur	Pile libre auto-hébergée	Entrepôt + règles en code
Coût pour un faible volume	–	=	– (serveurs)	+
Charge d’exploitation pour une personne	=	+	–	+
Règles et interface fournies au départ	+	+	=	– (à écrire)
Règles versionnées et testables	=	=	+	+
Maturité et support	+	+	=	–

2.3.2 MITRE ATT&CK et qualification assistée des alertes

Le référentiel MITRE ATT&CK [4] décrit les comportements des attaquants en *tactiques* (l’objectif : accès initial, collecte, impact) et *techniques* (le moyen : force brute T1110, exploitation d’une application exposée T1190) ; il sert de vocabulaire commun à l’analyse de risque, aux règles et à la qualification assistée.

Rapprocher chaque alerte brute d’un comportement connu est le premier facteur de fatigue d’un analyste seul ; le projet automatise cette seule étape — proposer les trois techniques les plus proches — et la décision reste humaine. Quatre approches ont été comparées : le dictionnaire de motifs (simple mais fragile aux reformulations), la similarité TF-IDF (peu coûteuse mais aveugle au sens), l’encodeur de phrases spécialisé et le grand modèle génératif (plus riche mais non déterministe). L’**encodeur de phrases spécialisé** est retenu (principe en figure B.3, annexe B) : l’architecture Sentence-BERT [19] transforme une phrase en vecteur tel que deux phrases de sens proche donnent des vecteurs proches ; les modèles généralistes connaissant mal le vocabulaire de la cybersécurité, le projet utilise **ATT&CK-BERT**, publié avec le travail SMET [21]. Le critère décisif est le *déterminisme* : une même alerte produit toujours les mêmes candidats, ce qui rend le résultat reproductible et testable. Le modèle génératif est écarté précisément parce qu’il ne l’est pas, et parce qu’un journal manipulé pourrait lui glisser des instructions qu’il tenterait d’exécuter. En contrepartie, l’encodeur ne fournit aucune explication en langage naturel.

2.4 Choix de la plateforme d'hébergement : Google Cloud

Rester sur un serveur unique, passer chez un hébergeur européen ou chez l'un des trois grands fournisseurs cloud sont des options légitimes. Les deux premières gardent l'avantage du coût à faible activité et de la réversibilité, mais n'offrent ni exécution sans serveur, ni identité machine managée, ni entrepôt analytique : ces briques seraient à construire et à exploiter, ce que K1 interdit. Les trois fournisseurs sont proches ; **Google Cloud** est retenu parce que les services dont le socle a besoin y forment un ensemble cohérent pour une personne seule (tableau 2.4), au prix d'une dépendance au fournisseur sur les briques managées. La contrainte économique est reformulée honnêtement : l'exécution descend à zéro hors trafic, mais le répartiteur, le filtrage et la base gardent un coût fixe ; l'objectif est un coût soutenable et identifié poste par poste (K2).

2.5 Technologies retenues, une par une

Chaque technologie est résumée par son critère décisif et la bonne pratique appliquée ; deux écarts avec le plan initial sont assumés : GitHub Actions remplace Cloud Build (fédération OIDC native) et Semgrep remplace SonarCloud (aucun service externe).

2.5.1 Terraform — l'infrastructure en code

Terraform [23] décrit l'infrastructure en modules paramétriques et produit un *plan* avant toute application : ce plan, relu par un humain, est la porte de la chaîne d'infrastructure. Retenu face à Pulumi, OpenTofu et aux scripts `gcloud` pour le plan et l'écosystème ; pratiques : état distant chiffré et versionné, verrous de destruction, un répertoire par environnement.

Tab. 2.4 – Services Google Cloud mobilisés, rôle dans le socle et bonne pratique appliquée.

Service	Rôle dans le socle	Bonne pratique appliquée
Cloud Load Balancing + Cloud Armor [30]	Point d'entrée HTTPS unique, certificat géré, WAF (règles OWASP), limitation de débit	Entrée Cloud Run réservée au répartiteur ; verdicts WAF journalisés
Cloud Run (services et jobs) [27]	Exécution des conteneurs : ELSON, API, tableau de bord, encodeur, jobs de détection et d'enrichissement	Une identité par service ; image épinglée par empreinte ; entrée réservée au répartiteur ; réseau privé par connecteur d'accès VPC
Cloud SQL PostgreSQL [28]	Données applicatives et comptes du tableau de bord	Adresse privée uniquement (appairage de services privés) ; TLS obligatoire ; PITR ; une base et un utilisateur par application
BigQuery [22]	Entrepôt de sécurité : journaux, détections, vecteurs ATT&CK	Tables en ajout seul ; un auteur par table ; jeu de données chiffré par une clé KMS du socle
Cloud Logging, Log Router [29]	Collecte des journaux du WAF, des services et de l'audit ; puits en flux vers BigQuery	Filtrage à la source ; un seul auteur pour la table des journaux bruts
Cloud Scheduler, Cloud Monitoring [31]	Cadence du job de détection (chaque minute) ; alertes de plateforme ; SLO	Alerte d'absence de journaux ; budget
IAM + Workload Identity Federation [16]	Identités, droits par ressource, chaîne CI/CD sans clé	Zéro clé de compte de service ; fédération restreinte au dépôt et à la branche
Secret Manager, Cloud KMS	Secrets par usage, clés de chiffrement (état Terraform, graines TOTP)	Lecture par référence ; accès journalisés
Artifact Registry, Cloud Storage	Registre d'images ; état Terraform chiffré et versionné	Image scannée avant publication ; état accessible au seul administrateur

2.5.2 GitHub Actions et fédération OIDC — la chaîne de livraison

GitHub Actions [24] exécute la chaîne à chaque demande de fusion et à chaque fusion sur main, sans serveur à gérer ; sa fédération OIDC native vers Google Cloud rend possible une chaîne *sans aucune clé*. Pratiques : permissions minimales par job (`id-token` aux seuls jobs de publication et de déploiement), actions épinglées au SHA de commit, ordre des portes imposé par dépendances explicites.

2.5.3 Gitleaks, Semgrep, pytest/jest, Trivy — les quatre portes

Gitleaks cherche les secrets dans le code *et l'historique complet* : un secret ajouté puis retiré reste dans les objets Git. Semgrep exécute un jeu de règles versionné en mode *diff-aware* : seuls les nouveaux constats bloquent. pytest (API) et jest (tableau de bord) conditionnent la construction. Trivy analyse l'image et bloque sur toute CVE *critique corrigeable* (`-ignore-unfixed`) : une vulnérabilité sans correctif est signalée sans arrêter la chaîne ; il produit aussi le SBOM et un rapport chargé dans l'entrepôt — la chaîne nourrit la supervision.

2.5.4 Docker multi-étape et Cloud Run — l'exécution

L'image finale ne contient que l'exécutable et ses dépendances, tourne sous un utilisateur non privilégié et n'embarque pas de gestionnaire de paquets ; elle transite entre étapes en artefact interne au run, si bien que publier une image non scannée est *techniquement* impossible. Cloud Run l'exécute sans serveur à administrer, avec mise à l'échelle à zéro ; le prix payé est le démarrage à froid (chapitre 6).

2.5.5 PostgreSQL sur Cloud SQL — les données transactionnelles

PostgreSQL 15, déjà utilisé par ELSON, est conservé sur Cloud SQL pour la continuité et le support managé (sauvegardes, PITR). Pratiques : aucune adresse publique, TLS obligatoire, rôle Client Cloud SQL par identité de service, une base et un utilisateur par application (`elson_db`, `menal_db`), rôle super-utilisateur retiré aux applications, contrôle d'isolation quotidien.

2.5.6 FastAPI — l'API de supervision

FastAPI [33] (Python) porte l'API SIEM : validation typée des entrées, contrat OpenAPI généré, écosystème BigQuery natif ; retenu face à Flask et Django REST. Pratiques : un seul composant de vérification du jeton sur chaque route, lecture seule sur l'entrepôt, verdicts en ajout seul, limitation de débit par identifiant.

2.5.7 Next.js — le tableau de bord

Next.js [34] (React, *App Router*) porte le tableau de bord MENAL Sentinel. Critère décisif : le rendu côté serveur — le navigateur n'appelle jamais l'entrepôt, seule l'API le fait, et le jeton reste dans un cookie `httpOnly`. Pratiques : rafraîchissement toutes les dix secondes, aucune donnée en cache, bandeau d'état honnête (« Données à jour » / « API injoignable »).

2.5.8 Authentification à deux facteurs — TOTP

L'administrateur est le seul humain à accéder au socle ; son compte est la cible la plus rentable, et le second facteur retenu est le code temporel à usage unique (TOTP, RFC 6238 [32]), généré

par une application d'authentification à partir d'une graine partagée à l'enrôlement par QR code. Retenu face au SMS (interceptable) et aux clés FIDO2 (matériel à acheter et à gérer) pour son coût nul et son absence de dépendance externe. La conception va plus loin : le mot de passe donne un jeton intermédiaire *sans aucun droit*, que seul le code TOTP convertit en jeton d'accès (chapitre 4).

2.5.9 BigQuery, Log Router et Cloud Scheduler — la chaîne de détection

BigQuery [22] est à la fois l'entrepôt des journaux, le moteur des sept règles SQL et l'index vectoriel de la qualification assistée (VECTOR_SEARCH) : un seul moteur pour trois usages. Les journaux ne sont pas exportés par lots : le routeur de journaux (*Log Router*) écrit chaque entrée retenue dans BigQuery par un puits en flux, et Cloud Scheduler déclenche *chaque minute* le job de détection [31] — la raison du délai attaque → incident inférieur à trois minutes (chapitre 4).

2.5.10 ATT&CK-BERT et ONNX — la qualification assistée

Le modèle ATT&CK-BERT (768 dimensions) est exporté au format ONNX en précision flottante simple et exécuté dans un service Cloud Run interne, sans cadre d'apprentissage ni sortie vers Internet ; une version quantifiée sur huit bits, quatre fois plus légère, a été rejetée sur un critère fixé à l'avance (chapitre 6). Le référentiel encodé est la matrice Enterprise v17.1 (872 vecteurs).

2.6 Référentiels mobilisés et positionnement

Le projet s'aligne sur plusieurs référentiels sans se déclarer conforme à aucun : NIST SP 800-207 et CISA ZTMM 2.0 (Zero Trust), NIST SP 800-218 (chaîne), OWASP Top 10 [25] (risques couverts par le WAF), CIS Google Cloud Benchmark [26] (configuration), MITRE ATT&CK (vocabulaire), EBIOS RM et STRIDE (risque), RGPD et loi n° 2017-020 (données personnelles). Son apport tient à trois questions laissées ouvertes : l'*échelle* (Zero Trust et DevSecOps appliqués et prouvés par une personne avec quelques dizaines d'euros), la *supervision sur entrepôt généraliste* en quasi temps réel à coût connu, et la *qualification d'alertes courtes* par un encodeur spécialisé.

Conclusion

L'état de l'art conduit à une architecture limitée mais cohérente : l'identité comme critère d'autorisation, quatre portes et l'infrastructure en code, une supervision sur entrepôt en flux et une qualification assistée déterministe, dont le chapitre suivant fait un modèle vérifiable.

Chapitre 3

Analyse des besoins et modélisation des menaces

De la valeur à protéger à l'exigence de sécurité vérifiable.

Introduction

Ce chapitre transforme le cadre des chapitres 1 et 2 en un modèle vérifiable : acteurs, cas d'utilisation et besoins ; flux de données et frontières de confiance ; menaces analysées flux par flux avec STRIDE, en partant des valeurs métier à la manière d'EBIOS Risk Manager. Il en résulte douze exigences, chacune rattachée à la menace qui la justifie, au contrôle qui la met en œuvre et au test qui la vérifiera au chapitre 6.

3.1 Acteurs et cas d'utilisation

Le modèle sépare les acteurs légitimes, porteurs des usages attendus, des acteurs hostiles, porteurs des objectifs adverses (tableau 3.1) ; la frontière n'est pas absolue, puisqu'un compte volé ou un participant qui abuse d'une fonction autorisée devient le point de départ d'un scénario hostile. Deux remarques orientent la conception : A4 n'est pas un administrateur (la chaîne ne reçoit que les droits de publier une image et de déployer un service) ; A6 est le plus difficile à traiter, car le filtrage au point d'entrée ne le distingue pas d'un usage légitime.

Tab. 3.1 – Les six acteurs du modèle.

Acteur	Nature	Rôle ou motivation	Moyens
A1 — Utilisateur final	légitime	Participe à ELSON ou à une application accueillie ensuite	Navigateur, point d'entrée public
A2 — Administrateur	légitime	Exploite le socle : applique les plans, consulte et qualifie les alertes, accueille les applications	Console cloud, dépôt Git, mot de passe + second facteur
A3 — Développeur	légitime	Modifie le code des applications ou la description de l'infrastructure	Dépôt Git, demandes de fusion
A4 — Chaîne CI/CD	légitime (machine)	Construit, analyse et déploie à chaque modification	Identité fédérée, sans clé stockée
A5 — Attaquant externe	hostile	Opportuniste (robots) ou ciblé (intérêt pour le corpus)	Outils automatisés, vulnérabilités publiques
A6 — Compte détourné	hostile	Participant fraudeur, session ou clé volée, dépendance compromise	Droits légitimes existants

Six cas d'utilisation structurent le socle (figure 3.1) : **UC1** utiliser l'application hébergée (A1, F1) ; **UC2** livrer une version — la chaîne construit, contrôle et déploie (A3 exécuté par A4, F3) ; **UC3** modifier l'infrastructure — proposition en code, revue et application du plan par l'administrateur (A3, A2, F2) ; **UC4** consulter alertes et indicateurs (A2, F4) ; **UC5** qualifier une alerte — techniques proposées, décision, verdict (A2, F4) ; **UC6** accueillir une nouvelle application par paramétrage (A2, F2 et F3). UC6 porte la réutilisabilité annoncée au chapitre 1 ; la détection automatique est un comportement du système, non un cas d'utilisation.

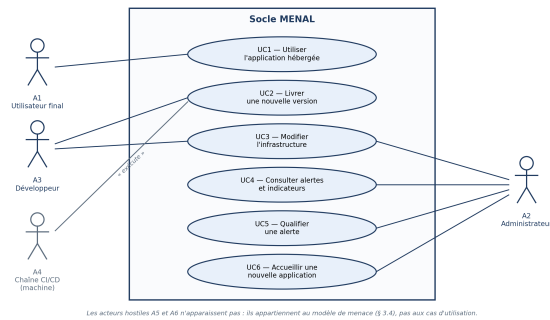


Fig. 3.1 – Diagramme des cas d'utilisation du socle. A4 exécute la livraison déclenchée par A3 ; les acteurs hostiles appartiennent au modèle de menace, non aux cas d'utilisation.

3.2 Spécification des besoins

Les besoins traduisent constats et cas d'utilisation en propriétés attendues, avec une cible mesurable (tableau 3.2) ; la description en code ne couvre pas la totalité des ressources (exceptions au chapitre 5) et la qualification assistée propose des candidats, non une classification certaine.

Tab. 3.2 – Besoins fonctionnels (BF) et non fonctionnels (BNF) du socle.

#	Besoin	Cible ou précision	Origine
BF1	Exposer les applications par un point d'entrée unique, chiffré, filtré et journalisé	Aucun autre chemin d'entrée public	C1, O1
BF2	Une identité distincte par service et par personne, avec les seuls droits nécessaires	Aucune identité partagée	C1, O1
BF3	Décrire l'infrastructure durable en code et détecter toute modification hors code	Exceptions recensées	C2, O2
BF4	Livrer par une chaîne automatique dont les contrôles critiques sont bloquants	Secrets, code, tests, image	C3, O3
BF5	Centraliser les journaux et exécuter des règles de détection versionnées	Incident affiché ≤ 3 min après l'attaque	C4, O4
BF6	Proposer pour chaque alerte des techniques ATT&CK candidates avec un score	Trois candidats ; la bonne technique parmi eux dans 80 % des cas	O4
BF7	Corréler les détections par entité, présenter alertes et indicateurs, enregistrer le verdict	Verdict conservé sans modifier la preuve	O4, UC5
BF8	Accueillir une nouvelle application par paramétrage	Aucun module modifié	UC6
BNF1	Disponibilité	≥ 99 % par mois (point d'entrée, applications)	K1
BNF2	Latence	95 % des requêtes en moins d'une seconde	UC1
BNF3	Coût	≤ 50 €/mois, poste dominant identifié	K2
BNF4	Exploitabilité	Procédures écrites (restauration, retour arrière, accueil) exécutables par une personne	K1
BNF5	Reproductibilité	Reconstruction d'un environnement depuis le dépôt en moins d'une heure	O2
BNF6	Auditabilité	Toute action privilégiée attribuable à une identité et datée	K3, O1
BNF7	Résilience	RTO < 1 h ; RPO < 5 min	K3

3.3 Flux de données et frontières de confiance

Le socle met en œuvre quatre flux, un par objectif : **F1 — accès applicatif** (O1) : la requête d'un utilisateur traverse le point d'entrée (répartiteur HTTPS et WAF), atteint le service Cloud Run, qui interroge la base privée ; **F2 — provisionnement** (O2) : un changement Terraform est contrôlé par la chaîne, puis revu et appliqué par l'administrateur ; **F3 — livraison** (O3) : un changement de code est analysé, testé, construit, scanné, publié et déployé par la chaîne ; **F4 — supervision** (O4) : les journaux du point d'entrée, des services et du plan de contrôle sont acheminés vers l'entrepôt, les règles produisent des détections, la qualification les enrichit, l'administrateur enregistre ses verdicts. La session administrateur emprunte le chemin de l'utilisateur (F1, TB1).

Définition — frontière de confiance

Ligne du diagramme de flux dont le franchissement fait changer une donnée ou une requête de niveau de contrôle ou d'autorité. Elle ne suppose pas nécessairement un pare-feu : elle signale qu'un contrôle d'autorisation doit s'exercer.

La figure 3.2 situe les quatre flux et trois frontières : **TB1** Internet → point d'entrée (F1 ; requêtes malveillantes, force brute, déni de service ; TLS, WAF, limitation de débit) ; **TB2** services → données privées (F1 ; accès direct à la base, service lisant les données d'un autre ; adresse privée, identité par service) ; **TB3** humains et automates → plan de contrôle cloud — IAM, secrets, registre, état, entrepôt (F2, F3, F4 ; usurpation, droits excessifs, altération ; identités distinctes, droits minimaux, jetons courts, fédération, journaux d'audit). TB3 est déterminante : administration, livraison et supervision passent par les interfaces du fournisseur, joignables de partout, qu'aucune règle de pare-feu ne protège — *seule l'identité décide*.

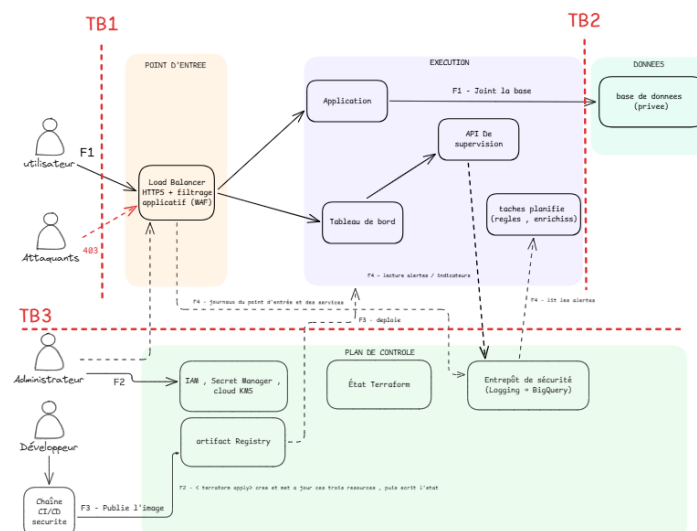


Fig. 3.2 – Flux de données et frontières de confiance. TB1 et TB2 sont franchies par un chemin réseau ; TB3 par des appels aux services managés, où l'autorisation dépend d'abord de l'identité présentée.

3.4 Analyse du risque

À la manière d'EBIOS RM [5], l'analyse part de quatre **valeurs métier** : le corpus (dont la copie détruirait l'exclusivité), les données des participants, la disponibilité du service pendant la

compétition et les preuves de supervision (sans journaux fiables, un incident ne peut plus être établi), portées par des biens supports — point d’entrée, services, base, entrepôt, registre, secrets, chaîne, état, identités. Les sources de risque sont A5 et A6.

STRIDE [6] passe en revue six catégories sur chaque flux — usurpation, altération, répudiation, divulgation, déni de service, élévation de privilège — et l’on retient les combinaisons réalistes pour A5 ou A6 qui touchent une valeur métier : douze menaces (tableau 3.3), cotées en gravité et vraisemblance sur trois niveaux, le risque étant leur produit. Huit menaces sont de niveau élevé, trois moyen, une faible (matrice en annexe B.1). Résultat contre-intuitif : les chaînes automatisées (F2, F3) concentrent autant de menaces élevées que l’entrée Internet (F1), ce qui confirme le choix de traiter TB3 par l’identité. M12 est cotée faible mais traitée, car le contrôle est peu coûteux et protège la valeur dont dépend la vérification de toutes les autres. Trois menaces ne sont volontairement pas retenues : la fraude métier (hors périmètre), la panne du fournisseur (risque accepté) et l’adversaire étatique.

Tab. 3.3 – Les douze menaces retenues, par flux et catégorie STRIDE, avec leur cotation (gravité $\times$ vraisemblance).

#	Flux	STRIDE	Menace	Valeur touchée	$G \times V =$ risque
M1	F1	T	Requête malveillante vers l’application (SQLi, XSS, traversée de chemin, LFI)	corpus, données	$3 \times 3 = 9$ élevé
M2	F1	D	Force brute sur l’authentification, rafale saturant le service	disponibilité	$2 \times 3 = 6$ élevé
M3	F1	I	Connexion directe à la base depuis Internet	corpus, données	$3 \times 2 = 6$ élevé
M4	F1	E	Un service accède aux données ou secrets d’un autre (droits trop larges)	corpus, données	$3 \times 2 = 6$ élevé
M5	F1	S	Vol ou rejeu d’une session administrateur	preuves, données	$3 \times 1 = 3$ moyen
M6	F2	R	Modification d’infrastructure à la main, hors code, sans trace (dérive)	disponibilité, preuves	$2 \times 3 = 6$ élevé
M7	F2	I	Lecture ou altération de l’état Terraform	corpus, données	$3 \times 1 = 3$ moyen
M8	F3	I	Secret publié dans le dépôt ou son historique	corpus, données	$3 \times 2 = 6$ élevé
M9	F3	T	Déploiement d’un code ou d’une image vulnérable	corpus, disponibilité	$2 \times 3 = 6$ élevé
M10	F3	S	Vol de la clé d’authentification de la chaîne CI/CD	corpus, données	$3 \times 2 = 6$ élevé
M11	F4	R	Attaque non détectée : journaux non collectés ou collecte interrompue	preuves	$2 \times 2 = 4$ moyen
M12	F4	T	Altération ou suppression des preuves par un composant compromis	preuves	$2 \times 1 = 2$ faible

3.5 Exigences de sécurité et traçabilité

Chaque menace est traduite en une **exigence** vérifiable par un test (tableau 3.4) ; l’exigence dit le résultat attendu sans figer la technologie, qui appartient au chapitre 4. Une règle unique s’applique : la menace M_n donne l’exigence EX_n , mise en œuvre par un contrôle et éprouvée par le test T_n . Cette relation un-pour-un ne garantit pas que les douze tests seront conformes ; elle

garantit que chaque propriété annoncée possède un moyen explicite d’être confirmée, nuancée ou réfutée (chaîne de traçabilité en annexe B.1, figure B.2).

Tab. 3.4 – Matrice de traçabilité : exigence, critère de vérification, contrôle prévu, test et référentiel.

Exig.	Exigence et critère de vérification	Contrôle prévu	Test / référentiel
EX1	Toute requête publique passe par le WAF ; les charges d’attaque des familles activées sont refusées avant l’application	LB HTTPS + Cloud Armor, règles gérées (SQLi, XSS, LFI, RFI, anomalies de protocole)	T1 — OWASP A03, A05
EX2	Tentatives répétées et rafales limitées ; au-delà du seuil, refus de débit	Limitation de débit au point d’entrée + compteur applicatif par identifiant	T2 — OWASP A07 ; NIST P6
EX3	La base n’a aucune adresse publique ; une connexion depuis Internet échoue	Cloud SQL en adresse privée (appairage), accès par le connecteur VPC, TLS obligatoire	T3 — CIS GCP 6.x ; NIST P1
EX4	Chaque service possède sa propre identité et n’accède qu’à son périmètre	Un compte de service par service, droits IAM par ressource, un secret par usage	T4 — NIST P4 ; CIS GCP 1.x
EX5	Accès administrateur à deux facteurs ; jetons courts et sans droit avant le second facteur	Jeton intermédiaire (5 min, sans rôle) puis jeton d’accès (60 min) après code TOTP	T5 — NIST P3, P6
EX6	L’infrastructure durable est en code ; une modification hors code est détectable	Terraform seule voie d’écriture ; plan de dérive ; alerte de plateforme A2	T6 — NIST P5 ; CIS GCP
EX7	L’état est stocké à distance, chiffré, versionné, accessible aux seules identités autorisées	Cloud Storage chiffré KMS, versionné, droits restreints à l’administrateur	T7 — NIST P1 ; CIS GCP 5.x
EX8	Un secret dans le code ou l’historique bloque la chaîne avant toute construction	Gitleaks en première porte	T8 — OWASP A02 ; SP 800-218
EX9	Le déploiement est bloqué sur code dangereux, test en échec ou CVE critique corrigé	Semgrep, pytest/jest et Trivy en portes bloquantes	T9 — OWASP A06 ; SP 800-218
EX10	La chaîne s’authentifie par fédération ; aucune clé de compte de service n’existe	WIF restreinte au dépôt et à la branche ; inventaire des clés vide	T10 — NIST P3 ; CIS GCP 1.x
EX11	Journaux centralisés en flux ; règles actives ; interruption de collecte alertée	Puits Log Router → BigQuery ; job de détection à la minute ; alerte d’absence A3	T11 — NIST P5, P7 ; CIS GCP 2.x
EX12	Les composants d’analyse n’écrivent pas dans les tables de preuves ; verdicts en table séparée, ajout seul	Droits d’écriture par table ; table de verdicts append-only	T12 — NIST P1 ; CIS GCP 2.x

Conclusion

Les constats sont devenus un modèle vérifiable : six acteurs, six cas d’utilisation, quinze besoins, quatre flux, trois frontières, douze menaces cotées et douze exigences appariées à douze tests. Deux enseignements guident la conception : la moitié des menaces élevées porte sur les chaînes automatisées et le plan de contrôle, où l’autorisation repose sur l’identité ; la réutilisabilité (BF8, UC6) impose une architecture paramétrable.

Chapitre 4

Conception de l'architecture

Environnements, déploiement, réseau, identités, session à deux facteurs, données, chaîne de détection en flux et décisions datées.

Introduction

Le chapitre 3 a produit douze exigences vérifiables. Ce chapitre conçoit l'architecture qui les réalise. Il pose quatre principes directeurs, présente les trois environnements et le diagramme de déploiement, puis découpe le socle en cinq couches. Cinq sujets sont détaillés parce qu'ils portent l'essentiel de la sécurité : le point d'entrée et le réseau, les identités et les secrets, la session administrateur à deux facteurs, les données, et la chaîne de détection en flux. Le chapitre se termine par le registre des décisions, chacune datée, justifiée et associée à une preuve.

4.1 Les quatre principes directeurs

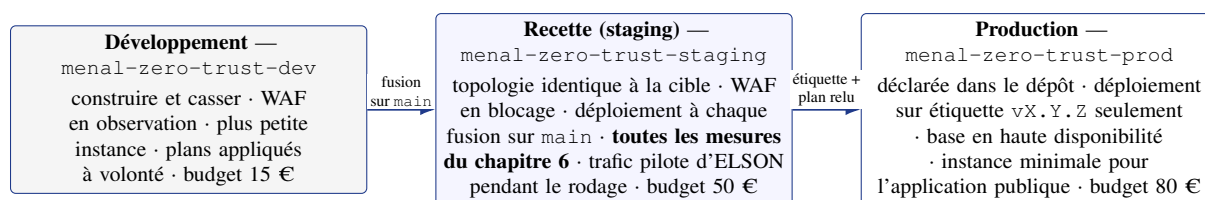
- « **L'identité décide.** » Chaque composant — service, tâche, chaîne, personne — reçoit sa propre identité ; les droits suivent l'identité, jamais l'emplacement réseau, et se limitent au nécessaire (moindre privilège). Porte EX4, EX5, EX10, EX12.
- « **Tout est en code.** » L'infrastructure durable est décrite dans le dépôt Git et créée par Terraform ; toute modification à la main devient une dérive détectable. Porte EX6, EX7 ; ses exceptions sont nommées au chapitre 5.
- « **Refus par défaut.** » Réseau, identités et sorties sont fermés ; chaque ouverture est nominative, justifiée et journalisée. Porte EX1, EX3, complète EX4.
- « **Soutenable avant tout.** » Un composant qui exigerait une exploitation permanente est écarté, même s'il renforçait la sécurité théorique (K1, K2). Ce principe explique les composants volontairement absents (section 4.10).

4.2 Vue d'ensemble : trois environnements, un déploiement

4.2.1 Trois environnements décrits par le même code

Le socle suit le modèle classique des trois environnements (figure 4.1). Un environnement est un *projet* Google Cloud distinct, un répertoire `envs/<env>` et un fichier d'état Terraform séparé ; les mêmes modules servent partout, seules les variables changent. La séparation par projet rend l'isolation structurelle : aucune ressource d'un environnement n'est adressable depuis l'autre, et une configuration affaiblie en développement ne peut pas être promue par inadvertance — elle n'existe que dans le fichier de variables de `dev`.

Le choix de répertoires plutôt que d'espaces de travail Terraform tient à la lisibilité : état, variables et droits sont visibles dans l'arbre du dépôt, et une application lancée dans `envs/staging` ne peut pas atteindre la production par une erreur de sélection. Les variables



Chemin de promotion : demande de fusion → portes → fusion → recette (déploiement automatique F3, plan et application manuels F2) → validation T1-T12 → étiquette → production.

Fig. 4.1 – Les trois environnements du socle et leur chemin de promotion. Un projet Google Cloud, un répertoire et un état Terraform par environnement ; le code est identique.

qui diffèrent sont peu nombreuses (annexe A) : projet, taille et haute disponibilité de l'instance SQL, instances minimales, mode et seuils du WAF, rétention des journaux, budget. Tout le reste est commun : c'est ce qui rend la recette représentative et la promotion une simple application du plan.

4.2.2 Le diagramme de déploiement

La figure 4.2 montre *ce qui est déployé sur quoi et par quel chemin*. Elle se lit en quatre zones. **Le point d'entrée** : une seule adresse publique, celle du répartiteur HTTPS, qui termine TLS avec un certificat géré, applique le WAF et la limitation de débit de Cloud Armor et transmet aux services ; utilisateurs et administrateur y passent sans exception. **L'exécution** : cinq services Cloud Run — le site et l'API d'ELSON, l'API de supervision, le tableau de bord, l'encodeur — et trois jobs (détection, enrichissement, contrôle d'isolation des bases), chacun sous sa propre identité ; les quatre services publics n'acceptent que le trafic du répartiteur, l'encodeur n'est joignable que depuis le réseau interne avec authentification. **Le réseau privé et les données** : un connecteur d'accès VPC relie les services au réseau privé, où la base Cloud SQL n'est joignable que par son adresse privée ; l'entrepôt BigQuery est chiffré par une clé du socle. **Les services managés** : journalisation et acheminement en flux (Cloud Logging, Log Router), planification (Cloud Scheduler), identités et fédération (IAM, WIF), secrets et clés (Secret Manager, KMS), registre d'images et état Terraform, alertes (Cloud Monitoring). Ces services sont appelés par les API du fournisseur : aucune règle réseau du socle ne s'applique à ces appels, l'autorisation repose sur le jeton présenté (frontière TB3).

Trois lectures s'en dégagent : **un seul chemin entrant** (utilisateur et administrateur passent par la même liaison HTTPS) ; **un seul chemin vers les données** (la liaison PostgreSQL part des services vers une adresse privée et ne traverse jamais Internet) ; **des artefacts identifiés, pas seulement nommés** (chaque service exécute une image désignée par son empreinte sha256, celle qui a passé les portes, sous une identité propre ; les poids du modèle sont embarqués dans l'image de l'encodeur et vérifiés par empreinte, d'où l'absence de toute sortie Internet pour ce service).

4.3 Le modèle en cinq couches

L'architecture se décompose en cinq couches (tableau 4.1). Les trois premières suivent le chemin d'une requête utilisateur (F1) ; les deux dernières sont transverses. Chaque couche porte un sous-ensemble des douze exigences, ce qui localise sans ambiguïté où chacune est réalisée.

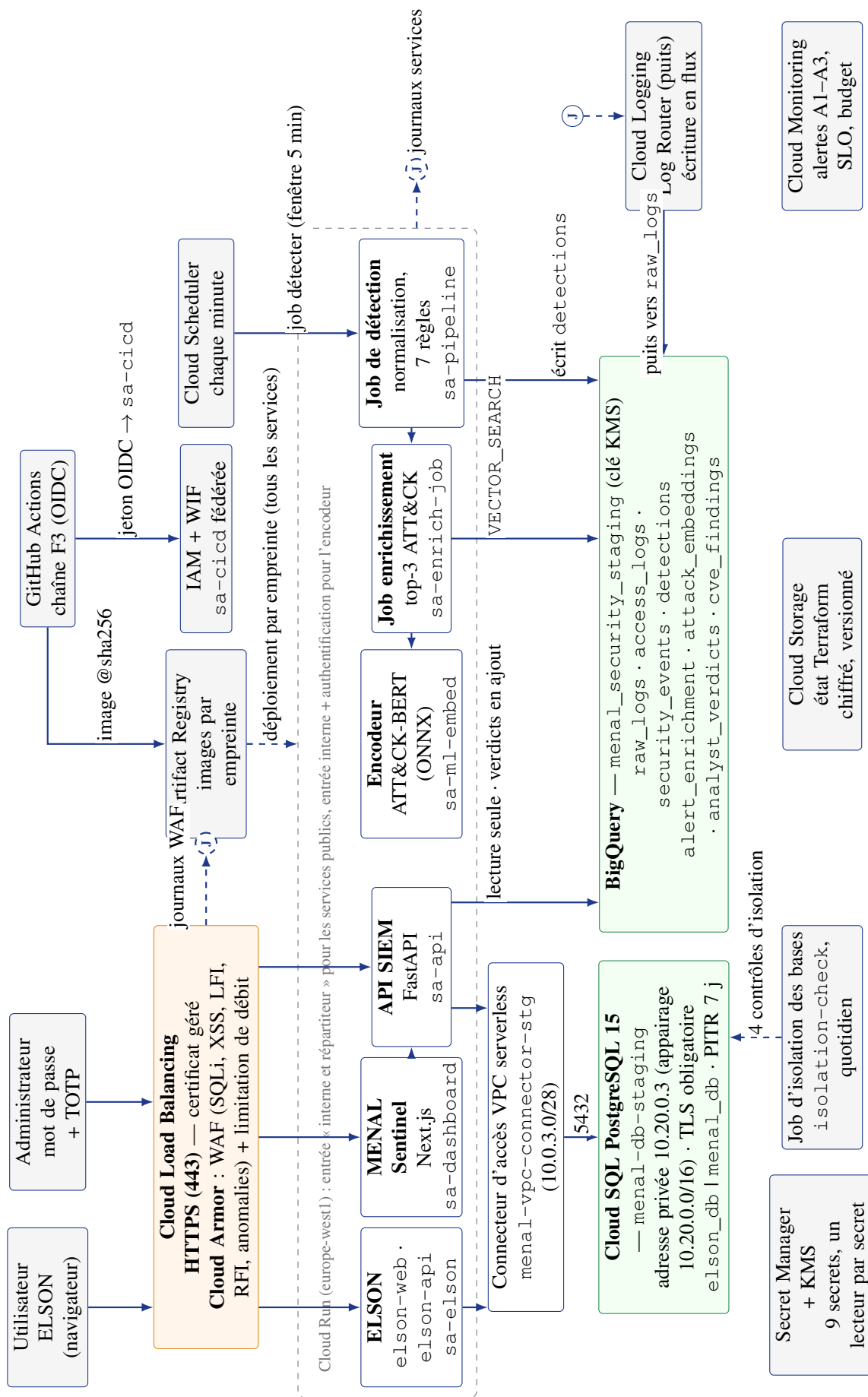


Fig. 4.2 – Diagramme de déploiement du socle (environnement de recette, relevé du 08/09/2026). Un seul chemin entrant (443 vers le répartiteur), un seul chemin vers la base (connecteur d'accès VPC puis adresse privée 10.20.0.3 sur le port 5432), une journalisation en flux (journaux du WAF et des services, repère J, acheminés par le puits vers BigQuery) et des liaisons vers les services managés dont l'autorisation dépend de l'identité, non du réseau. Les flèches horizontales de la zone Cloud Run représentent : tableau de bord → API (jeton), job de détection → enrichissement (déclenchement), enrichissement → encodeur (encodage).

Tab. 4.1 – Les cinq couches du socle et les exigences qu’elles portent.

Couche	Rôle	Services et outils	Exigences
1 — Point d’entrée	Entrée unique, TLS, WAF, limitation de débit	Cloud Load Balancing, certificat géré, Cloud Armor	EX1, EX2
2 — Exécution	Exécuter applications et composants sans serveur ; une identité par service ; échelle à zéro ; accès au réseau privé par le connecteur VPC	Cloud Run (cinq services, trois jobs), connecteur d’accès VPC, images par empreinte	EX4
3 — Données	Base privée par application ; entrepôt de sécurité ; tables de preuves protégées ; PITR	Cloud SQL PostgreSQL, BigQuery	EX3, EX12
4 — Identité et secrets	Comptes de service, fédération, secrets par usage, clés, session à deux facteurs	IAM, WIF, Secret Manager, KMS, JWT + TOTP	EX4, EX5, EX7, EX10
5 — Automatisation et supervision	IaC, chaîne à portes, collecte en flux, règles, enrichissement, tableau de bord	Terraform, GitHub Actions, Gitleaks, Semgrep, Trivy, Log Router, Scheduler, SQL, FastAPI, Next.js	EX6, EX8, EX9, EX11

4.4 Le point d’entrée et le réseau

Le point d’entrée regroupe trois mécanismes. Le *chiffrement* : terminaison TLS avec certificat renouvelé automatiquement, redirection de HTTP vers HTTPS, trafic chiffré jusqu’aux services. Le *filtrage applicatif* : Cloud Armor [30] applique les règles gérées dérivées du référentiel OWASP — injection SQL, script inter-sites, inclusion de fichier locale et distante, et, depuis la correction du test T1 (chapitre 6), les anomalies de protocole qui couvrent les traversées de chemin encodées ; une requête qui déclenche l’une d’elles reçoit un refus 403 avant d’atteindre l’application, et le refus est journalisé avec la règle qui l’a déclenché (EX1). La *limitation de débit* : une règle limite le nombre de requêtes par adresse source sur une fenêtre glissante (60 par minute sur les chemins d’authentification) puis bannit temporairement l’adresse ; dans l’API, un second niveau compte les tentatives par identifiant — la onzième tentative de mot de passe erroné en quinze minutes reçoit un refus 429 quelle que soit l’adresse (EX2). Pour que ce point d’entrée soit réellement unique, l’entrée des services Cloud Run est réglée sur « interne et répartiteur » : l’adresse directe attribuée à chaque service n’accepte plus les connexions venant d’Internet ; sans ce réglage, le WAF pourrait être contourné.

Le réseau est volontairement réduit à ce qui porte une propriété de sécurité : en conteneurs sans serveur, un sous-réseau par application n’apporterait aucun contrôle que l’identité ne fournit déjà. Il compte donc quatre éléments (relevés du 08/09/2026, annexe D) : un connecteur d’accès VPC serverless (`menal-vpc-connector-stg`, plage 10.0.3.0/28, région europe-west1) par lequel les services Cloud Run atteignent le réseau privé ; une plage d’appairage de services privés (`google-services-staging`, 10.20.0.0/16) dans laquelle Cloud SQL reçoit son adresse privée 10.20.0.3 (EX3) ; l’accès privé Google, qui achemine les appels à BigQuery, Secret Manager et Logging sans passer par Internet ; un pare-feu en *refus par défaut en sortie*, journalisé, avec deux autorisations nominatives appliquées aux instances du connecteur — vers 10.20.0.0/16 sur le port 5432 et vers les plages des API Google sur le port 443. Une seule adresse publique existe, celle du répartiteur. La séparation entre applications hébergées est portée par l’identité, par une base et un utilisateur par application et par des secrets propres, non par le réseau.

4.5 Les identités et les secrets

Puisque l'identité décide, la matrice des identités est le principal instrument de cloisonnement du socle. **Neuf identités** existent (tableau 4.2, inventaire IAM du 08/09/2026) : une personne et huit comptes de service — sept créés pour le socle, un par composant, et le compte par défaut de Compute Engine, créé par la plateforme. Chaque compte dédié ne possède que les droits nécessaires à sa fonction, accordés au niveau du projet pour les rôles techniques (exécution de requêtes, client Cloud SQL, écriture de journaux) et au niveau de la ressource pour les données (jeu de données, table, secret) ; ce sont les *interdictions* de la matrice qui constituent la segmentation. L'administrateur, seul humain de l'entreprise sur le projet, en détient le rôle de propriétaire ; son compte est protégé par la validation en deux étapes et chacune de ses actions est un événement d'audit tracé et lu par les alertes de plateforme.

Une règle organise la matrice : **une table n'a qu'un auteur, et l'auteur d'une table de preuves n'est jamais le composant qui s'en sert**. Les journaux bruts sont écrits par l'agent de journalisation du puits ; les journaux normalisés et les détections par le job de détection ; les candidats par le job d'enrichissement ; les CVE par la chaîne de livraison. La table des verdicts fait seule exception : l'API en est l'auteur et le lecteur, mais elle n'y écrit qu'en ajout, au nom de l'administrateur authentifié. Un composant compromis ne peut ainsi effacer la trace de l'intrusion qu'en détenant également l'identité qui écrit la table visée (M12).

Un secret par usage, un lecteur par secret. L'audit avait révélé qu'un même secret protégeait plusieurs mécanismes (C1). Le socle applique la règle inverse : neuf secrets dans Secret Manager, chacun lu par référence et par une seule identité (relevé du 08/09/2026, annexe D) — cinq pour ELSON (mot de passe de base, clé d'API, signature des jetons, poivre des codes OTP, URL signée des fichiers audio), trois pour l'API de supervision (mot de passe de base, clé de signature des jetons, clé de chiffrement des graines TOTP) et un pour le tableau de bord. Huit sont chiffrés par une clé Cloud KMS du socle (`menal-secrets-key-staging`) ; une seconde clé (`menal-api-key-staging`) chiffre le jeu de données de l'entrepôt. Les accès aux secrets sont journalisés ; la fuite d'un secret ne compromet plus l'ensemble.

La chaîne sans clé. GitHub Actions s'authentifie par fédération : elle présente un jeton signé par GitHub, que le fournisseur vérifie avant d'accorder un accès de courte durée sous l'identité `sa-cicd`. La fédération est restreinte au dépôt et à la branche principale — sans cette condition, n'importe quelle branche obtiendrait l'identité de déploiement. La condition est déclarée côté fournisseur, hors du dépôt : la décision d'autorisation appartient au vérificateur, jamais à l'émetteur. Aucune clé n'existe pour `sa-cicd`, et cette absence est vérifiable par inventaire (EX10).

4.6 La session administrateur à deux facteurs (MFA)

L'accès au tableau de bord est le seul accès humain au socle ; il est conçu avec soin (figure 4.3). La connexion se déroule en deux temps. Le **mot de passe** (haché par `bcrypt` dans `menal_db`) donne un *jeton intermédiaire* valable cinq minutes qui **ne porte aucun rôle** : toute route métier le refuse (403). Le **code TOTP** à six chiffres, produit par l'application d'authentification de l'administrateur, transforme ce jeton en *jeton d'accès* de soixante minutes, seul porteur du rôle.

Tab. 4.2 – Matrice des identités : les interdictions sont la segmentation (inventaire IAM du 08/09/2026 : sept comptes de service dédiés sans aucune clé, plus le compte Compute par défaut).

Identité	Peut	Ne peut pas
Administrateur (A2, humain, propriétaire du projet)	Appliquer les plans Terraform ; lire et écrire l'état ; lire l'entrepôt ; écrire des verdicts ; accueillir une application	Agir sans second facteur ; agir sans trace (BNF6) : chaque action est un événement d'audit
sa-elson (site et API ELSON)	Client Cloud SQL sur elson_db ; écrire ses journaux ; lire ses cinq secrets	Accéder à l'entrepôt, au registre, à menal_db ou aux secrets d'un autre composant (EX4)
sa-api (API SIEM)	Lire le jeu de données ; client Cloud SQL sur menal_db ; ajouter un verdict au nom de l'administrateur ; lire ses trois secrets (base, signature, chiffrement MFA)	Écrire dans les tables de preuves ; accéder à elson_db ou aux secrets d'ELSON
sa-dashboard (Next.js)	Relayer vers l'API les appels de l'administrateur ; lire son secret de session	Accéder aux bases ou aux secrets applicatifs : ses appels métier portent le jeton de l'administrateur
sa-pipeline (job de détection)	Lire les journaux bruts ; écrire access_logs, security_events et detections	Modifier ou supprimer une ligne ; lire un secret applicatif ; accéder aux bases
sa-enrich-job (enrichissement)	Lire le jeu de données ; appeler l'encodeur ; écrire alert_enrichment	Écrire dans detections (EX12, test T12) ; accéder aux bases ; sortir vers Internet
sa-ml-embed (encodeur)	Répondre aux appels authentifiés du job ; écrire ses journaux	Accéder à l'entrepôt, aux bases, aux secrets ; être joint autrement que par le réseau interne
sa-cicd (chaîne, fédérée)	Publier une image ; déployer une révision ; charger les CVE dans cve_findings	Détenir une clé ; lire un secret ; appliquer Terraform ; toucher l'état (EX10)
Compte Compute par défaut	— (aucune charge ne l'utilise : chaque service porte son identité)	Détenir une clé ; son rôle Éditeur hérité de la création du projet est retiré au plan d'amélioration (§ 6.7)

Dans l'intervalle, la session est authentifiée mais non autorisée : le contrôle réside dans cette séparation davantage que dans le second facteur lui-même (EX5).

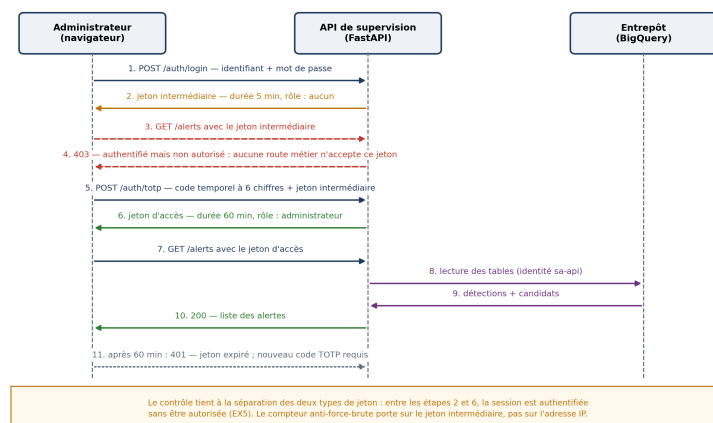


Fig. 4.3 – Séquence de connexion de l'administrateur : le jeton intermédiaire (étape 2) n'ouvre aucun accès ; seul le jeton d'accès délivré après le code TOTP (étape 6) autorise les appels. Le tableau de bord, simple relais vers l'API, n'est pas représenté.

Quatre dispositions d'ingénierie complètent le dispositif. *Deux compteurs applicatifs* doublent

la limitation de débit du point d'entrée : les tentatives de mot de passe sont comptées par identifiant (dixième refus → 429) et les tentatives de code TOTP par jeton intermédiaire, invalidé à la dixième ; aucun n'est indexé sur l'adresse IP, car le tableau de bord et l'API partagent une adresse de sortie et un compteur par adresse bloquerait toutes les sessions d'administration. *L'algorithme de signature* des jetons est fixé dans le code et jamais lu depuis l'en-tête, ce qui écarte l'attaque par confusion d'algorithme. *La graine TOTP* n'est jamais stockée en clair : la classe utilisateur ne porte qu'une référence vers Secret Manager, si bien que le secret n'entre ni dans un journal, ni dans une trace d'erreur, ni dans une sauvegarde. *L' enrôlement* se fait dans le tableau de bord (page « Sécurité (MFA) ») : affichage du QR code, confirmation par un premier code, puis activation obligatoire pour le rôle administrateur. Ce mécanisme protège l'accès, non le poste de l'administrateur, qui n'est pas géré par le socle (limite évaluée au chapitre 6).

4.7 Les données

4.7.1 Données applicatives : une base par application, une instance privée

La base transactionnelle est une instance unique Cloud SQL PostgreSQL 15 (menal-db-staging), **sans adresse publique** : `ipv4Enabled = false`, adresse privée 10.20.0.3 dans la plage d'appairage, TLS obligatoire (`sslMode = ENCRYPTED_ONLY`), joignable uniquement à travers le connecteur d'accès VPC (EX3). Les services s'y connectent avec le rôle Client Cloud SQL et des utilisateurs distincts : chaque application reçoit sa propre base et son propre utilisateur — `elson_db` pour ELSON, `menal_db` pour les comptes, rôles et journaux d'audit du tableau de bord — et le rôle `cloudsqlsuperuser` est retiré aux deux. Un job Cloud Run quotidien (`elson-sql-isolation-check`) vérifie quatre propriétés et journalise le résultat : aucun des deux utilisateurs n'est super-utilisateur, l'utilisateur d'ELSON est refusé sur `menal_db`, l'utilisateur de l'API est refusé sur `elson_db`, chacun garde l'accès à sa propre base. Sauvegardes automatiques et restauration à un instant donné (PITR, rétention sept jours) visent le RPO de cinq minutes (BNF7). La haute disponibilité régionale, qui double le coût, est un paramètre : fausse en recette, vraie en production. Le compromis est explicite : une instance partagée n'est facturée qu'une fois ; la séparation logique, vérifiée chaque jour, est proportionnée au niveau de sensibilité actuel, et le passage à des instances séparées reste l'évolution d'un paramètre.

4.7.2 L'entrepôt de sécurité

L'entrepôt est un jeu de données BigQuery (`menal_security_staging`, chiffré par une clé KMS du socle) de huit tables (tableau 4.3), gouverné par trois principes : un auteur par table (section 4.5), ajout seul pour les tables de preuves et de verdicts, et une colonne `service` pour attribuer chaque événement à l'application qui l'a émis. Le verdict de l'administrateur est conservé dans sa propre table, reliée à la détection par son identifiant : la preuve reste telle qu'elle a été produite, la décision humaine s'y ajoute sans la modifier. Le modèle objet de l'API (annexe B.3) reproduit cette propriété en code : les classes `Detection`, `Enrichment` et `Verdict` n'exposent aucune opération de modification ni de suppression ; les droits IAM et le code sont deux protections indépendantes, vérifiées séparément (test T12 et tests unitaires).

Tab. 4.3 – Modèle de données de l’entrepôt de sécurité : un auteur par table (volumes relevés le 08/09/2026).

Table	Contenu	Écrite par	Lue par	Lignes
raw_logs	Journaux bruts du répartiteur et du WAF, des services et de l’audit, acheminés en flux par le puits de journaux	agent de journalisation (puits)	job de détection	232 254
access_logs	Requêtes normalisées : horodatage, source, service, chemin, statut, latence, agent, utilisateur	sa-pipeline	règles R1–R7, API	80 307
security_events	Événements de sécurité normalisés (refus WAF, échecs d’authentification, erreurs)	sa-pipeline	règles R1–R7, API	63 269
detections	Détections des règles : règle, entité, message, sévérité, tactique et technique ATT&CK, identifiant SHA-256	sa-pipeline, ajout seul	API, job d’enrichissement	284
alert_enrichment	Trois candidats ATT&CK par détection, score, version du modèle	sa-enrich-job, ajout seul	API	282
attack_embeddings	Vecteurs de référence (768 valeurs) des techniques ATT&CK v17.1	administrateur, chargement versionné	job d’enrichissement	872
analyst_verdicts	Verdict (confirmé, faux positif, pris en compte, ignoré), commentaire, auteur, date	API au nom de l’admin, ajout seul	API	23
cve_findings	CVE de l’image livrée (Trivy), sévérité, correctif, empreinte de l’image	sa-cicd	API	par livraison

4.8 La chaîne de détection en flux : de l’événement à l’incident

La première conception (juin) exécutait les règles en requêtes planifiées BigQuery toutes les cinq minutes sur une fenêtre de quinze minutes ; le délai entre un événement et son alerte était borné à vingt minutes et mesuré à 15 min 05 s le 19/08. Ce délai était dicté par la cadence minimale des requêtes planifiées, non par le calcul. La conception retenue en septembre (décision D9, écart É5) remplace les requêtes planifiées par un job Cloud Run de détection déclenché chaque minute par Cloud Scheduler, les journaux étant déjà acheminés en flux par le puits de journaux. La figure 4.4 déroule la chaîne et le budget de temps de chaque maillon.

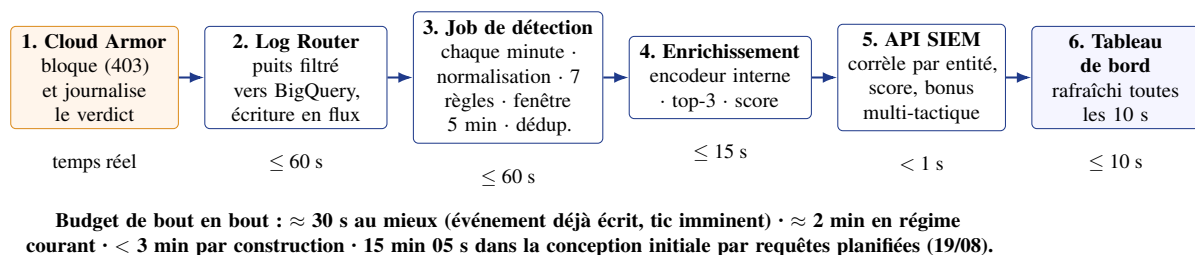


Fig. 4.4 – La chaîne de détection en flux et le budget de temps de chaque maillon. Le délai n’est pas dans le calcul mais dans l’acheminement et la cadence ; chaque maillon est borné et mesurable sur les horodatages des tables.

Chaque maillon est justifié par une propriété vérifiable. **(1)** Le verdict du WAF est rendu au bord, en temps réel, et journalisé dans le journal du répartiteur avec la règle déclenchée : la supervision voit donc même les attaques *bloquées*. **(2)** Le routeur de journaux (*Log Router*) applique un filtre à la source — les sondes de santé ne sont pas acheminées — et écrit chaque entrée retenue dans `raw_logs` par le puits vers BigQuery, dont l'écriture est continue et non par lots ; l'agent de journalisation est le seul auteur de cette table. **(3)** Cloud Scheduler déclenche chaque minute le job `menal-realtime-detector` (identité `sa-pipeline`), qui normalise les nouvelles entrées dans `access_logs` et `security_events`, puis exécute les sept règles sur une fenêtre glissante de cinq minutes ; l'identifiant d'une détection est un condensé SHA-256 de (règle, entité, message, horodatage), ce qui rend la déduplication déterministe (`NOT EXISTS`) : une même rafale ne produit pas dix alertes mais une, dont le compteur indique l'ampleur. **(4)** Le job d'enrichissement (`menal-enrich-job`) encode le message de chaque nouvelle détection par l'encodeur interne et interroge l'index vectoriel (`VECTOR_SEARCH`, trois plus proches voisins) ; les candidats sont écrits avec leur score et, sous le seuil de 0,60, marqués incertains — la part d'alertes ainsi non mappées est affichée dans la vue d'ensemble. **(5)** L'API corrèle à la lecture : toutes les détections d'une entité forment un *incident* dont le score est la somme pondérée par la sévérité, majorée de quinze points si au moins deux tactiques MITRE distinctes sont impliquées, plafonnée à cent. **(6)** Le tableau de bord se rafraîchit toutes les dix secondes. La somme des bornes donne moins de trois minutes ; le chapitre 6 mesure 2 min 05 s entre l'envoi d'une attaque et l'incident affiché.

Trois **alertes de plateforme**, distinctes des sept règles, sont portées par Cloud Monitoring sur les journaux d'audit : A1, refus IAM répétés par une identité ; A2, modification d'une ressource gérée par une identité autre que l'administrateur ou la chaîne (EX6) ; A3, absence de journaux du répartiteur pendant dix minutes, qui protège la chaîne contre sa propre défaillance silencieuse (EX11).

4.9 L'automatisation

Tout ce qui précède est décrit et déployé depuis un dépôt Git unique (Mansour37/`menal-zero-trust`, annexe A) en trois ensembles : le *code d'infrastructure* — cinq modules Terraform, un par couche (`network`, `run-service`, `cloud-sql`, `security`, `detection`), instanciés par trois répertoires d'environnement ; les *chaînes GitHub Actions* — livraison applicative (F3) et contrôle de l'infrastructure (F2), qui s'arrête volontairement avant l'application du plan ; le *code applicatif du socle* — API, tableau de bord, job de détection avec ses sept règles SQL, encodeur et job d'enrichissement. Accueillir une application (UC6) revient à instancier le module `run-service` avec une base et un jeu de secrets : le module commun ne change pas. Les séquences de la livraison figurent en annexe B.5 ; le chapitre 5 en donne les durées.

4.10 Registre des décisions et composants écartés

Les choix d'architecture sont datés, motivés et vérifiés (tableau 4.4) ; une décision n'est close que lorsque sa preuve est établie. Écarter est aussi une décision : au titre du principe

« soutenable avant tout », Kubernetes (exploitation hors de portée d’une personne), le maillage de services (redondant avec l’identité par service), le SIEM dédié (redondant avec l’entrepôt) et le multi-région (disproportionné) sont volontairement absents.

Tab. 4.4 – Registre des décisions d’architecture D1 à D9.

#	Décision	Date	Critère décisif	Prix payé	Preuve
D1	Google Cloud, fournisseur unique	12/05	Cohérence des services managés	Dépendance au fournisseur	Comparatif ; coût mesuré
D2	Cloud Run pour toute l’exécution	15/05	Coût nul hors trafic, aucun serveur	Démarrage à froid	Coût ; démarrage mesuré
D3	Point d’entrée unique (répartiteur + WAF)	20/05	Un seul endroit pour TLS, WAF et débit	Coût fixe du répartiteur	T1, T2
D4	Cloud SQL privée, une base par application	22/05	Base managée, sauvegarde et PITR inclus	Coût fixe ; HA doublerait ce coût	T3 ; restauration
D5	Terraform, état chiffré, trois environnements	28/05	Plan avant application ; dérive détectable	Discipline ; état sensible	Reconstruction ; T6, T7
D6	BigQuery entrepôt, règles en SQL	02/06	Un moteur pour journaux, détection, vecteurs	Pas de SIEM complet	T11, T12 ; scénarios
D7	Encodeur non génératif (ATT&CK-BERT, ONNX)	05/06	Déterminisme, reproductibilité	Pas d’explication en langage naturel	Évaluation ch. 6
D8	GitHub Actions fédérée, quatre portes	18/06	Aucune clé stockée ; portes éprouvées	Dépendance à GitHub	T8–T10 ; 5 refus
D9	Détection en flux : puits de journaux et job à la minute	04/09	Incident affiché en moins de trois minutes	Un job et un planificateur de plus	Scénario du 07/09

Conclusion

L’architecture réalise les douze exigences : trois environnements identiques en code, cinq couches, un point d’entrée unique, neuf identités dont les interdictions font la segmentation, une session à deux facteurs, un entrepôt à auteur unique, une chaîne de détection en flux bornée à trois minutes ; chaque décision est datée et adossée à une preuve. Le chapitre suivant rend compte de la construction.

Chapitre 5

Réalisation

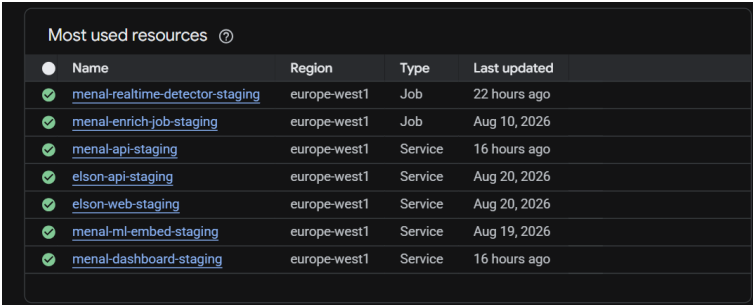
Chaînes automatisées, supervision en flux, tableau de bord, mise en service d'ELSON et conduite du projet.

Introduction

Le chapitre 4 décrivait l'architecture cible ; celui-ci rend compte de ce qui a été construit. Il suit l'ordre des flux fixés au chapitre 3 : la chaîne de provisionnement de l'infrastructure (F2), la chaîne de livraison applicative (F3) et la chaîne de supervision (F4) avec le tableau de bord qui la restitue. Il présente ensuite la mise en service d'ELSON sur le socle, puis la conduite du projet : tâches quotidiennes, difficultés, écarts entre conception et réalisation, planning. Les mesures appartiennent au chapitre 6 ; ce chapitre s'en tient à ce qui a été fait.

5.1 Environnement de travail et organisation du dépôt

Les trois projets Google Cloud décrits en section 4.2.1 sont instanciés depuis le dépôt unique Mansour37/menal-zero-trust, dont l'arbre reprend le modèle en couches (annexe A) : terraform/modules (network, run-service, cloud-sql, security, detection), terraform/envs (dev, staging, prod), .github/workflows (infra-checks.yml, app-delivery.yml), api (FastAPI, job d'enrichissement, encodeur), dashboard (Next.js), ml-pipeline (job de détection : normalisation et sept règles), tests, demo/gate-tests (rejeu des refus) et docs (décisions D1–D9, procédures). Toute modification atteint main par une demande de fusion relue, même quand l'auteur et le relecteur ne font qu'un : la relecture oblige à parcourir le plan Terraform ou le résultat des portes. La figure 5.1 montre ce que la recette exécute réellement : cinq services (le site et l'API d'ELSON, l'API de supervision, le tableau de bord, l'encodeur) et trois jobs (détection, enrichissement, contrôle d'isolation des bases), tous en europe-west1.



Most used resources ⓘ			
Name	Region	Type	Last updated
menal-realtime-detector-staging	europe-west1	Job	22 hours ago
menal-enrich-job-staging	europe-west1	Job	Aug 10, 2026
menal-api-staging	europe-west1	Service	16 hours ago
elson-api-staging	europe-west1	Service	Aug 20, 2026
elson-web-staging	europe-west1	Service	Aug 20, 2026
menal-ml-embed-staging	europe-west1	Service	Aug 19, 2026
menal-dashboard-staging	europe-west1	Service	16 hours ago

Fig. 5.1 – Services et jobs Cloud Run de la recette (relevé du 08/09/2026) : cinq services et trois jobs, chacun sous sa propre identité.

5.2 Chaîne de provisionnement de l'infrastructure (F2)

La chaîne d'infrastructure est **automatisée jusqu'au contrôle et humaine au-delà**, et cette différence avec la chaîne applicative est une décision. À chaque demande de fusion touchant `terraform/`, trois étapes s'exécutent — format, validation sur les trois environnements, analyse de la description par Trivy en mode configuration (bloquante sur gravité critique : stockage public, base exposée, droit trop large) — et aucune ne s'authentifie auprès du fournisseur : la chaîne ne peut structurellement pas modifier l'infrastructure. Le plan est produit et appliqué par l'administrateur, avec ses identifiants et son second facteur ; automatiser l'application aurait exigé d'accorder à la chaîne des droits étendus sur tout le projet, la concentration de privilège que le chapitre 4 refuse. Le plan se lit intégralement, et l'application est la seule voie d'écriture.

L'**état Terraform** est traité comme une donnée sensible : espace Cloud Storage chiffré par une clé KMS, versionné, accessible au seul administrateur (EX7) ; aucune valeur de secret n'y figure. La **détection de dérive** repose sur le plan produit avant chaque application, un plan hebdomadaire selon une procédure écrite, et l'alerte de plateforme A2. Les ressources irremplaçables — instance de base, clés, espace d'état, tables de preuves — portent un verrou de destruction dans le code (levé en développement pour permettre les reconstructions de test). Cinq **exceptions** au « tout est en code » sont nommées : les valeurs des secrets (saisies dans Secret Manager) ; les données applicatives et comptes créés à l'exécution ; les enregistrements DNS, tenus chez le bureau d'enregistrement ; le chargement initial des vecteurs ATT&CK par un script versionné ; l'image déployée de chaque service, propriété de la chaîne de livraison — l'infrastructure possède le service, la chaîne possède l'image.

5.3 Chaîne de livraison applicative (F3)

5.3.1 Huit étapes, quatre portes

La chaîne est décrite dans un fichier unique, `app-delivery.yml`, calqué étape pour étape sur le diagramme de conception : un job par boîte, dans le même ordre, avec le même numéro (figure 5.2). Elle se déclenche sur deux événements. Une *demande de fusion* exécute les étapes 2 à 6 sans rien publier : aucun job de cette phase ne détient la permission `id-token`, la chaîne est donc *techniquement incapable* de prendre une identité cloud. Une *fusion sur main* exécute les étapes 1 à 8, filtrées par les chemins modifiés — sans ce filtre, une correction de documentation redéploierait l'application. Chaque job attend le précédent (`needs`) : l'ordre est appliqué, pas décoratif.

L'ordre des portes n'est pas arbitraire : un secret poussé est compromis à la seconde où il est publié, on le cherche donc *en premier* ; on ne construit pas une image pour du code qui n'a pas passé l'analyse ni les tests ; entre l'étape 5 et la porte 6, l'image ne transite qu'en *artefact interne au run*, si bien que publier une image non scannée est techniquement impossible. La publication référence l'image par son empreinte `sha256` relue depuis le registre, jamais par une étiquette mobile ; le déploiement épingle la révision à cette empreinte, puis une sonde interroge le service *par son domaine public*, donc à travers le WAF. Le 07/09, l'empreinte publiée et celle en service

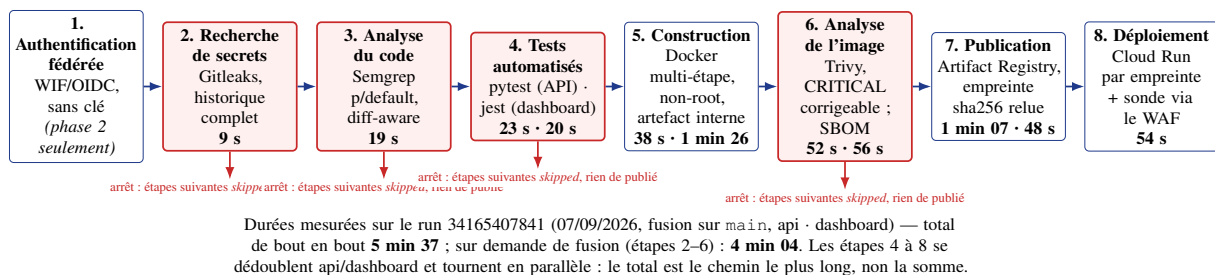


Fig. 5.2 – La chaîne de livraison applicative : huit étapes, dont quatre portes bloquantes (rouge). Une porte rouge ne fait pas qu’échouer : tous les jobs suivants passent en *skipped*, aucune image n’est publiée, et la chaîne ne s’authentifie même pas au cloud.

sur Cloud Run étaient identiques : l’image scannée est exactement celle qui sert le trafic. Le retour arrière tient en une commande, rappelée par le job en cas d’échec de la sonde : 11,6 s.

5.3.2 Les refus vérifiés

Une chaîne se juge sur ses refus : sans cas de refus observé, rien ne distingue une porte qui laisse passer des artefacts sains d’une porte qui ne fonctionne pas. Le 07/09/2026, chacune des quatre portes a été attaquée par une demande de fusion jetable portant *une seule vraie faille*, le reste du dépôt intact (tableau 5.1) ; les cinq runs sont rejouables par le script `demo/gate-tests`, et la chaîne s’éteint chaque fois pile au maillon attaqué. S’y ajoute un refus *non provoqué*, survenu en exploitation le 20/08 : la porte d’analyse d’image a bloqué une livraison du tableau de bord sur une vulnérabilité critique corrigéable (ip 1.1.8, CVE-2023-42282) embarquée par le gestionnaire de paquets de l’image de base ; aucune mise à jour du code livré ne pouvait la corriger, la cause a été retirée — le gestionnaire de paquets n’a pas sa place dans l’image finale — plutôt qu’inscrite sur une liste d’exception. Une porte a trouvé dans le socle lui-même un défaut que la conception n’avait pas anticipé.

Tab. 5.1 – Les cinq refus provoqués du 07/09/2026 : une faille injectée, un type détecté, un verdict daté (identifiants de run GitHub Actions).

Porte	Faillle injectée	Ce que la porte a rendu	Verdict en	Run
2 — Gitleaks	Clé à haute entropie dans le fichier <code>_leaked_config.py</code> (répertoire <code>api/app/</code>)	Règle <code>generic-api-key</code> , 2 fuites, commit et empreinte identifiés ; étapes 3–8 ignorées	46 s (job rouge en 6 s)	34165824113
3 — Semgrep	<code>eval()</code> sur entrée utilisateur + <code>subprocess(..., shell=True)</code>	2 constats bloquants sur 290 règles exécutées (exécution de code, injection de commande) ; 4–8 ignorées	37 s (20 s)	34165832135
4 — pytest	Régression volontaire (<code>assert 1 == 2</code>)	1 échec, 29 réussites ; 5–8 ignorées	1 min 23 (24 s)	34165841139
4 — jest	Régression front (<code>expect(1).toBe(2)</code>)	1 échec, 22 réussites : la porte réagit à la régression précise, elle ne casse pas en bloc	1 min 09 (25 s)	34165850179
6 — Trivy	Dépendance <code>PyYAML==5.3.1</code> dans <code>requirements.txt</code>	Total : 1 (CRITICAL : 1) — CVE-2020-14343, correctif 5.4 disponible ; 7–8 ignorées, aucune image publiée	3 min 06 (33 s)	34165859372

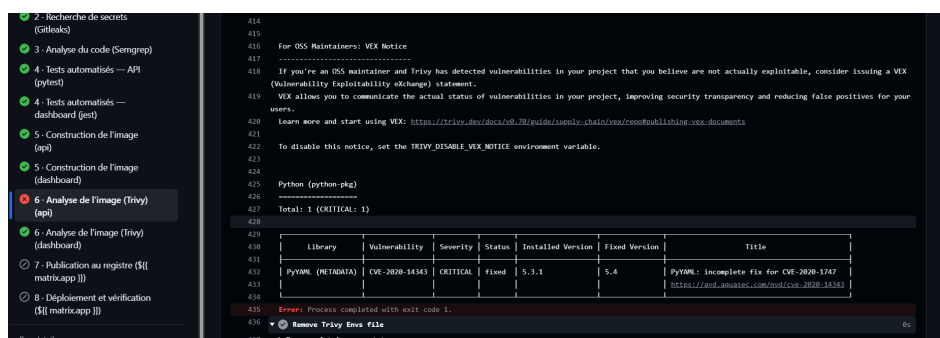


Fig. 5.3 – Porte 6 fermée (run 34165859372) : Trivy identifie la seule CVE critique corrigeable de l’image (PyYAML 5.3.1, correctif 5.4), le job s’arrête en code 1 et les étapes 7 et 8 sont ignorées. Les relevés des quatre autres portes figurent en annexe C.

Une porte n’est utile que si l’on résiste à la tentation de la désactiver : chaque exclusion d’une règle d’analyse porte une justification écrite ancrée sur un fichier et une ligne, versionnée avec le code. Une limite est assumée : la suite de tests de bout en bout, qui affichait un job rouge de douze minutes sans rien bloquer, a été sortie de la chaîne et reste lançable à la demande.

5.4 Chaîne de supervision et de détection (F4)

5.4.1 Collecte en flux et normalisation

Quatre sources alimentent l’entrepôt : les journaux du répartiteur (dont les verdicts du WAF), ceux des services Cloud Run, les journaux d’audit du plan de contrôle et les journaux de flux du VPC (refus du pare-feu). Le filtrage s’opère à la source — les sondes de santé, nombreuses et sans valeur, ne sont pas acheminées (BNF3). Le puits du routeur de journaux écrit les entrées retenues en flux dans `raw_logs` (232 254 lignes au 08/09/2026) ; à chaque exécution, le job de détection (`sa-pipeline`) met les nouvelles entrées au schéma commun de `access_logs` et `security_events`, avec le nom du service émetteur : 58 898 requêtes normalisées pour l’API de supervision, 8 074 pour l’API d’ELSON et 685 pour son site à cette date.

5.4.2 Les sept règles de détection

Les règles sont du code : sept fichiers SQL d’inspiration Sigma (mêmes concepts : condition, fenêtre, sévérité, correspondance MITRE), versionnés et revus en demande de fusion, exécutés chaque minute par le job de détection sur une fenêtre glissante de cinq minutes (tableau 5.2). Trois règles (R2, R3, R6) lisent aussi les verdicts du WAF : elles voient donc les attaques bloquées au bord, et la supervision ne dépend pas du fait que l’attaque ait réussi. Les sept techniques couvrent **cinq tactiques MITRE distinctes** — accès initial, accès aux identifiants, découverte, collecte, impact — une couverture de chaîne d’attaque, pas sept variantes de la même chose. La déduplication par identifiant SHA-256 garantit qu’une même rafale produit une détection et non dix.

5.4.3 La qualification assistée

Le job d’enrichissement (`sa-enrich-job`) lit les détections non encore enrichies, compose pour chacune un texte court (règle, entité, message), l’envoie à l’encodeur (`sa-ml-embed`) qui le transforme en vecteur de 768 valeurs, puis interroge la recherche vectorielle de BigQuery sur

Tab. 5.2 – Les sept règles de détection R1 à R7 (job à la minute, fenêtre de cinq minutes).

#	Nom	Sévérité	Ce qui la déclenche	Tactique / technique	Source
R1	Force brute auth.	Haute	> 5 réponses 401/403 par IP	TA0006 Credential Access / T1110	access_logs
R2	Pic WAF	Moyenne	> 10 blocages Cloud Armor par IP	TA0040 Impact / T1498	WAF
R3	Path traversal	Haute	motifs <code>../</code> , <code>..%2f</code> , <code>..%5c</code>	TA0001 Initial Access / T1190	application + WAF
R4	User-agent suspect	Moyenne	agents <code>curl</code> , <code>python</code> , <code>wget</code> , <code>go-http</code> sur <code>/api/</code>	TA0007 Discovery / T1046	application
R5	Latence anormale	Faible	requête > 5000 ms	TA0040 Impact / T1499	application
R6	Pattern injection	Critique	SQLi (<code>1=1</code> , <code>UNION SELECT</code>), XSS (<code><script></code>), LFI (<code>/etc/passwd</code>)	TA0001 Initial Access / T1190	application + WAF
R7	Fichier sensible	Haute	<code>.env</code> , <code>.git</code> , <code>/config</code> , <code>/actuator</code> , <code>/wp-admin</code>	TA0009 Collection / T1005	application

`attack_embeddings` (872 vecteurs, 697 techniques et sous-techniques, matrice v17.1). Les trois techniques les plus proches au-dessus de 0,60 sont écrites dans `alert_enrichment` avec leur score et la version du modèle (`attack-bert-onnx-fp32@v1.0`). Sur une inclusion de `/etc/passwd`, le modèle propose typiquement T1003.008 (*OS Credential Dumping : /etc/passwd*) autour de 0,70 alors qu’aucun mot « credential » ne figure dans la requête : il comprend le sens. Les poids du modèle sont un artefact tiers intégré à l’image de l’encodeur et vérifiés par empreinte ; l’encodeur n’a aucune sortie vers Internet.

5.4.4 L’API de supervision

Tout passe par l’API FastAPI : le tableau de bord n’interroge jamais l’entrepôt ni les bases, et ses seuls droits sont ceux du jeton qu’il transmet. L’API expose une dizaine de routes — connexion, second facteur, enrôlement MFA, détections, incidents et fiche d’incident (`/siem/incidents/{entité}`), verdict, indicateurs, couverture ATT&CK, vulnérabilités, santé des règles, santé du service. Toutes les routes métier déclarent le même composant de vérification, qui valide le jeton, en extrait le rôle et refuse l’appel sinon : un seul contrôle à auditer, déclaré partout. Deux rôles existent : administrateur (verdicts, enrôlement) et lecture (consultation). Le verdict est écrit en ajout seul dans `analyst_verdicts` — `CONFIRMED`, `FALSE_POSITIVE`, `ACKNOWLEDGED`, `IGNORED` — la lecture prenant le dernier par entité : la preuve n’est jamais écrasée. En recette et en production, une API injoignable produit un écran d’erreur explicite, jamais un faux « 0 alerte ».

5.4.5 Le tableau de bord MENAL Sentinel et l’accès à deux facteurs

Le tableau de bord (Next.js) est organisé en trois groupes : *Surveillance* (vue d’ensemble, journaux et alertes API), *Détection et analyse* (détections, incidents, couverture ATT&CK, vulnérabilités, santé des règles) et *Compte* (sécurité MFA) ; un sélecteur filtre toutes les vues par application (`menal`, `elson`). L’affichage est réellement *live* : toutes les dix secondes la page recharge ses données côté serveur sans rechargement visible ni cache, et le bandeau indique « Données à jour » ou « API injoignable ».

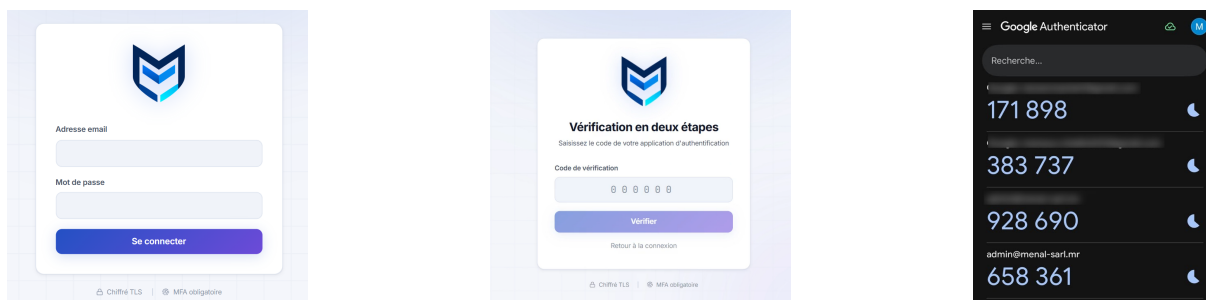


Fig. 5.4 – Accès administrateur à deux facteurs : mot de passe (gauche), puis code de vérification TOTP (centre) généré par l’application d’authentification du compte `admin@menal-sarl.mr` (droite). Entre les deux écrans, la session n’a aucun droit.

La **vue d’ensemble** (figure 5.5) répond à la première question de l’administrateur le matin : que s’est-il passé depuis hier ? Cinq indicateurs sur vingt-quatre heures (requêtes, taux d’erreur, échecs d’authentification, alertes actives, latence moyenne et p99), la carte SIEM — détections et entités distinctes, blocages WAF, part d’alertes non mappées par le modèle (indicateur d’honnêteté, jamais caché) et version du modèle actif — la répartition par sévérité et la courbe de trafic. La vue distingue le trafic applicatif de la détection SIEM : deux plans qui ne se confondent pas.

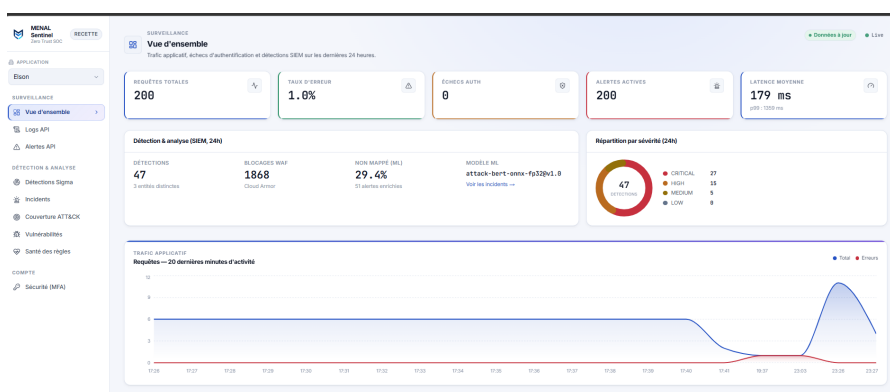


Fig. 5.5 – Vue d’ensemble de MENAL Sentinel en recette (07/09/2026, 23 h 27, deux minutes après l’attaque du scénario du chapitre 6) : 47 détections, 1 868 blocages WAF, modèle `attack-bert-onnx-fp32@v1.0`, pic de trafic visible à 23 h 26.

L’écran des **détections** (figure 5.6) restitue la sortie brute des sept règles : une ligne par détection avec horodatage, règle, sévérité, entité, application, tactique et technique ATT&CK et message ; l’entité est cliquable et ouvre la fiche d’incident. L’écran des **incidents** (figure 5.7) est le vrai apport : sept alertes séparées deviennent *un* incident par entité, avec son score, sa sévérité dérivée, le nombre de détections, les tactiques distinctes (badge « chaîné » au-delà de deux), les techniques et le verdict. La fiche d’incident réunit la jauge de score, les candidats du modèle (« le socle propose ») et le panneau de décision humaine (quatre boutons de verdict, commentaire, historique horodaté) : *le socle propose, l’analyste décide*.

Deux vues secondaires bouclent la chaîne (figure B.6, annexe B). La **santé des règles** donne, par règle, les déclenchements sur trente jours et le taux de faux positifs calculé à partir des verdicts — une règle sans verdict affiche « inconnu », pas 0 %, et une règle silencieuse est signalée comme telle. Les **vulnérabilités** restituent les CVE de l’image chargées par la chaîne de livraison (boucle « la chaîne scanne, la supervision voit »), triées par exploitation confirmée (catalogue KEV), puis

Timestamp	Règle	Sévérité	Entité	App	Mitre Attack	Message
07/09/2026 23:26:06	R6 — Pattern injection detecte	CRITICAL	203.0.113.7	menal-elson-api-backend-staging	TAG001 T1190	Pattern injecté (bloque par le WAF) sur https://elson.menal-sarl.com/api/search?id=1&1=1 dep...
07/09/2026 23:25:44	R2 — Pic WAF	MEDIUM	203.0.113.7	elson-api-staging	TAG048 T1498	11 requêtes bloquées par Cloud Armor depuis 43.98.375.52 en 15 min
07/09/2026 23:25:44	R6 — Pattern injection detecte	CRITICAL	203.0.113.7	elson-api-staging	TAG001 T1190	Pattern injecté (bloque par le WAF) sur https://elson.menal-sarl.com/api/search?path=/del...
07/09/2026 23:25:44	R3 — Path traversal	HIGH	203.0.113.7	elson-api-staging	TAG001 T1190	Tentative path traversal (bloque par le WAF) sur https://elson.menal-sarl.com/api/search?...
07/09/2026 23:25:44	R6 — Pattern injection detecte	CRITICAL	203.0.113.7	elson-api-staging	TAG001 T1190	Pattern injecté (bloque par le WAF) sur https://elson.menal-sarl.com/api/search?file=../del...
07/09/2026 23:25:44	R6 — Pattern injection detecte	CRITICAL	203.0.113.7	elson-api-staging	TAG001 T1190	Pattern injecté (bloque par le WAF) sur https://elson.menal-sarl.com/api/search?q=admin...
07/09/2026 23:25:44	R3 — Path traversal	HIGH	203.0.113.7	elson-api-staging	TAG001 T1190	Tentative path traversal (bloque par le WAF) sur https://elson.menal-sarl.com/api/search?...
07/09/2026 23:25:44	R6 — Pattern injection detecte	CRITICAL	203.0.113.7	elson-api-staging	TAG001 T1190	Pattern injecté (bloque par le WAF) sur https://elson.menal-sarl.com/api/search?file=../del...
07/09/2026 23:25:44	R3 — Path traversal	HIGH	203.0.113.7	elson-api-staging	TAG001 T1190	Tentative path traversal (bloque par le WAF) sur https://elson.menal-sarl.com/api/search?...
07/09/2026 23:25:44	R6 — Pattern injection detecte	CRITICAL	203.0.113.7	elson-api-staging	TAG001 T1190	Pattern injecté (bloque par le WAF) sur https://elson.menal-sarl.com/api/search?file=../del...
07/09/2026 23:25:44	R3 — Path traversal	HIGH	203.0.113.7	elson-api-staging	TAG001 T1190	Tentative path traversal (bloque par le WAF) sur https://elson.menal-sarl.com/api/search?...
07/09/2026 23:25:44	R6 — Pattern injection detecte	CRITICAL	203.0.113.7	elson-api-staging	TAG001 T1190	Pattern injecté (bloque par le WAF) sur https://elson.menal-sarl.com/api/search?file=../del...
07/09/2026 23:25:44	R3 — Path traversal	HIGH	203.0.113.7	elson-api-staging	TAG001 T1190	Tentative path traversal (bloque par le WAF) sur https://elson.menal-sarl.com/api/search?...
07/09/2026 23:25:44	R6 — Pattern injection detecte	CRITICAL	203.0.113.7	elson-api-staging	TAG001 T1190	Pattern injecté (bloque par le WAF) sur https://elson.menal-sarl.com/api/search?file=../del...
07/09/2026 23:25:44	R3 — Path traversal	HIGH	203.0.113.7	elson-api-staging	TAG001 T1190	Tentative path traversal (bloque par le WAF) sur https://elson.menal-sarl.com/api/search?...
07/09/2026 23:25:44	R6 — Pattern injection detecte	CRITICAL	203.0.113.7	elson-api-staging	TAG001 T1190	Pattern injecté (bloque par le WAF) sur https://elson.menal-sarl.com/api/search?file=../del...

Fig. 5.6 – Écran des détections (recette, 07/09/2026) : les règles R6 (injection, critique), R3 (traversée de chemin, haute) et R2 (pic WAF, moyenne) déclenchées par une même adresse source (masquée, RFC 5737), chacune portant sa tactique et sa technique MITRE.

Détection & Analyse							
Incidents Détections regroupées par entité (IP / secteur). Score = somme pondérée par sévérité, + bonus +15 si l'entité a déclenché 2 tactiques MITRE distinctes ou plus (chaîne d'attaque probable). 1 chaîne(s) détectée(s).							
Entité	App	Score	Sévérité	Détections	Tactiques	Techniques	Verdict
203.0.113.7	menal-elson-api-backend-staging	100	CRITICAL	47	2 (chaîne)	T1190, T1498	Confirmé

Fig. 5.7 – Écran des incidents : les 47 détections de l’adresse source sont corrélées en un incident unique de score 100, sévérité critique, deux tactiques distinctes (chaîne d’attaque probable, T1190 et T1498), verdict « Confirmé » par l’administrateur à 23 h 28.

par technique observée sur l’environnement, puis par probabilité d’exploitation (EPSS).

5.5 Mise en service d’ELSON sur le socle

5.5.1 Prérequis et procédure d’accueil

Le besoin BF8 exigeait d’accueillir une application sans reconception. Quatre prérequis s’imposent à toute application candidate : ne conserver aucun état sur le disque local, se configurer par variables d’environnement, consommer un secret par usage lu par référence, écouter sur le port fourni et exposer un point de santé. ELSON — un site et une API Node/Express déployés en deux services, une base PostgreSQL — les remplissait après deux adaptations : la configuration par variables et le remplacement d’un contrôle d’accès administrateur fondé sur une liste d’adresses lue dans un en-tête de proxy, forgeable derrière un répartiteur — ce que la revue d’accueil sert précisément à détecter : *un changement d’hôte peut invalider un contrôle de sécurité*. La procédure compte six étapes : (1) déclarer les paramètres (nom, image, dimensionnement) dans le fichier de variables ; (2) produire, relire et appliquer le plan — identité sa-elson, conteneurs de secrets, base elson_db et utilisateur, service Cloud Run, routage au point d’entrée, étiquette réseau ; (3) renseigner les secrets dans Secret Manager ; (4) raccorder le répertoire de l’application au fichier de chaîne ; (5) livrer une première version ; (6) vérifier — sonde par le point d’entrée, contrôle d’isolation des bases, apparition de l’application dans le tableau de bord.

5.5.2 Ce qui a été mis en service et pourquoi

ELSON est servie à l’adresse `https://elson.menal-sarl.com` (figure 5.8) : le domaine pointe vers l’adresse unique du répartiteur, le certificat est géré par la plateforme, et toute requête traverse le WAF et la limitation de débit avant d’atteindre l’un des deux services Cloud Run

d'ELSON — `elson-web` (site) et `elson-api` (API Node) — qui n'acceptent que le trafic du répartiteur et portent la même identité `sa-elson`. La base `elson_db` vit sur l'instance privée, isolée de `menal_db`, et le job quotidien de contrôle vérifie que l'utilisateur d'ELSON est refusé sur `menal_db` et l'utilisateur de l'API sur `elson_db` ; ses cinq secrets (base, clé d'API, jetons, poivre OTP, URL audio) ne sont lisibles que par `sa-elson` ; ses journaux alimentent la supervision avec le nom du service émetteur, si bien que les sept règles et le sélecteur d'application la couvrent dès le premier jour, sans une ligne de code de supervision dans l'application.

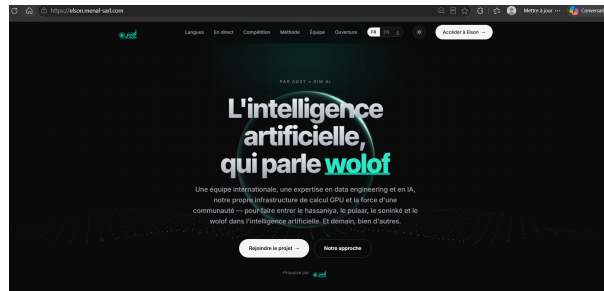


Fig. 5.8 – Page d'accueil d'ELSON servie par le socle à `elson.menal-sarl.com` (HTTPS par le répartiteur, WAF actif) : plateforme trilingue de constitution de corpus pour les langues nationales.



Fig. 5.9 – Parcours d'un contributeur ELSON derrière le socle : connexion (identifiant masqué), validation d'une contribution (phrase source, traduction hassaniya, audio à évaluer), espace contributeur (points de la session de compétition) et classement. Ces écrans concentrent la valeur à protéger ; identités des utilisateurs et logique métier restent la responsabilité de l'application.

Trois choix de mise en service sont défendus. **Servir le trafic pilote depuis la recette pendant le rodage** : la recette a la topologie exacte de la production et porte toutes les mesures du chapitre 6 ; y servir le domaine public pendant trois semaines permet de calibrer les seuils de R1 et R2 sur du trafic réel avant de figer la production, la bascule vers `prod` n'étant qu'un changement DNS et l'application d'un plan déjà répété (41 min, section 6.5). **Une instance de base partagée, des bases séparées** : une instance par application multiplierait le coût fixe dominant ; la séparation logique est proportionnée à la sensibilité actuelle et réversible par un paramètre. **Pas de haute disponibilité en recette** : elle double le coût de la base et n'a de sens qu'en production, avec l'engagement de disponibilité.

5.5.3 Portée de l'isolation et frontière de responsabilité

L'isolation obtenue joue sur quatre plans : *identité et secrets*, totale (test T4) ; *données*, séparation logique sur une instance partagée vérifiée chaque matin ; *réseau*, sous-réseau partagé avec refus par défaut en sortie, la séparation étant portée par l'identité ; *stockage d'objets*, qui serait commun à plusieurs applications — limite publiée au chapitre 6. Héberger l'application d'un tiers oblige enfin à trancher qui répond de quoi (tableau 5.3) : cette frontière correspond à ce qu'un hébergeur peut techniquement contrôler et pourrait fonder un contrat de service.

5.6 Conduite du projet

5.6.1 Tâches quotidiennes

Cinq activités quotidiennes : revue des incidents et saisie des verdicts (une dizaine de minutes), relecture des plans Terraform et des résultats de la chaîne à chaque demande de fusion, veille sur les vulnérabilités signalées par la porte d'analyse d'image, développement selon la phase, tenue du journal de décisions et des procédures ; un point hebdomadaire fixait les priorités, une démonstration clôturait chaque itération.

5.6.2 Difficultés rencontrées et corrections

Chaque difficulté a produit une correction et une leçon (tableau 5.4) ; aucune n'a fait abandonner un objectif, trois ont modifié la conception (É1, É4, É5).

5.6.3 Écarts entre conception et réalisation

Cinq écarts séparent la conception initiale de l'implémentation (tableau 5.5) : quatre résorbés, un assumé. Les nommer permet au chapitre 6 de mesurer ce qui existe, non ce qui était prévu.

5.6.4 Planning du stage

Le stage s'est déroulé du 3 mars au 7 septembre 2026, soit vingt-sept semaines, en neuf phases et sept jalons (figure 5.10). Les phases se chevauchent volontairement : la validation a débuté

Tab. 5.3 – Frontière de responsabilité entre le socle (hébergeur) et l'éditeur de l'application hébergée.

Domaine	Le socle	L'éditeur de l'application
Domaine, certificat, TLS, WAF et limitation de débit	Intégral : politique commune	Déclare ses chemins d'authentification
Réseau : refus par défaut, autorisations	Intégral	Déclare ses destinations légitimes
Identité machine et secrets	Intégral : identité dédiée, un secret par usage	Consomme les secrets par référence
Base : instance, sauvegarde, restauration	Intégral	Cohérence applicative après restauration
Chaîne de livraison : portes, registre, déploiement	Intégral	Corrige les constats bloquants
Supervision : collecte, règles, tableau de bord	Intégral pour les signaux de transport et de plateforme	Émet ses journaux selon la convention du socle
Identités des utilisateurs finaux, logique métier (dont la fraude), conformité réglementaire	Moyens techniques seulement	Intégral : base légale, droits des personnes, conservation

Tab. 5.4 – Les sept difficultés rencontrées, leur correction et la leçon retenue.

#	Difficulté	Cause	Correction	Leçon
1	Démarrage à froid de l'enrichissement (27 s, pic 94 s)	Modèle de plusieurs centaines de Mo chargé à chaque réveil	Poids intégrés à l'image, encodeur séparé	Un coût mesuré se gouverne
2	Blocage collectif par la limitation de débit	Compteur indexé sur l'IP, partagée par le tableau de bord et l'API	Compteurs par identifiant et par jeton	Mesurer la bonne entité avant de fixer un seuil
3	Porte d'analyse statique en « faux vert »	Action Semgrep dépréciée : échec interne rapporté réussi	CLI direct, code de sortie propagé ; 74 constats révélés	Une porte déclarée n'est pas une porte vérifiée
4	État Terraform divergent	Application interrompue par le réseau	Comparaison explicite puis resynchronisation	Un échec partiel fausse le plan suivant
5	Règle de sortie réseau sans effet	Le trafic sortant passait par le connecteur d'accès VPC, hors des étiquettes visées par la règle	Règles réécrites pour les instances du connecteur (11–25/08)	Un contrôle qui ne voit pas le trafic est une déclaration
6	Coût inattendu des requêtes planifiées	Forfait minimal par table balayée (78 Go facturés pour 24 Mo)	Règles regroupées, partitions du jour : coût divisé par cinq	Lire la grille tarifaire avant de planifier
7	Alerte quinze minutes après l'attaque	Cadence minimale des requêtes planifiées BigQuery	Job de détection à la minute (D9, É5)	Le délai est dans la cadence, pas dans le calcul

Tab. 5.5 – Les cinq écarts entre conception et réalisation.

#	Prévu	Constaté	Sort	Date
É1	Règles de pare-feu de sortie portées par les services	Le trafic passait par le connecteur d'accès VPC (10.0.3.0/28), hors des étiquettes visées	Résorbé : règles appliquées aux instances du connecteur	25/08
É2	Seuils de débit déclarés en code	Réglés en console pendant les essais	Résorbé : module <code>network</code>	07/08
É3	Tableau de bord passant par l'API	Première version lisant l'entrepôt en direct	Résorbé côté code ; le rôle de lecture résiduel sur le jeu de données est au plan d'amélioration	30/07
É4	Enrichissement à chaque détection	Par lots toutes les quinze minutes	Résorbé par É5 : job déclenché à la suite de la détection	04/09
É5	Règles en requêtes planifiées toutes les cinq minutes	Délai réel de quinze minutes (cadence minimale des requêtes planifiées)	Assumé comme évolution (D9) : job de détection à la minute	04/09

pendant la réalisation, les tests étant prêts avant les composants ; la dernière semaine a été consacrée au passage en flux et aux refus provoqués.

Conclusion

Le socle est construit : infrastructure en code pour trois environnements, chaîne à quatre portes éprouvées par des refus, sept règles à la minute sur des journaux en flux, qualification assistée et tableau de bord à deux facteurs, ELSON servie avec sa base, son identité et ses secrets. Le chapitre 6 le mesure.

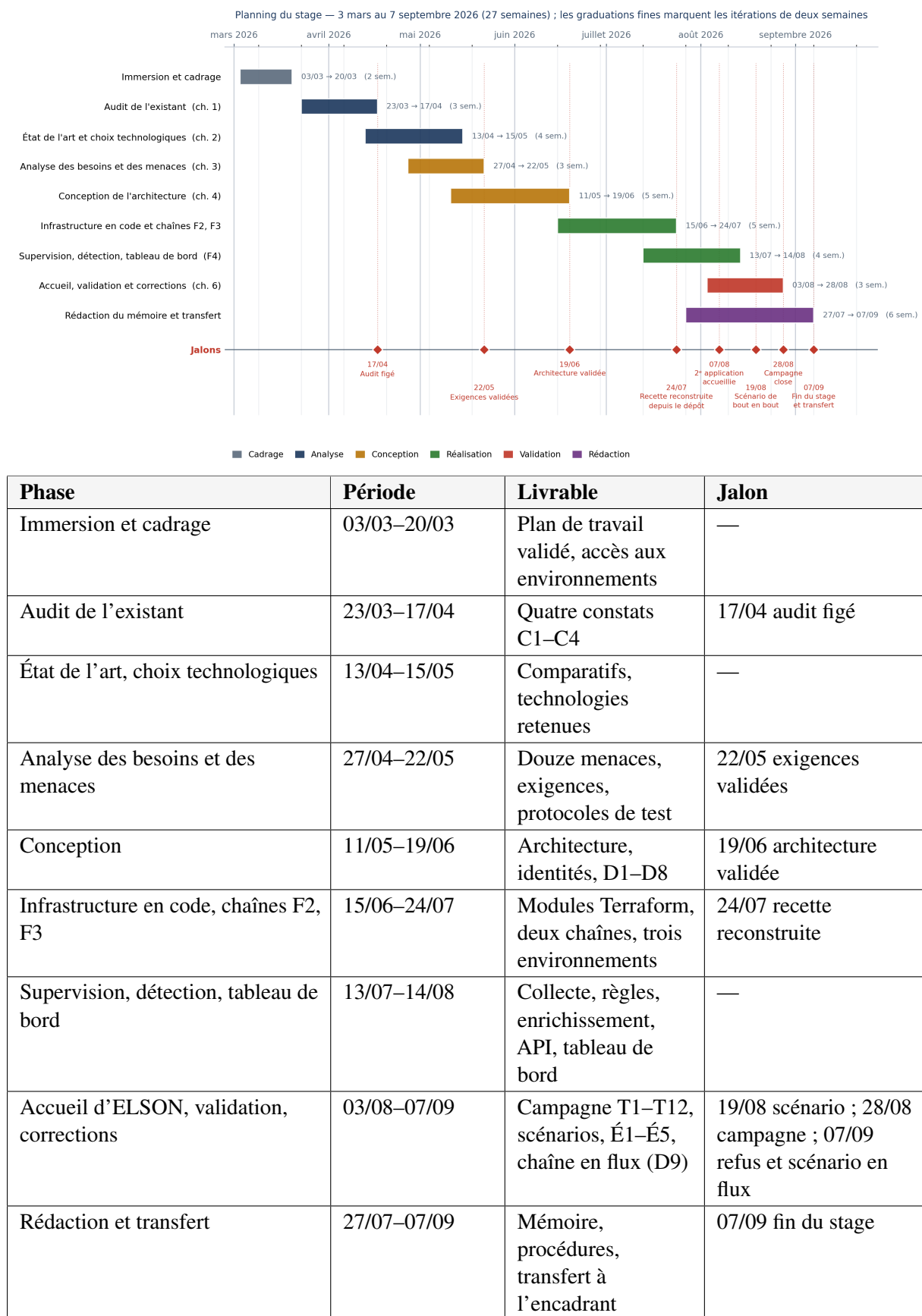


Fig. 5.10 – Planning du stage du 03/03/2026 au 07/09/2026 : neuf phases et sept jalons. Les phases se chevauchent volontairement ; les itérations sont de deux semaines.

Chapitre 6

Validation, résultats et limites

Une architecture Zero Trust ne s'affirme pas, elle se démontre.

Introduction

Les chapitres précédents ont analysé, conçu et construit ; celui-ci mesure : douze tests dérivés du chapitre 3, deux scénarios de bout en bout (par lots le 19/08, en flux le 07/09), l'évaluation de la qualification assistée, les mesures de performance, de résilience et de coût, puis la confrontation aux principes du NIST et aux objectifs du chapitre 1. Les résultats défavorables sont conservés et expliqués.

6.1 Stratégie de validation

Trois résultats sont possibles pour chaque test : **conforme** quand une preuve nommée couvre le critère du tableau 3.4 ; **partiellement conforme** quand la part manquante est désignée avec précision ; **non conforme** quand le critère n'est pas tenu, l'exigence étant alors rouverte avec sa menace et un correctif. Les preuves sont classées en quatre **rangs** fixés à l'avance, du plus fort au plus faible : rang 1, test automatisé rejoué sans intervention humaine ; rang 2, refus réellement survenu en exploitation ; rang 3, vérification en direct et datée ; rang 4, configuration déclarée en code et contrôlée sur la plateforme. Une capture d'écran ne figure pas dans cette échelle : elle *restitue* une preuve, elle n'en est jamais une ; celles du mémoire portent toujours le rang et la date de la preuve qu'elles illustrent. Les résultats ont été obtenus du 19 au 28/08/2026 sur la recette, puis complétés le 07/09/2026 après le passage en flux (D9) ; la recette a la topologie de la cible mais un trafic pilote seulement : les seuils comportementaux ne sont pas encore calibrés (section 6.7).

6.2 Résultats de la campagne de tests

Le tableau 6.1 donne, pour chacun des douze tests, le résultat, la preuve, sa date et son rang ; les relevés figurent en annexe D. **Bilan : dix conformes, deux partiellement conformes, aucun non conforme.** Cinq résultats appellent un commentaire.

T1, corrigé. Le 23/08, une traversée de chemin brute recevait une redirection au lieu d'un refus : le répartiteur normalisait l'URL avant l'évaluation des règles. Le correctif — règles d'anomalies de protocole, en observation puis en blocage — a été appliqué, et le re-test du 07/09 refuse les treize charges, dont // //etc/passwd. La non-conformité est conservée : elle prouve que les tests ont été écrits avant la conception. **T6, partiel.** Une ressource créée hors code par l'administrateur lui-même n'est signalée par aucun contrôle automatique ; le correctif envisagé est une alerte A2 élargie à toute création par l'administrateur hors d'une fenêtre d'application déclarée. **T11, partiel.** La collecte est désormais quasi immédiate, mais

Tab. 6.1 – Synthèse de la campagne de douze tests, à la date du 07/09/2026.

Test	Exigence	Résultat	Preuve invoquée	Date	Rang
T1	EX1 filtrage	✓ Conforme	23/08 : SQLi, XSS, LFI refusés (403) mais traversée brute . . . // redirigée (302) : non conforme, correctif par règles d'anomalies de protocole. Re-test 07/09 : 13 charges, 13 refus 403 (figure 6.1)	07/09	3
T2	EX2 débit	✓ Conforme	Onze tentatives de mot de passe sur un identifiant : dix 401 puis un 429 avec <code>retry-after</code> ; test automatisé à chaque livraison	19/08	1
T3	EX3 base privée	✓ Conforme	Instance <code>menal-db-staging</code> : <code>ipv4Enabled = false</code> , adresse privée 10.20.0.3 de type <code>PRIVATE</code> , TLS obligatoire ; connexion directe depuis Internet en échec (relevé du 08/09, annexe D)	21/08, 08/09	3, 4
T4	EX4 identités	✓ Conforme	Un lecteur par secret (neuf secrets, relevé du 08/09) ; job quotidien de contrôle d'isolation des bases : cinq exécutions consécutives réussies du 04 au 08/09, quatre contrôles vrais à chaque fois (annexe D)	08/09	1, 3
T5	EX5 session	✓ Conforme	Jeton intermédiaire refusé (403) sur toutes les routes métier ; jeton expiré refusé (401) ; enrôlement TOTP éprouvé ; test automatisé	23/08	1
T6	EX6 dérive	■ Partiel	Un attribut modifié en console apparaît dans le plan (« 1 to change ») ; une ressource créée hors code par l'administrateur n'apparaît ni dans le plan ni dans l'alerte A2, qui exclut l'administrateur par conception	19/08	3
T7	EX7 état	✓ Conforme	Accès à l'espace d'état refusé à trois identités de service ; versions actives ; chiffrement KMS vérifié	20/08	3, 4
T8	EX8 secrets	✓ Conforme	Clé factice dans une branche : chaîne arrêtée à la porte 2 (règle <code>generic-api-key</code>) en 46 s, historique complet analysé, étapes 3–8 ignorées	07/09	3
T9	EX9 vulnérabilités	✓ Conforme	Refus réel non provoqué (20/08, <code>ip 1.1.8</code>) ; refus provoqués du 07/09 aux portes 3, 4 et 6 (Semgrep 2 constats ; pytest et jest 1 échec ; Trivy PyYAML 5.3.1)	20/08, 07/09	2, 3
T10	EX10 fédération	✓ Conforme	Inventaire du 08/09 : zéro clé sur les sept comptes de service dédiés ; condition de la fédération vérifiée (dépôt <code>Mansour37/menal-zero-trust</code> , branche <code>main</code>) ; aucune permission <code>id-token</code> sur demande de fusion (annexe D)	08/09	3, 4
T11	EX11 collecte	■ Partiel	Collecte en flux effective : 232 254 journaux bruts dans l'entrepôt au 08/09, l'agent du puits en étant le seul auteur ; alerte d'absence reçue après 36 min (cible 30) dans la conception initiale ; l'alerte A3 (10 min) est configurée et non encore chronométrée	22/08, 08/09	3
T12	EX12 preuves	✓ Conforme	Droits du jeu de données relevés le 08/09 : écriture réservée au job de détection, à la chaîne et à l'agent du puits, lecture pour l'API, le tableau de bord et l'enrichissement ; test automatisé sous l'identité du job d'enrichissement : lecture réussie, insertion dans <code>detections</code> refusée, rejoué à chaque livraison	19/08, 08/09	1, 3

l'alerte d'absence n'a été chronométrée que dans la conception initiale (36 min pour 30) : la nouvelle alerte A3 n'est pas déclarée conforme sans relevé, ce qui contredirait la méthode ; sa mesure figure au plan d'amélioration (tableau 6.4). **T12 et T9** portent les preuves les plus fortes : T12 tente réellement, à chaque livraison, l'écriture interdite et n'est vert que si elle est refusée — il échouait tant que le job partageait son identité avec les règles, et la séparation des identités a

été faite *parce qu'il échouait* ; T9 réunit un refus que personne n'avait provoqué et quatre refus provoqués rejouables. Quatre tests sont **automatisés et rejoués** sans intervention (T2, T5, T12 à chaque livraison, T4 chaque matin) : la campagne établit qu'un dispositif tient quand il est construit, la validation continue qu'il tient encore après avoir évolué.

6.3 Démonstration de bout en bout

Un scénario de bout en bout relie en une seule séquence les preuves du point d'entrée, de la collecte, de la détection, de la qualification et du verdict. Le 07/09/2026, treize charges d'attaque — inclusion de fichier (/etc/passwd), traversées de chemin brutes et encodées, injections SQL (1=1, admin' -) — ont été envoyées depuis un poste Kali vers `elson.menal-sarl.com/api/search` et ont reçu treize refus 403 (figure 6.1). La figure 6.2 déroule la chronologie relevée sur les captures horodatées du tableau de bord : les règles R6, R3 et R2 ont produit 47 détections, corrélées en un incident unique de score 100 à deux tactiques, enrichi et affiché **2 min 05 s** après l'envoi ; le verdict « Confirmé » est enregistré à 2 min 23 s. Le résultat tient dans la borne de trois minutes établie par construction (section 4.8) et divise par sept le délai mesuré le 19/08 avec les requêtes planifiées (15 min 05 s).

```

kali@kali: ~/Desktop
Session Actions Edit View Help

(kali@kali)-[~/Desktop]
$ for u in "path=../../etc/passwd" "file=/etc/passwd" "path=../../../../etc/passwd" "file=../../etc/passwd" "path=../../etc/passwd" "q=admin'" "id=1" "file=/etc/passwd" "path=../../etc/passwd" "file=../../etc/passwd" "path=/etc/passwd" "file=/etc/passwd"; do printf "%s" "${curl -s -o /dev/null -w "%{http_code}" --http1.1 --max-time 15 "https://elson.menal-sarl.com/api/search?${u}"; sleep 0.4; done; echo
403 403 403 403 403 403 403 403 403 403 403 403 403
(kali@kali)-[~/Desktop]
$

```

Fig. 6.1 – Scénario du 07/09/2026, 23 h 25 min 38 s : treize charges d'attaque envoyées vers `elson.menal-sarl.com/api/search` depuis un poste Kali ; treize refus 403 rendus par Cloud Armor avant l'application (T1, rang 3).

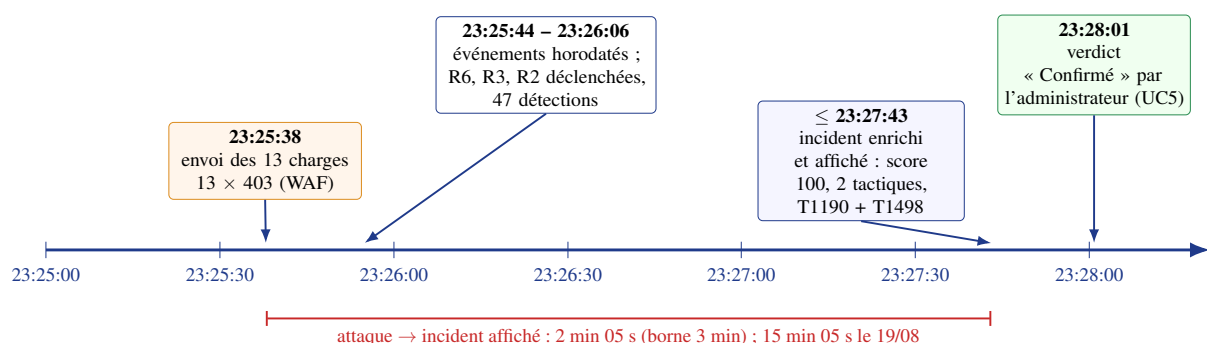


Fig. 6.2 – Chronologie du scénario du 07/09/2026, relevée sur la capture Kali et les captures horodatées du tableau de bord (figures 5.5 à 5.7). Le verdict humain intervient 2 min 23 s après l'envoi de l'attaque.

6.4 Évaluation de la détection et de la qualification assistée

Sept règles couvrent sept techniques et cinq tactiques ATT&CK (tableau 5.2) ; le mémoire publie surtout **ce que le socle ne détecte pas** : le mouvement latéral refusé par le pare-feu (tracé dans les journaux de flux, aucune règle ne les lit encore) ; l'exfiltration vers une destination autorisée

(indistinguable d'un usage normal) ; l'action de l'administrateur lui-même (exclu de A2, protégé par le second facteur) ; la fraude métier (hors périmètre) ; l'attaque distribuée ou le réseau mobile partagé (limitation de débit indexée sur l'adresse) ; et les seuils comportementaux de R1, R2 et R5, qui ne seront calibrés qu'avec du trafic réel.

La qualification assistée a été évaluée sur quarante alertes étiquetées à l'aveugle — vingt issues de la recette, vingt rejouées depuis les scénarios — selon un protocole rejouable (annexe D.6) ; quatre méthodes sont comparées sur la précision au rang 1 et aux rangs 1 à 3, qui correspond à l'usage réel (figure D.11, annexe D.6). **La cible est atteinte** : dans 88 % des cas la bonne technique figure parmi les trois candidats, pour environ 40 ms par alerte. L'écart avec TF-IDF (0,70) mesure l'apport de la compréhension du sens sur des alertes courtes — la réponse à la question ouverte du chapitre 2. Le rejet de la version quantifiée est confirmé chiffres à l'appui (0,05 au rang 1) : un gain de démarrage à froid ne vaut pas la perte de la fonction. La limite est claire : quarante alertes étiquetées par une seule personne forment un jeu indicatif et non une évaluation statistique robuste.

6.5 Performance, résilience et coût

Deux objectifs de service sont suivis sur trente jours glissants : une disponibilité de 99 % et 95 % des requêtes servies en moins d'une seconde (BNF1, BNF2), les erreurs clientes étant exclues puisqu'un refus d'authentification n'est pas une indisponibilité. Reconstruction, restauration et retour arrière ont été *exécutés et chronométrés* (tableau 6.2). La structure du coût importe davantage que son montant : trois postes fixes indépendants du trafic — répartiteur, WAF, base — portent 88 % du total, l'exécution des conteneurs coûte moins d'un euro hors trafic, et le coût marginal d'une application supplémentaire se limiterait à son exécution et à son stockage. Le passage de 53 à 41 € tient au forfait minimal des requêtes planifiées, divisé par cinq par le regroupement des règles ; le job de détection à la minute (D9) remplace ces requêtes planifiées et reste dans le même ordre de grandeur, le coût de septembre étant mesuré à la clôture du mois comme les précédents.

6.6 Confrontation aux principes Zero Trust et maturité

Le socle est confronté aux sept principes du NIST (tableau 6.3) en ne mobilisant que des preuves déjà établies : P1, P3, P4 et P6 sont tenus par des tests ; P2 par construction ; P5 avec réserves (T6 et T11 partiels) ; P7 tenu, la boucle de retour restant manuelle. L'auto-évaluation selon le modèle CISA place au niveau *avancé* les identités, les applications, la visibilité et l'automatisation ; au niveau *initial* les réseaux (pas de micro-segmentation par application — un choix), les données (stockage d'objets commun) et la gouvernance (décisions datées, aucune politique écrite) ; le pilier des terminaux reste *traditionnel*, vide par absence de périmètre. Le socle met en œuvre un Zero Trust mature sur le chemin nord-sud et un cloisonnement est-ouest porté par l'identité ; ses points faibles sont ceux d'une entreprise sans politique de sécurité écrite.

Tab. 6.2 – Mesures de performance, de résilience, d’exploitabilité et de coût.

Mesure	Cible	Valeur observée	Période	Statut
Disponibilité du point d’entrée (hors 4xx)	$\geq 99 \%$	99,6 %	30 j au 28/08	Tenue
Latence API de supervision (médiane / p95 / max)	p95 < 1 s	118 / 640 / 1 004 ms (un point, démarrage à froid)	07–08/08	Tenue
Délai attaque → incident affiché	≤ 3 min	2 min 05 s (15 min 05 s par lots)	07/09	Tenue
Chaîne de livraison complète	—	5 min 37 (fusion) ; 4 min 04 (demande de fusion)	07/09	Mesuré
Verdict d’une porte fermée	—	46 s (secrets) à 3 min 06 (image)	07/09	Mesuré
Démarrage à froid de l’encodeur	mesuré	27 s en moyenne, pic 94 s sur 12 réveils	août	Mesuré
Reconstruction complète de la recette	< 1 h	41 min, dont ≈ 15 min pour l’instance de base	24/07	Tenue
Restauration de la base (PITR)	RTO < 1 h ; RPO < 5 min	RTO 32 min 45 s ; RPO 0 (314 lignes attendues, 314 restaurées)	03/08	Tenue
Retour arrière applicatif	documenté, mesuré	11,6 s vers la révision précédente	12/08	Tenue
Temps d’exploitation hebdomadaire	une personne	≈ 1 h 30 (revue des incidents, plan hebdomadaire, verdicts)	29/07–28/08	Tenue
Coût mensuel de la recette	≤ 50 €	41 € en août (53 en juillet) ; trois postes fixes = 88 %	juillet–août	Tenue

Tab. 6.3 – Confrontation du socle aux sept principes du NIST SP 800-207 et réponse aux objectifs.

Principe / objectif	Preuves	Constat
P1 Toute ressource protégée	T3, T7, T12	Tenu
P2 Communication sécurisée	Chiffrement en transit et au repos (rang 4)	Tenu par construction
P3 Accès par session	T10 (aucune clé), T5 (jetons 5 et 60 min)	Tenu
P4 Politique fondée sur l’identité	T4, T7, T10	Tenu
P5 Surveillance continue	T11, T6, scénario du 07/09	Tenu avec réserves
P6 Authentification avant chaque accès	T5, T2	Tenu
P7 Les observations améliorent la politique	Verdicts, santé des règles ; T12 a fait corriger l’architecture	Tenu, boucle manuelle
O1 Identité	T2–T5, T10 conformes ; T1 corrigé	Réalisé, mesuré
O2 Infrastructure reproductible	Reconstruction 41 min ; T7 ; T6 partiel	Réalisé, mesuré ; réserve T6
O3 Livraison contrôlée	Quatre portes, cinq refus provoqués, un refus réel ; T8–T10	Réalisé, mesuré
O4 Supervision	Incident en 2 min 05 s ; T12 ; T11 partiel ; précision 0,88	Réalisé, mesuré ; réserve T11
K1 / K2 / K3	Restauration et retour arrière exécutés, 1 h 30/semaine ; 41 € ; chiffrement, journaux d’accès, restauration sans perte	Tenues (K3 : moyens techniques)

6.7 Limites et perspectives

Les résultats valent pour la recette, représentative de la cible mais avec un trafic pilote : seuils non calibrés, disponibilité d’un service peu sollicité, latence mesurée sur la seule API, alerte

d'absence non encore chronométrée dans la chaîne en flux, jeu d'évaluation petit. Deux limites sont structurelles : une faille de logique métier est invisible au socle (le trafic est légitime), et la limitation de débit par adresse est sans effet contre un adversaire distribué tout en frappant les utilisateurs derrière une adresse partagée, courante sur les réseaux mobiles du pays ; en sortir demande une limitation portée par l'identité de l'utilisateur, que seule l'application peut fournir.

Le socle héberge une application ; entièrement décrit en code, il en accueillera une autre en instanciant les mêmes modules, et ce qui se vend n'est pas l'hébergement mais la sécurité et la supervision qui viennent avec. Trois seuils le séparent d'un service commercialisable : *alerter réellement* (notification vers une personne d'astreinte avec accusé de prise en charge) ; *s'engager par écrit* (disponibilité, procédure d'incident, frontière de responsabilité — 99 % suppose la haute disponibilité de la base, environ 55 € par mois) ; *cloisonner la donnée jusqu'au stockage d'objets* et verrouiller la rétention des journaux d'audit. Chacun s'ajoute à l'architecture sans la modifier.

Tab. 6.4 – Plan d'amélioration issu de la revue finale du 08/09/2026.

#	Constat et action	Origine	Effet attendu
1	Le compte de service Compute par défaut conserve le rôle Éditeur hérité de la création du projet alors qu'aucune charge ne l'utilise : retirer le rôle et désactiver le compte	Inventaire IAM (annexe D)	Moindre privilège complet (EX4)
2	Le tableau de bord conserve un droit de lecture sur le jeu de données, hérité de sa première version, alors qu'il passe par l'API : retirer le droit	Droits du jeu de données (annexe D), écart É3	Le tableau de bord ne détient que le jeton de l'administrateur
3	Le secret de session du tableau de bord est chiffré par une clé gérée par Google et non par la clé du socle : aligner sur les huit autres secrets	Secret Manager (annexe D)	Chiffrement uniforme des secrets
4	L'alerte d'absence de journaux A3 n'a pas été chronométrée depuis le passage en flux : interrompre le puits et mesurer	T11 partiel	Clôture de T11
5	Une ressource créée hors code par l'administrateur n'est pas signalée : étendre A2 à toute création hors fenêtre d'application déclarée	T6 partiel	Clôture de T6
6	Les journaux de flux du VPC sont collectés mais aucune règle ne les lit : ajouter une règle sur les refus du pare-feu	Section 6.4	Visibilité du mouvement latéral refusé
7	Les images sont déployées par empreinte mais non signées : signer à la publication et vérifier au déploiement	Chapitre 5	Traçabilité complète de l'artefact

Conclusion

Les douze tests portent une preuve nommée, datée et hiérarchisée : dix conformes, deux partiels, conservés et expliqués. Les preuves les plus fortes ne sont pas des captures mais des tests rejoués, des portes fermées par des failles réelles et un scénario qui relie en deux minutes une attaque bloquée à son incident confirmé. Les quatre objectifs sont réalisés et mesurés.

Conclusion générale

Récapitulatif de la démarche. Ce projet est parti d'une question concrète : comment une entreprise émergente, avec une seule personne technique et quelques dizaines d'euros par mois, peut-elle héberger ses applications selon les principes Zero Trust et DevSecOps, et le démontrer plutôt que l'affirmer ? L'audit de l'hébergement d'ELSON a fixé quatre constats — confiance implicite au réseau, infrastructure non reproductible, livraison non contrôlée, supervision absente — et le projet a suivi une chaîne unique, de la valeur à la preuve : technologies choisies par un critère décisif et un prix payé ; douze menaces et douze exigences reliées, une pour une, à douze tests écrits avant la conception ; architecture construite en trois environnements identiques en code, mise en service pour ELSON, puis mesurée.

Résultats. Les quatre objectifs sont atteints et mesurés. *O1* : neuf identités, aucune clé stockée, base sans adresse publique, chaîne fédérée, session administrateur à deux facteurs ; *T1*, non conforme le 23/08, a été corrigé et rejoué le 07/09 avec treize refus sur treize charges. *O2* : la recette a été reconstruite depuis le seul dépôt en 41 minutes et l'état est protégé ; la détection de dérive reste partielle (*T6*). *O3* : quatre portes bloquantes, chacune fermée par un refus provoqué le 07/09 et l'une par un refus réel le 20/08 ; retour arrière en 11,6 s. *O4* : sept règles exécutées chaque minute sur des journaux en flux, une qualification assistée exacte à 88 % aux rangs 1–3, une corrélation par entité ; le passage en flux (*D9*) a ramené le délai attaque → incident de 15 min 05 s à 2 min 05 s. Les contraintes sont tenues (1 h 30 d'exploitation par semaine, 41 € par mois, restauration sans perte) ; deux tests restent partiels (*T6*, *T11*), conservés et expliqués.

Problèmes rencontrés. Sept difficultés ont chacune laissé une correction et une leçon (chapitre 5) ; trois retiennent l'attention : une porte d'analyse qui se déclarait verte sans rien vérifier — une porte déclarée n'est pas une porte vérifiée ; une règle réseau que le trafic ne traversait pas — un contrôle qui ne voit pas passer le trafic est une déclaration ; une alerte qui arrivait quinze minutes après l'attaque — le délai était dans l'acheminement, non dans le calcul.

Apports. Pour l'entreprise, le socle est un actif décrit en code, qui héberge ELSON et accueillera la prochaine application par paramétrage, avec des procédures chronométrées et une frontière de responsabilité écrite pouvant fonder une offre d'hébergement sécurisé. Sur le plan méthodologique, le projet montre qu'une architecture de sécurité peut être vérifiée par une personne seule si les tests précèdent la conception, si chaque preuve porte un rang et une date et si les résultats défavorables sont conservés ; sur le plan personnel, j'ai tenu les rôles d'auditeur, d'architecte, de développeur et d'exploitant.

Perspectives. Trois seuils séparent le socle d'un service commercialisable sans remettre en cause l'architecture : alerter une personne d'astreinte, s'engager par écrit sur la disponibilité, cloisonner la donnée jusqu'au stockage d'objets. Le plan d'amélioration du chapitre 6 fixe les travaux suivants : chronométrer l'alerte d'absence de collecte (*T11*), élargir l'alerte de dérive (*T6*), retirer les droits hérités sans usage, lire les journaux de flux, signer les images. La promotion d'ELSON vers la production, à l'étiquette $v1.0.0$, sera la prochaine preuve à dater.

Bibliographie / Nétographie

- [1] RÉPUBLIQUE ISLAMIQUE DE MAURITANIE, *Loi n° 2017-020 du 22 juillet 2017 relative à la protection des données à caractère personnel*, 2017.
- [2] PARLEMENT EUROPÉEN ET CONSEIL DE L'UNION EUROPÉENNE, *Règlement (UE) 2016/679 relatif à la protection des personnes physiques à l'égard du traitement des données à caractère personnel (RGPD)*, Journal officiel de l'Union européenne, L 119, p. 1–88, 2016. <https://eur-lex.europa.eu/eli/reg/2016/679/oj>
- [3] S. ROSE, O. BORCHERT, S. MITCHELL et S. CONNELLY, « Zero Trust Architecture », National Institute of Standards and Technology, NIST Special Publication 800-207, août 2020.
- [4] THE MITRE CORPORATION, *MITRE ATT&CK Enterprise Matrix*, version 17.1 (mai 2025), consultée le 07/09/2026. <https://attack.mitre.org>
- [5] AGENCE NATIONALE DE LA SÉCURITÉ DES SYSTÈMES D'INFORMATION, « EBIOS Risk Manager — La méthode », ANSSI, version 1.5, 2024.
- [6] A. SHOSTACK, *Threat Modeling: Designing for Security*. Indianapolis : Wiley, 2014.
- [7] J. KINDERVAG, « No More Chewy Centers: Introducing the Zero Trust Model of Information Security », Forrester Research, sept. 2010.
- [8] R. WARD et B. BEYER, « BeyondCorp: A New Approach to Enterprise Security », ;login: *The USENIX Magazine*, vol. 39, n° 6, p. 6–11, 2014.
- [9] CYBERSECURITY AND INFRASTRUCTURE SECURITY AGENCY, « Zero Trust Maturity Model, Version 2.0 », CISA, avril 2023. <https://www.cisa.gov/zero-trust-maturity-model>
- [10] M. SOUPPAYA, K. SCARFONE et D. DODSON, « Secure Software Development Framework (SSDF) Version 1.1 », NIST Special Publication 800-218, février 2022.
- [11] Z. RICE, *Gitleaks — Detect and Prevent Secrets in Git Repositories*, consulté le 07/09/2026. <https://github.com/gitleaks/gitleaks>
- [12] SEMGREP, INC., *Semgrep — Lightweight Static Analysis for Many Languages*, documentation, consultée le 07/09/2026. <https://semgrep.dev/docs/>
- [13] AQUA SECURITY, *Trivy — Comprehensive Vulnerability and Misconfiguration Scanner*, consulté le 07/09/2026. <https://trivy.dev/>
- [14] PRISMA CLOUD (BRIDGECREW), *Checkov — Static Analysis for Infrastructure as Code*, consulté le 07/09/2026. <https://www.checkov.io/>
- [15] K. MORRIS, *Infrastructure as Code: Dynamic Systems for the Cloud Age*, 2^e éd. O'Reilly Media, 2020.
- [16] GOOGLE CLOUD, *Workload Identity Federation*, documentation IAM, consultée le 07/09/2026. <https://cloud.google.com/iam/docs/workload-identity-federation>
- [17] OPEN SOURCE SECURITY FOUNDATION, *SLSA — Supply-chain Levels for Software Artifacts*, v1.0, 2023. <https://slsa.dev/spec/v1.0/>

- [18] F. ROTH et T. PATZKE, *Sigma — Generic Signature Format for SIEM Systems*, consulté le 07/09/2026. <https://github.com/SigmaHQ/sigma>
- [19] N. REIMERS et I. GUREVYCH, « Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks », in *Proceedings of EMNLP-IJCNLP 2019*, p. 3982–3992, 2019.
- [20] E. AGHAEI, X. NIU, W. SHADID et E. AL-SHAER, « SecureBERT: A Domain-Specific Language Model for Cybersecurity », in *SecureComm 2022*, LNICST vol. 462, Springer, p. 39–56, 2023.
- [21] B. ABDEEN, E. AL-SHAER, A. SINGHAL, L. KHAN et K. HAMLEN, « SMET: Semantic Mapping of CVE to ATT&CK and Its Application to Cybersecurity », in *DBSec 2023*, LNCS vol. 13942, Springer, p. 243–260, 2023. Modèle associé : ATT&CK-BERT, <https://huggingface.co/basel/ATTACK-BERT>
- [22] GOOGLE CLOUD, *BigQuery: Vector search overview; Scheduling queries; Customer-managed encryption keys*, documentation, consultée le 08/09/2026. <https://cloud.google.com/bigquery/docs>
- [23] HASHICORP, *Terraform Documentation — Google Cloud Platform Provider*, consultée le 07/09/2026. <https://developer.hashicorp.com/terraform/docs>
- [24] GITHUB, INC., *About security hardening with OpenID Connect*, GitHub Actions Documentation, consultée le 07/09/2026. <https://docs.github.com/en/actions/deployment/security-hardening-your-deployments/about-security-hardening-with-openid-connect>
- [25] OWASP FOUNDATION, *OWASP Top 10:2021*, 2021. <https://owasp.org/Top10/>
- [26] CENTER FOR INTERNET SECURITY, « CIS Google Cloud Platform Foundation Benchmark », version 3.0.0, 2024.
- [27] GOOGLE CLOUD, *Cloud Run: Restricting network ingress; Serverless VPC Access; Jobs*, documentation, consultée le 08/09/2026. <https://cloud.google.com/run/docs>
- [28] GOOGLE CLOUD, *Cloud SQL for PostgreSQL: Private IP; IAM database authentication; Point-in-time recovery*, documentation, consultée le 07/09/2026. <https://cloud.google.com/sql/docs/postgres>
- [29] GOOGLE CLOUD, *Cloud Logging: Route logs to BigQuery; Cloud Audit Logs*, documentation, consultée le 08/09/2026. <https://cloud.google.com/logging/docs>
- [30] GOOGLE CLOUD, *Cloud Armor: Preconfigured WAF rules; Rate limiting*, documentation, consultée le 07/09/2026. <https://cloud.google.com/armor/docs>
- [31] GOOGLE CLOUD, *Cloud Scheduler: Cron job schedules; Cloud Run: Execute jobs on a schedule*, documentation, consultée le 08/09/2026. <https://cloud.google.com/scheduler/docs>
- [32] D. M'RAIHI, S. MACHANI, M. PEI et J. RYDELL, « TOTP: Time-Based One-Time Password Algorithm », RFC 6238, IETF, mai 2011. <https://www.rfc-editor.org/rfc/rfc6238>
- [33] S. RAMÍREZ, *FastAPI — Security and dependencies*, documentation, consultée le 07/09/2026. <https://fastapi.tiangolo.com/>
- [34] VERCEL, INC., *Next.js Documentation — App Router, Server Components, Route Handlers*, consultée le 07/09/2026. <https://nextjs.org/docs>
- [35] J. ARKKO, M. COTTON et L. VEGODA, « IPv4 Address Blocks Reserved for Documentation », RFC 5737, IETF, janvier 2010 (adresses utilisées pour masquer les adresses réelles dans ce mémoire).

Chapitre A

Dépôt Git, environnements et extraits de code

A.1 Structure du dépôt

Un seul dépôt (Mansour37/menal-zero-trust) porte tout le socle ; son arbre reprend le modèle en couches du chapitre 4.

```
menal-zero-trust/
+-- terraform/
|   +-- modules/
|   |   +-- network/      VPC, sous-reseau, plage privée, pare-feu, journaux de flux
|   |   +-- run-service/  service Cloud Run paramétrique, son identité, ses secrets
|   |   +-- cloud-sql/    instance PostgreSQL privée ; une base et un utilisateur par app
|   |   +-- security/     comptes de service, federation WIF, secrets, clés KMS
|   |   +-- detection/    jeu de données, 8 tables, puits de journaux, jobs, planificateur, alertes
|   +-- envs/
|   |   +-- dev/          main.tf + dev.tfvars      (projet menal-zero-trust-dev)
|   |   +-- staging/      main.tf + staging.tfvars  (projet menal-zero-trust-staging)
|   |   +-- prod/         main.tf + prod.tfvars   (projet menal-zero-trust-prod, declare)
+-- .github/workflows/
|   +-- infra-checks.yml  F2 : format, validation, analyse de la description ; pas de plan
|   +-- app-delivery.yml  F3 : huit étapes, quatre portes, publication, déploiement
+-- api/                  API de supervision (FastAPI), job d'enrichissement, encodeur ONNX
+-- dashboard/            tableau de bord MENAL Sentinel (Next.js)
+-- ml-pipeline/           job de détection : normalisation des journaux et règles R1 à R7 en SQL
+-- tests/                T2, T5, T12 automatisés ; t1_waf_families.sh ; check_db_isolation.py
+-- demo/gate-tests/      rejou des cinq refus provoqués (une faille par branche jetable)
+-- docs/
|   +-- decisions/        D1 à D9, un fichier date par décision
|   +-- runbooks/         restauration, retour arrière, accueil d'une application, plan hebdo
```

A.2 Variables propres à chaque environnement

Le fichier `*.tfvars` d'un environnement ne contient que les valeurs ci-dessous ; tout le reste est commun aux trois environnements.

Tab. A.1 – Variables Terraform différant entre environnements.

Variable	Dév.	Recette	Production	Raison
project_id	...-dev	...-staging	...-prod	Isolation par projet
sql_tier	plus petite	petite	petite	Coût (K2)
sql_high_availability	false	false	true	Engagement de disponibilité
run_min_instances	0	0	1 pour l'application publique	Démarrage à froid contre coût
armor_mode	preview	enforce	enforce	Observation puis blocage
armor_ratelimit_per_min	600	60	60	Un seuil n'est réaliste qu'avec du trafic
detect_schedule	*/5 * * * *	* * * * *	* * * * *	Coût en développement, délai ailleurs (D9)
log_retention_days	7	30	30 ; audit : 400	Coût contre preuve
budget_alert_eur	15	50	80	K2 déclinée par environnement
allowed_repos_for_wif	dépôt, branche main	dépôt, branche main	dépôt, étiquette v*	Qui peut déployer où

A.3 Module run-service (extrait)

Module paramétrique qui rend l'accueil d'une application possible sans reconception (UC6).

```
# terraform/modules/run-service/main.tf --- extrait
variable "app_name" { type = string }
variable "image"    { type = string }      # reference par empreinte sha256
variable "secrets"  { type = map(string) } # NOM_ENV => identifiant du secret

resource "google_service_account" "app" { # une identite par service (EX4)
  account_id = "sa-${var.app_name}"
}

resource "google_cloud_run_v2_service" "app" {
  name      = var.app_name
  location  = var.region
  # entree reservee au repartiteur : aucune adresse publique (EX1)
  ingress   = "INGRESS_TRAFFIC_INTERNAL_LOAD_BALANCER"
  template {
    service_account = google_service_account.app.email
    scaling { min_instance_count = var.min_instances max_instance_count = 4 }
    vpc_access {
      connector = var.vpc_connector # menal-vpc-connector-stg, 10.0.3.0/28
      egress    = "ALL_TRAFFIC"    # tout le trafic sortant passe par le connecteur (E1)
    }
    containers {
      image = var.image
      dynamic "env" { # secrets lus par reference, jamais copies
        for_each = var.secrets
        content {
          name = env.key
          value_source { secret_key_ref { secret = env.value version = "latest" } }
        }
      }
    }
  }
}
# l'image appartient a la chaine de livraison, pas a l'infrastructure
lifecycle { ignore_changes = [template[0].containers[0].image] }

resource "google_secret_manager_secret_iam_member" "read" { # ses propres secrets seulement
  for_each = var.secrets
  secret_id = each.value
  role      = "roles/secretmanager.secretAccessor"
  member    = "serviceAccount:${google_service_account.app.email}"
}
```

A.4 Chaîne de livraison applicative : les portes (extrait de app-delivery.yml)

```
on:
  push: { branches: [main], paths: ["api/**", "dashboard/**", "tests/**"] }
  pull_request: { branches: [main], paths: ["api/**", "dashboard/**", "tests/**"] }
permissions: { contents: read } # id-token: write seulement aux jobs 7 et 8 (phase 2)

jobs:
  secrets: # 2 . PORTE BLOQUANTE
    name: 2 . Recherche de secrets (Gitleaks)
    steps:
      - uses: actions/checkout@<sha> # fetch-depth: 0 -> historique complet
        with: { fetch-depth: 0 }
      - uses: gitleaks/gitleaks-action@<sha>
  sast: # 3 . PORTE BLOQUANTE (diff-aware)
    needs: [secrets]
    steps:
      - run: semgrep scan --config p/default --baseline-commit "$BASE_SHA" --error --metrics=off
```

```

tests-api:      { needs: [sast], run: pytest -q }                # 4 . PORTE BLOQUANTE
tests-dashboard:{ needs: [sast], run: npm ci && npx jest }      # 4 . PORTE BLOQUANTE
build:          # 5 . image multi-etape, non root, artefact interne au run
  needs: [tests-api, tests-dashboard]
scan-image:     # 6 . PORTE BLOQUANTE
  needs: [build]
  steps:
    - uses: aquasecurity/trivy-action@<sha>
      with: { severity: CRITICAL, ignore-unfixed: true, exit-code: "1" }
    - uses: aquasecurity/trivy-action@<sha>                    # SBOM CycloneDX + rapport JSON -> BigQuery
      with: { format: cyclonedx, output: sbom.cdx.json }
publish:        # 7 . publication par empreinte sha256 (WIF, sans cle)
  needs: [scan-image]
  permissions: { contents: read, id-token: write }
  steps:
    - uses: google-github-actions/auth@<sha>                 # 1 . authentication federee
      with: { workload_identity_provider: ..., service_account: sa-cicd@... }
    - run: docker push "$IMAGE"
    - run: DIGEST=$(gcloud artifacts docker images describe "$IMAGE" | grep '@sha256:')
    - run: python scripts/load_cve_findings.py --report trivy-findings.json \
      --image-digest "$SHA"
deploy:         # 8 . deploiement par empreinte + sonde via le WAF
  needs: [publish]
  permissions: { contents: read, id-token: write }
  steps:
    - run: case "$REF" in *@sha256:*) ;; *) echo "pas d'empreinte"; exit 1 ;; esac
    - uses: google-github-actions/deploy-cloudrun@<sha>
      with: { image: "${{ steps.digest.outputs.ref }}" }
    - run: test "$(curl -s -o /dev/null -w '%{http_code}' https://$DOMAIN/health)" = "200"
    - if: failure()
      run: echo "::warning::Retour arriere (~12 s) : gcloud run services
        update-traffic $SERVICE --to-revisions=<precedente>=100"

```

A.5 Fédération d'identité de la chaîne (extrait)

```

resource "google_iam_workload_identity_pool_provider" "github" {
  attribute_condition = "assertion.repository == 'Mansour37/menal-zero-trust'
                        && assertion.ref == 'refs/heads/main'"
  oidc { issuer_uri = "https://token.actions.githubusercontent.com" }
}
resource "google_service_account_iam_member" "cicd_wif" {
  service_account_id = google_service_account.cicd.name          # sa-cicd : aucune cle (EX10)
  role = "roles/iam.workloadIdentityUser"
  member = "principalSet://iam.googleapis.com/${pool}/
            attribute.repository/Mansour37/menal-zero-trust"
}

```

A.6 Pare-feu de sortie (extrait)

```

resource "google_compute_firewall" "deny_all_egress" {          # refus par défaut, journalise
  direction = "EGRESS" priority = 65000
  deny { protocol = "all" }
  log_config { metadata = "INCLUDE_ALL_METADATA" }
}
resource "google_compute_firewall" "allow_connector_to_sql" {   # la seule route vers la base (EX3)
  direction = "EGRESS" priority = 1000 target_tags = ["vpc-connector"]
  destination_ranges = ["10.20.0.0/16"]                        # plage d'appairage de services privés
  allow { protocol = "tcp" ports = ["5432"] }
}

```

A.7 Règle de détection R6 (extrait simplifié)


```
-- ml-pipeline/rules/R6_injection_pattern.sql  (job a la minute, fenetre 5 min)
INSERT INTO `menal_security.detections`
(id, detected_at, rule_id, severity, entity, application_id,
tactic, technique, message, hit_count)
SELECT
TO_HEX(SHA256(CONCAT('R6|', src_ip, '|', ANY_VALUE(path), '|',
CAST(MIN(ts) AS STRING)))),
CURRENT_TIMESTAMP(), 'R6', 'CRITICAL', src_ip, application_id, 'TA0001', 'T1190',
CONCAT('Pattern injection : ', ANY_VALUE(path)), COUNT(*)
FROM `menal_security.access_logs` a
WHERE ts >= TIMESTAMP_SUB(CURRENT_TIMESTAMP(), INTERVAL 5 MINUTE)
AND (REGEXP_CONTAINS(LOWER(path), r"(l=1|union\s+select|<script|/etc/passwd)")
OR (log_source = 'armor' AND waf_rule IN ('sqli', 'xss', 'lfi')))
GROUP BY src_ip, application_id
HAVING NOT EXISTS (
-- deduplication
SELECT 1 FROM `menal_security.detections` d
WHERE d.rule_id = 'R6' AND d.entity = a.src_ip
AND d.detected_at >= TIMESTAMP_SUB(CURRENT_TIMESTAMP(), INTERVAL 5 MINUTE));
```

A.8 Test automatisé T12 (extrait)

```
# tests/test_t12_proof_tables_readonly.py
# rejoue a chaque livraison sous l'identite sa-enrich-job
def test_enrich_job_cannot_write_proof_table(bq_as_enrich_job):
    # lecture : autorisee
    bq_as_enrich_job.query("SELECT id FROM `menal_security.detections` LIMIT 1").result()
    # ecriture : refusee
    with pytest.raises(Forbidden):
        bq_as_enrich_job.query(
            "INSERT INTO `menal_security.detections` (id, rule_id) VALUES ('t12', 'TEST')"
        ).result()
```

Chapitre B

Diagrammes et écrans complémentaires

B.1 Matrice de risque

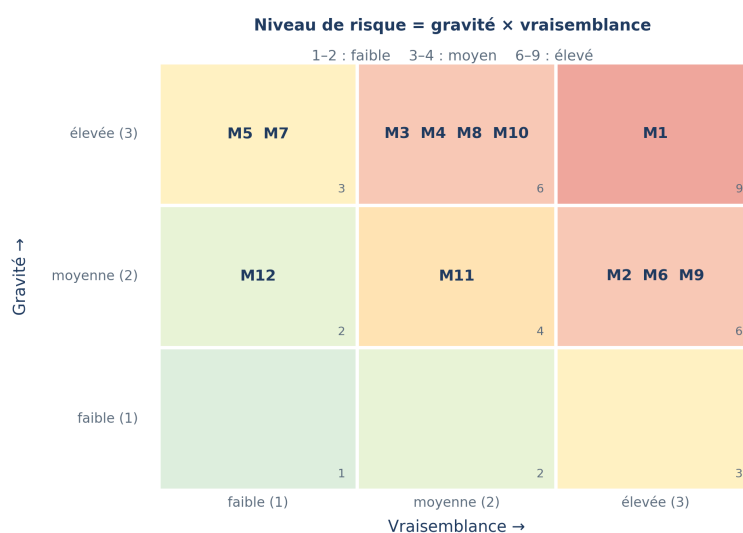


Fig. B.1 – Matrice de risque : position des douze menaces selon leur gravité et leur vraisemblance (tableau 3.3).

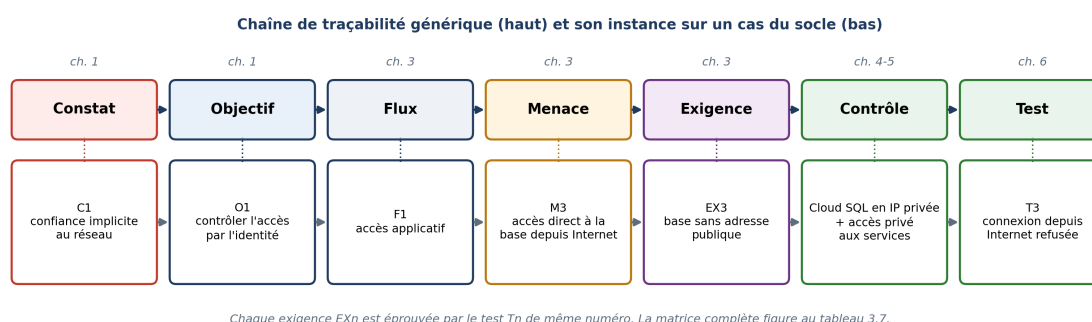


Fig. B.2 – Chaîne de traçabilité, de la valeur à la preuve, et son instance sur le cas M3 → EX3 → T3.

B.2 Principe de la qualification assistée

Principe : le texte de l'alerte et les descriptions des techniques sont transformés en vecteurs , les techniques les plus proches sont proposées.

Déterministe : à alerte identique, candidats identiques. Aucune génération de texte libre.

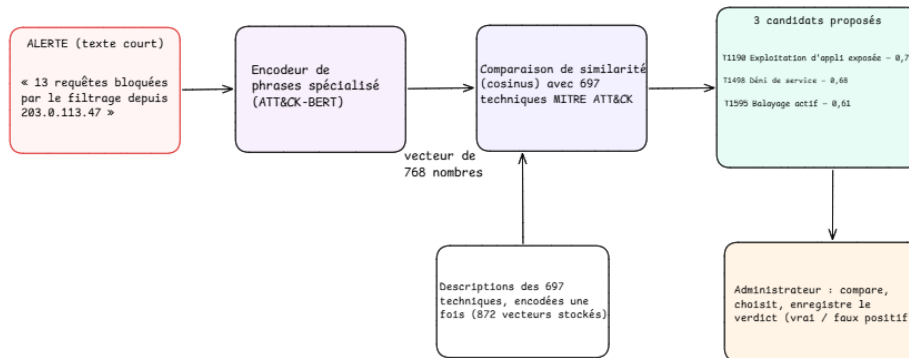


Fig. B.3 – Principe de la qualification assistée : l’alerte et les descriptions des techniques ATT&CK sont encodées en vecteurs ; les trois techniques les plus proches sont proposées à l’administrateur, qui décide.

B.3 Diagrammes de classes de l’API de supervision

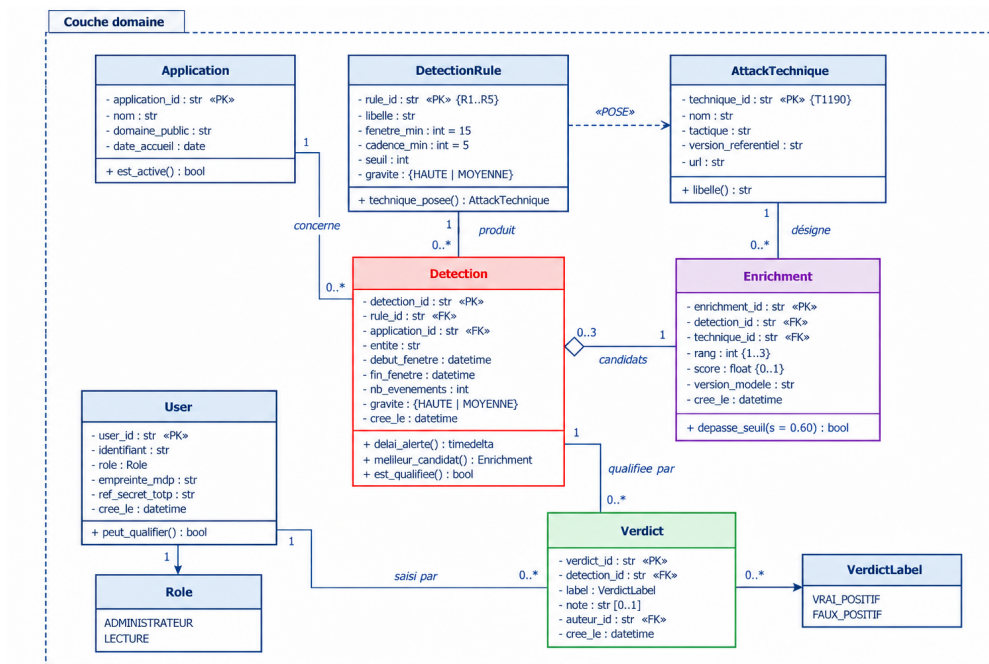


Fig. B.4 – Couche domaine : Detection, Enrichment et Verdict n’exposent aucune opération de modification ni de suppression ; l’immuabilité des preuves est portée par le modèle objet et par les droits IAM (T12).

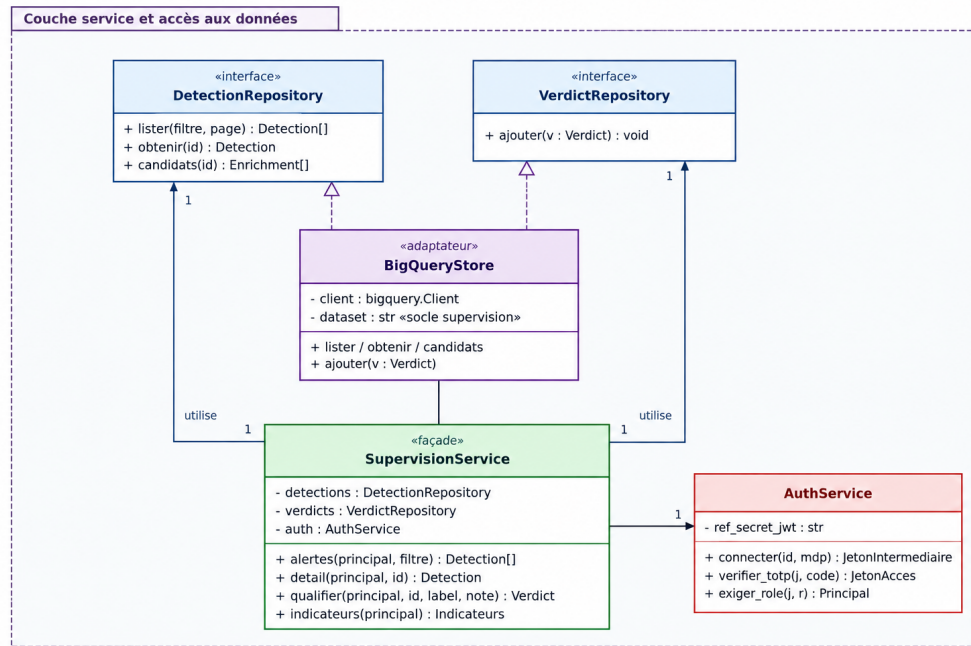


Fig. B.5 – Couche service : dépôts en lecture seule sur l’entrepôt, service d’authentification à deux jetons, service de corrélation par entité.

B.4 Écrans secondaires du tableau de bord

Règle	Sévérité	Technique	Déclenchement(s) récent(s)	Dernière mise à jour
R1 - Règle de sécurité	[CRITIQUE]	T1000	25	2023-09-09 10:00
R2 - Règle de sécurité	[HAUTE]	T1000	25	2023-09-09 10:00
R3 - Règle de sécurité	[HAUTE]	T1000	9	2023-09-09 10:00
R4 - Règle de sécurité	[HAUTE]	T1000	0	2023-09-09 10:00
R5 - Règle de sécurité	[HAUTE]	T1000	0	2023-09-09 10:00
R6 - Règle de sécurité	[HAUTE]	T1000	0	2023-09-09 10:00
R7 - Règle de sécurité	[HAUTE]	T1000	0	2023-09-09 10:00

CVE	Sévérité	Statut	Score	Version	Dernière mise à jour
CVE-2023-1234	[CRITIQUE]	En cours	9.5	1.0.0	2023-09-09 10:00
CVE-2023-1235	[CRITIQUE]	En cours	9.5	1.0.0	2023-09-09 10:00
CVE-2023-1236	[CRITIQUE]	En cours	9.5	1.0.0	2023-09-09 10:00
CVE-2023-1237	[CRITIQUE]	En cours	9.5	1.0.0	2023-09-09 10:00
CVE-2023-1238	[CRITIQUE]	En cours	9.5	1.0.0	2023-09-09 10:00
CVE-2023-1239	[CRITIQUE]	En cours	9.5	1.0.0	2023-09-09 10:00
CVE-2023-1240	[CRITIQUE]	En cours	9.5	1.0.0	2023-09-09 10:00
CVE-2023-1241	[CRITIQUE]	En cours	9.5	1.0.0	2023-09-09 10:00
CVE-2023-1242	[CRITIQUE]	En cours	9.5	1.0.0	2023-09-09 10:00
CVE-2023-1243	[CRITIQUE]	En cours	9.5	1.0.0	2023-09-09 10:00
CVE-2023-1244	[CRITIQUE]	En cours	9.5	1.0.0	2023-09-09 10:00
CVE-2023-1245	[CRITIQUE]	En cours	9.5	1.0.0	2023-09-09 10:00
CVE-2023-1246	[CRITIQUE]	En cours	9.5	1.0.0	2023-09-09 10:00
CVE-2023-1247	[CRITIQUE]	En cours	9.5	1.0.0	2023-09-09 10:00
CVE-2023-1248	[CRITIQUE]	En cours	9.5	1.0.0	2023-09-09 10:00
CVE-2023-1249	[CRITIQUE]	En cours	9.5	1.0.0	2023-09-09 10:00
CVE-2023-1250	[CRITIQUE]	En cours	9.5	1.0.0	2023-09-09 10:00

Fig. B.6 – Santé des règles (gauche) : les sept règles, leur sévérité, leur technique et leurs déclenchements sur trente jours — R6, R3 et R2 ont réagi au scénario du 07/09, les quatre autres sont silencieuses. Vulnérabilités priorisées (droite) : CVE de l’image livrée, alimentées par le rapport Trivy de la chaîne, avec statut KEV, score EPSS et version corrective.

B.5 Séquences de la livraison (UC2)

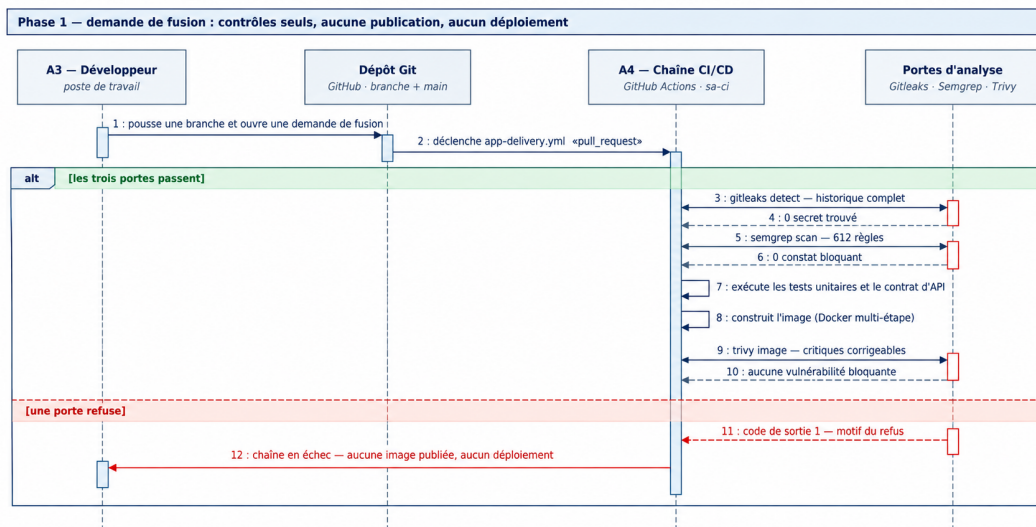


Fig. B.7 — Phase 1 — demande de fusion : les portes s'exécutent, l'image est construite mais rien n'est publié ni déployé ; aucun job ne détient de permission d'identité cloud.

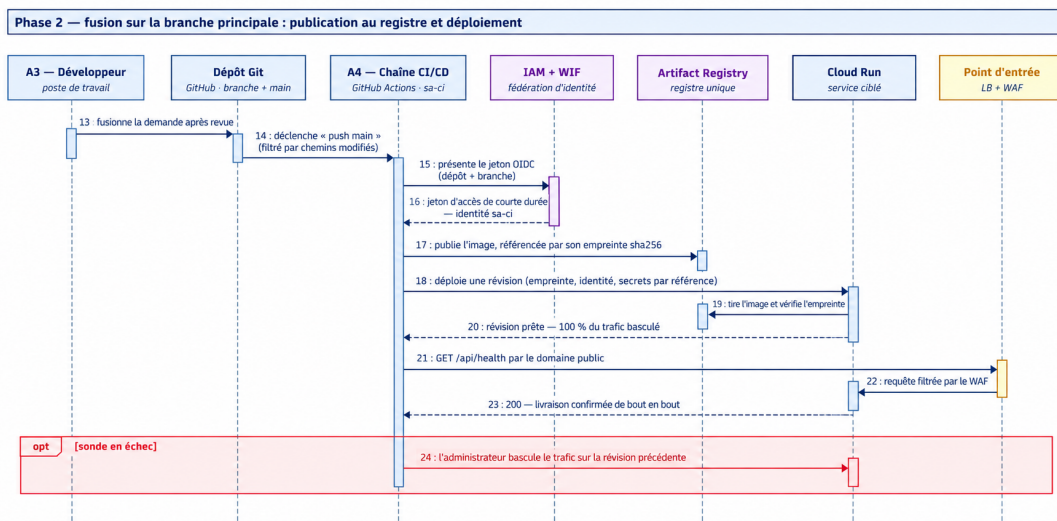


Fig. B.8 — Phase 2 — fusion sur la branche principale : authentification fédérée, publication par empreinte, déploiement, sonde par le point d'entrée ; le retour arrière est déclenché par l'administrateur.

Chapitre C

Relevés de la chaîne de livraison (07/09/2026)

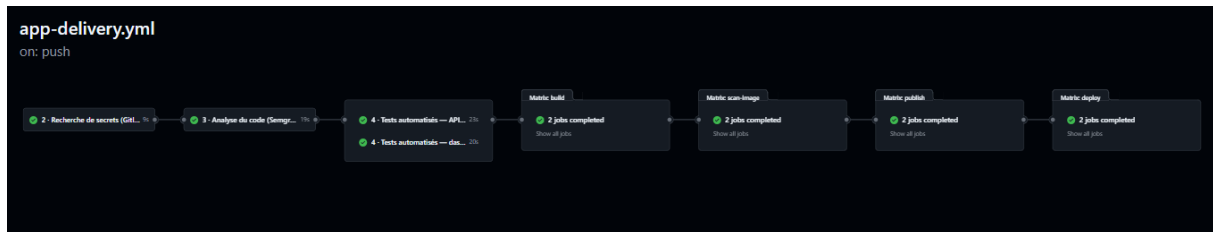


Fig. C.1 – Run vert de référence (34165407841, fusion sur main) : les huit étapes dans l'ordre du diagramme, dédoublées api/dashboard à partir de l'étape 4 ; total 5 min 37.

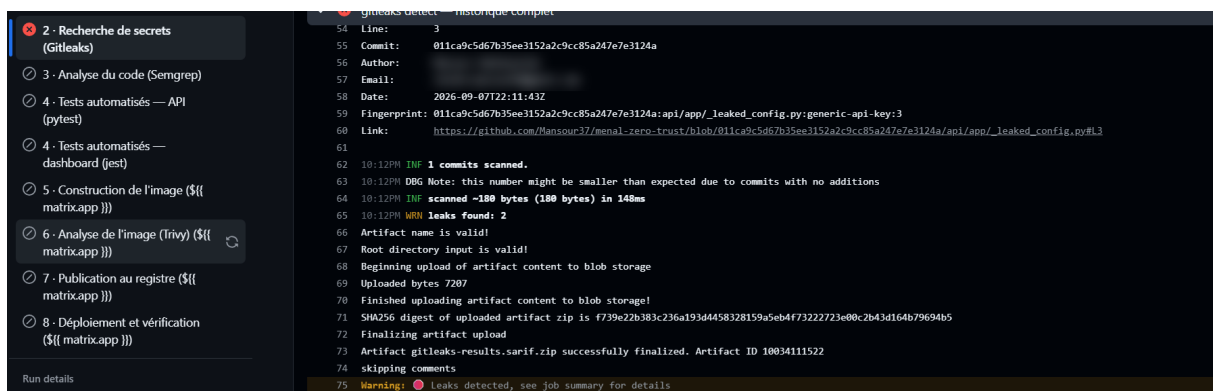


Fig. C.2 – Porte 2 fermée (run 34165824113) : Gitleaks détecte la clé injectée dans `api/app/_leaked_config.py` (règle `generic-api-key`), commit et empreinte identifiés ; auteur et courriel masqués.

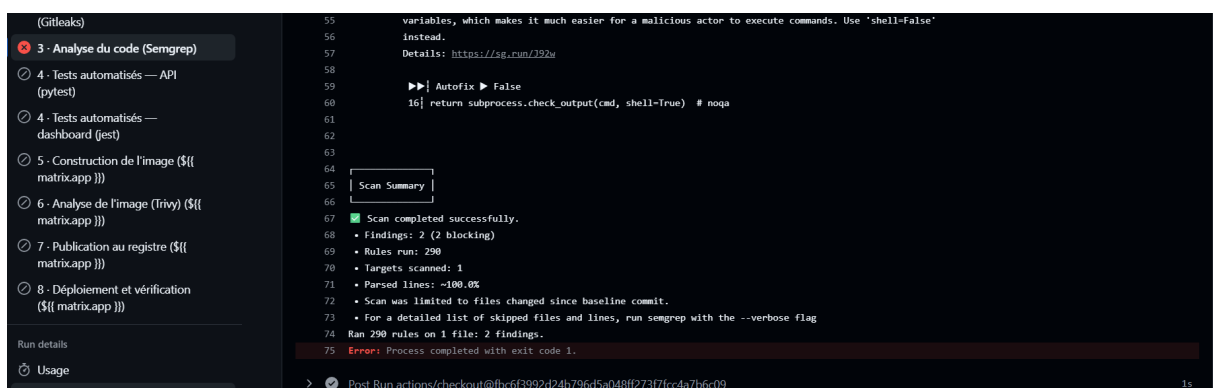


Fig. C.3 – Porte 3 fermée (run 34165832135) : Semgrep signale les deux constats injectés (`eval` sur entrée utilisateur, `subprocess` avec `shell=True`) sur 290 règles exécutées ; les étapes 4 à 8 sont ignorées.

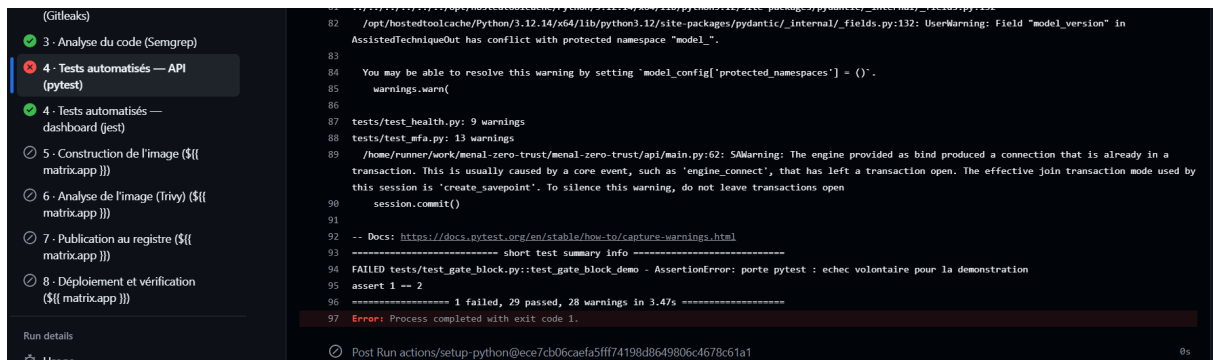


Fig. C.4 – Porte 4 fermée côté API (run 34165841139) : un test en échec sur trente ; construction, analyse d’image, publication et déploiement ignorés.



Fig. C.5 – Porte 4 fermée côté tableau de bord (run 34165850179) : un test en échec sur vingt-trois ; la porte réagit à la régression précise sans casser en bloc.

Chapitre D

Relevés de configuration et de tests

Les relevés ci-dessous ont été effectués sur l'environnement de recette (menal-zero-trust-staging) les 07 et 08/09/2026 avec les outils gcloud et bq et depuis la console Google Cloud. Chaque relevé porte la commande et sa sortie ; le texte n'en recopie pas les valeurs. Les adresses de courriel personnelles sont masquées. Un tiers disposant des droits de l'administrateur sur la recette peut rejouer chaque commande.

D.1 Point d'entrée et réseau (EX1, EX3)

☐	☒ Name ↑	Deployment type	Req/sec ⓘ	Region	Cost ⓘ	Authentication ⓘ	Ingress ⓘ
☐	☒ elson-api-staging	(🔗) Container	0	europe-west1	—	Public access	Internal + Load Balancing
☐	☒ elson-web-staging	(🔗) Container	0	europe-west1	—	Public access	Internal + Load Balancing
☐	☒ menal-api-staging	(🔗) Container	0.02	europe-west1	—	Public access	Internal + Load Balancing
☐	☒ menal-dashboard-staging	(🔗) Container	0	europe-west1	—	Public access	Internal + Load Balancing
☐	☒ menal-ml-embed-staging	(🔗) Container	0	europe-west1	—	Require authentication	Internal

Fig. D.1 – Services Cloud Run (console) : les quatre services publics n'acceptent que le trafic interne et celui du répartiteur (*Internal + Load Balancing*) ; l'encodeur est interne et exige une authentification.

```
PS C:\Users\manso> gcloud compute networks vpc-access connectors list `
>> --project=menal-zero-trust-staging `
>> --region=europe-west1 `
>> --format="table(name,ipCidrRange,state)"
CONNECTOR_ID: menal-vpc-connector-stg
IP_CIDR_RANGE: 10.0.3.0/28
STATE: READY
PS C:\Users\manso>
```

```
Windows PowerShell
PS C:\Users\manso> gcloud compute addresses list `
>> --global `
>> --project=menal-zero-trust-staging `
>> --format="table(name,address,prefixLength,purpose,status)"
NAME: google-services-staging
ADDRESS: 10.20.0.0
PREFIX_LENGTH: 16
PURPOSE: VPC_PEERING
STATUS: RESERVED
```

Fig. D.2 – Connecteur d'accès VPC menal-vpc-connector-stg (10.0.3.0/28, état READY) et plage d'appairage de services privés google-services-staging (10.20.0.0/16) qui porte l'adresse de Cloud SQL.

D.2 Base de données privée et isolée (T3, T4)

```
PS C:\Users\manso> gcloud sql instances list `
>> --project=menal-zero-trust-staging `
>> --format="table(name,databaseVersion,ipAddresses[0].ipAddress:label=IP,ipAddresses[0].type:label=TYPE)"
NAME: menal-db-staging
DATABASE_VERSION: POSTGRES_15
IP: 10.20.0.3
TYPE: PRIVATE
PS C:\Users\manso> |
>> --project=menal-zero-trust-staging
>> --format="table(name,charset)"
NAME: postgres
CHARSET: UTF8

NAME: menal_db
CHARSET: UTF8

NAME: elson_db
CHARSET: UTF8
PS C:\Users\manso> gcloud sql instances describe menal-db-staging `
>> --project=menal-zero-trust-staging `
>> --format="value(settings.ipConfiguration.ipv4Enabled,settings.ipConfiguration.sslMode)"
False ENCRYPTED_ONLY
PS C:\Users\manso> |
```

Fig. D.3 – T3 (rang 3) : instance menal-db-staging, PostgreSQL 15, adresse 10.20.0.3 de type PRIVATE ; trois bases (postgres, menal_db, elson_db) ; ipv4Enabled = False, sslMode = ENCRYPTED_ONLY.

```
Windows PowerShell
PS C:\Users\manso> gcloud run jobs executions list `
>> --project=menal-zero-trust-staging `
>> --region=europe-west1 `
>> --job=elson-sql-isolation-check-staging `
>> --limit=5
>> --format="table(metadata.name,status.conditions[0].type,status.succeededCount,metadata.creationTimestamp)"
NAME: elson-sql-isolation-check-staging-w8j5r
TYPE: Completed
SUCCEEDED_COUNT: 1
CREATION_TIMESTAMP: 2026-09-08T04:00:38.767231Z

NAME: elson-sql-isolation-check-staging-kwvts
TYPE: Completed
SUCCEEDED_COUNT: 1
CREATION_TIMESTAMP: 2026-09-07T04:00:05.992941Z

NAME: elson-sql-isolation-check-staging-snvlg
TYPE: Completed
SUCCEEDED_COUNT: 1
CREATION_TIMESTAMP: 2026-09-06T04:00:11.639465Z

NAME: elson-sql-isolation-check-staging-n4c6t
TYPE: Completed
SUCCEEDED_COUNT: 1
CREATION_TIMESTAMP: 2026-09-05T04:00:30.695729Z

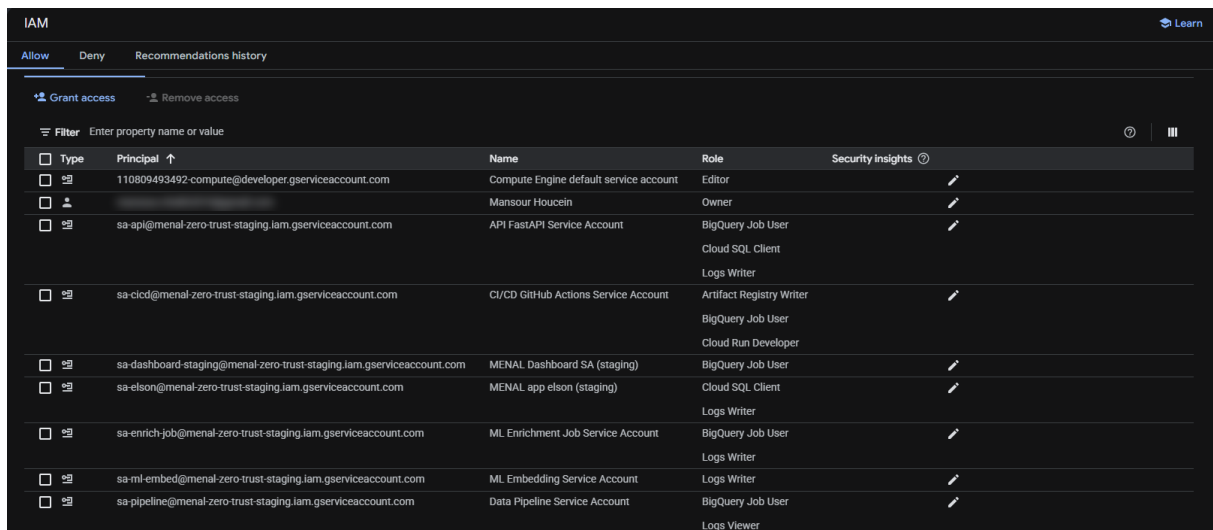
NAME: elson-sql-isolation-check-staging-mk6lv
TYPE: Completed
SUCCEEDED_COUNT: 1
CREATION_TIMESTAMP: 2026-09-04T04:00:10.188771Z
PS C:\Users\manso>

PS C:\Users\manso> gcloud logging read 'resource.labels.job_name="elson-sql-isolation-check-staging"' `
>> --project=menal-zero-trust-staging `
>> --limit=6 `
>> --freshness=2d `
>> --format="value(textPayload,jsonPayload)"

Container called exit(0).
checks=[{'detail': 'pg_has_role(api_user, cloudsqsuperuser, member) = false', 'name': 'api_user_not_cloudsqlsuperuser', 'ok': True}, {'detail': 'pg_has_role(elson_user, cloudsqsuperuser, member) = false', 'name': 'elson_user_not_cloudsqlsuperuser', 'ok': True}, {'detail': 'has_database_privilege(elson_user, menal_db, CONNECT) = false', 'name': 'cross_connect_rejected_elson_user_menal_db', 'ok': True}, {'detail': 'has_database_privilege(api_user, elson_db, CONNECT) = false', 'name': 'cross_connect_rejected_api_user_elson_db', 'ok': True}, {'detail': 'has_database_privilege(elson_user, elson_db, CONNECT) = true', 'name': 'self_connect_preserved', 'ok': True}, {'detail': 'tentative de connexion réelle elson_user -> menal_db rejetée', 'name': 'cross_connect_probe_rejected', 'ok': True}];isolated=True;message=SQL_ISOLATION_CHECK
npm notice run 'tsx' src/scripts/sql-isolation-check.ts
npm notice run hassaniya-backend@2.0.0 npx
PS C:\Users\manso> |
```

Fig. D.4 – T4 (rang 1) : cinq exécutions quotidiennes consécutives du job elson-sql-isolation-check-staging (du 04 au 08/09/2026) et le journal de la dernière : aucun utilisateur super-utilisateur, connexions croisées refusées dans les deux sens, accès propres préservés, isolated = True.

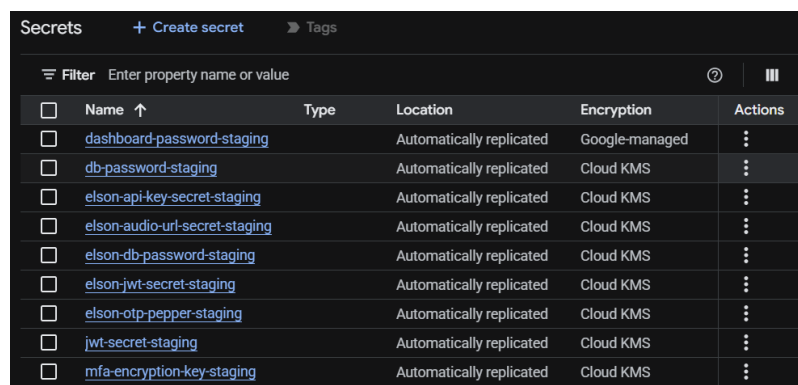
D.3 Identités, secrets et fédération (T4, T10)



The screenshot shows the IAM console with a table of service accounts. The table has columns for Type, Principal, Name, Role, and Security insights. The first row is the Compute Engine default service account (Editor role). The second row is the Owner (Mansour Houcein). The third row is the API FastAPI Service Account (BigQuery Job User, Cloud SQL Client, Logs Writer roles). The fourth row is the CI/CD GitHub Actions Service Account (Artifact Registry Writer, BigQuery Job User, Cloud Run Developer roles). The fifth row is the MENAL Dashboard SA (BigQuery Job User role). The sixth row is the MENAL app elson (Cloud SQL Client, Logs Writer roles). The seventh row is the ML Enrichment Job Service Account (BigQuery Job User, Logs Writer roles). The eighth row is the ML Embedding Service Account (Logs Writer role). The ninth row is the Data Pipeline Service Account (BigQuery Job User, Logs Viewer roles).

Type	Principal	Name	Role	Security insights
	110809493492-compute@developer.gserviceaccount.com	Compute Engine default service account	Editor	
		Mansour Houcein	Owner	
	sa-api@menal-zero-trust-staging.iam.gserviceaccount.com	API FastAPI Service Account	BigQuery Job User Cloud SQL Client Logs Writer	
	sa-cicd@menal-zero-trust-staging.iam.gserviceaccount.com	CI/CD GitHub Actions Service Account	Artifact Registry Writer BigQuery Job User Cloud Run Developer	
	sa-dashboard-staging@menal-zero-trust-staging.iam.gserviceaccount.com	MENAL Dashboard SA (staging)	BigQuery Job User	
	sa-elson@menal-zero-trust-staging.iam.gserviceaccount.com	MENAL app elson (staging)	Cloud SQL Client Logs Writer	
	sa-enrich-job@menal-zero-trust-staging.iam.gserviceaccount.com	ML Enrichment Job Service Account	BigQuery Job User Logs Writer	
	sa-ml-embed@menal-zero-trust-staging.iam.gserviceaccount.com	ML Embedding Service Account	Logs Writer	
	sa-pipeline@menal-zero-trust-staging.iam.gserviceaccount.com	Data Pipeline Service Account	BigQuery Job User Logs Viewer	

Fig. D.5 – Rôles IAM du projet (console) : un compte de service par composant avec ses seuls rôles techniques ; le compte Compute par défaut conserve le rôle Éditeur hérité de la création du projet (plan d’amélioration).



The screenshot shows the Secret Manager console with a table of secrets. The table has columns for Name, Type, Location, Encryption, and Actions. The first row is dashboard-password-staging (Automatically replicated, Google-managed). The second row is db-password-staging (Automatically replicated, Cloud KMS). The third row is elson-api-key-secret-staging (Automatically replicated, Cloud KMS). The fourth row is elson-audio-url-secret-staging (Automatically replicated, Cloud KMS). The fifth row is elson-db-password-staging (Automatically replicated, Cloud KMS). The sixth row is elson-jwt-secret-staging (Automatically replicated, Cloud KMS). The seventh row is elson-otp-pepper-staging (Automatically replicated, Cloud KMS). The eighth row is jwt-secret-staging (Automatically replicated, Cloud KMS). The ninth row is mfa-encryption-key-staging (Automatically replicated, Cloud KMS).

Name	Type	Location	Encryption	Actions
dashboard-password-staging		Automatically replicated	Google-managed	
db-password-staging		Automatically replicated	Cloud KMS	
elson-api-key-secret-staging		Automatically replicated	Cloud KMS	
elson-audio-url-secret-staging		Automatically replicated	Cloud KMS	
elson-db-password-staging		Automatically replicated	Cloud KMS	
elson-jwt-secret-staging		Automatically replicated	Cloud KMS	
elson-otp-pepper-staging		Automatically replicated	Cloud KMS	
jwt-secret-staging		Automatically replicated	Cloud KMS	
mfa-encryption-key-staging		Automatically replicated	Cloud KMS	

Fig. D.6 – Secret Manager : neuf secrets, un par usage ; huit chiffrés par la clé Cloud KMS du socle.

```

PS C:\Users\manso> $secrets = @(
>> gcloud secrets list `
>> --project=menal-zero-trust-staging `
>> --format="value(name)"
>> )
PS C:\Users\manso>
PS C:\Users\manso> foreach ($secret in $secrets) {
>>
>> $json = gcloud secrets get-iam-policy $secret `
>> --project=menal-zero-trust-staging `
>> --format=json | Out-String
>>
>> $policy = $json | ConvertFrom-Json
>>
>> $members = @(
>> $policy.bindings |
>> ForEach-Object { $_.members } |
>> Where-Object { $_ -match "serviceAccount:sa-" } |
>> Sort-Object -Unique
>> )
>>
>> "{0,-32} :: {1}" -f $secret, ($members -join " ")
>> }
dashboard-password-staging :: serviceAccount:sa-dashboard-staging@menal-zero-trust-staging.iam.gserviceaccount.com
db-password-staging :: serviceAccount:sa-api@menal-zero-trust-staging.iam.gserviceaccount.com
elson-api-key-secret-staging :: serviceAccount:sa-elson@menal-zero-trust-staging.iam.gserviceaccount.com
elson-audio-url-secret-staging :: serviceAccount:sa-elson@menal-zero-trust-staging.iam.gserviceaccount.com
elson-db-password-staging :: serviceAccount:sa-elson@menal-zero-trust-staging.iam.gserviceaccount.com
elson-otp-pepper-staging :: serviceAccount:sa-elson@menal-zero-trust-staging.iam.gserviceaccount.com
elson-jwt-secret-staging :: serviceAccount:sa-api@menal-zero-trust-staging.iam.gserviceaccount.com
mfa-encryption-key-staging :: serviceAccount:sa-api@menal-zero-trust-staging.iam.gserviceaccount.com
PS C:\Users\manso> |

```

Fig. D.7 – T4 (rang 3) : pour chaque secret, l'unique identité autorisée à le lire (sa-elson pour les cinq secrets d'ELSON, sa-api pour la base, la signature des jetons et la clé MFA, sa-dashboard pour son secret de session).

```

Windows PowerShell
PS C:\Users\manso> gcloud iam workload-identity-pools providers describe menal-github-provider `
>> --project=menal-zero-trust-staging `
>> --location=global `
>> --workload-identity-pool=menal-github-pool `
>> --format="value(attributeCondition)"
assertion.repository == 'Mansour37/menal-zero-trust' && assertion.ref == 'refs/heads/main'
PS C:\Users\manso> |

Windows PowerShell
PS C:\Users\manso> $serviceAccounts = @(
>> "sa-api",
>> "sa-cicd",
>> "sa-pipeline",
>> "sa-enrich-job",
>> "sa-elson",
>> "sa-dashboard-staging"
>> )
PS C:\Users\manso>
PS C:\Users\manso> foreach ($sa in $serviceAccounts) {
>>
>> $email = "$sa@menal-zero-trust-staging.iam.gserviceaccount.com"
>>
>> $keys = @(
>> gcloud iam service-accounts keys list `
>> --managed-by=user `
>> --project=menal-zero-trust-staging `
>> --iam-account=$email `
>> --format="value(name)"
>> ) | Where-Object { $_.Trim() -ne "" }
>>
>> Write-Host "$($sa): $($keys.Count)"
>> }
sa-api: 0
sa-cicd: 0
sa-pipeline: 0
sa-enrich-job: 0
sa-elson: 0
sa-dashboard-staging: 0
PS C:\Users\manso> |

```

Fig. D.8 – T10 (rang 3 et 4) : condition de la fédération d'identité (dépôt Mansour37/menal-zero-trust, branche main) et inventaire des clés : zéro clé sur les sept comptes de service dédiés.

D.4 Entrepôt : droits par table et volumes (T11, T12)

```
PS C:\Users\manso> bq show --format=prettyjson "menal-zero-trust-staging:menal_security_staging"
{
  "access": [
    {
      "role": "WRITER",
      "specialGroup": "projectWriters"
    },
    {
      "role": "WRITER",
      "userByEmail": "sa-cicd@menal-zero-trust-staging.iam.gserviceaccount.com"
    },
    {
      "role": "WRITER",
      "userByEmail": "sa-pipeline@menal-zero-trust-staging.iam.gserviceaccount.com"
    },
    {
      "role": "WRITER",
      "userByEmail": "service-118809493492@gcp-sa-logging.iam.gserviceaccount.com"
    },
    {
      "role": "OWNER",
      "specialGroup": "projectOwners"
    },
    {
      "role": "OWNER",
      "userByEmail": "sa-enrich@menal-zero-trust-staging.iam.gserviceaccount.com"
    },
    {
      "role": "READER",
      "specialGroup": "projectReaders"
    },
    {
      "role": "READER",
      "userByEmail": "sa-api@menal-zero-trust-staging.iam.gserviceaccount.com"
    },
    {
      "role": "READER",
      "userByEmail": "sa-dashboard-staging@menal-zero-trust-staging.iam.gserviceaccount.com"
    },
    {
      "role": "READER",
      "userByEmail": "sa-enrich-job@menal-zero-trust-staging.iam.gserviceaccount.com"
    }
  ]
}
```

Fig. D.9 – T12 (rang 3) : droits du jeu de données menal_security_staging — écriture pour sa-cicd, sa-pipeline et l’agent du puits de journaux ; lecture pour sa-api, sa-dashboard et sa-enrich-job ; chiffrement par la clé KMS menal-api-key-staging.

```
PS C:\Users\manso> $q = @'
>> SELECT 'raw_logs' AS table_name, COUNT(*) AS lignes FROM 'menal_security_staging.raw_logs'
>> UNION ALL
>> SELECT 'access_logs', COUNT(*) FROM 'menal_security_staging.access_logs'
>> UNION ALL
>> SELECT 'security_events', COUNT(*) FROM 'menal_security_staging.security_events'
>> UNION ALL
>> SELECT 'detections', COUNT(*) FROM 'menal_security_staging.detections'
>> UNION ALL
>> SELECT 'alert_enrichment', COUNT(*) FROM 'menal_security_staging.alert_enrichment'
>> UNION ALL
>> SELECT 'analyst_verdicts', COUNT(*) FROM 'menal_security_staging.analyst_verdicts'
>> ORDER BY lignes DESC
>> '@
PS C:\Users\manso> bq query --project_id=menal-zero-trust-staging --use_legacy_sql=false --format=pretty $q
+-----+-----+
| table_name | lignes |
+-----+-----+
| raw_logs   | 232254 |
+-----+-----+
PS C:\Users\manso>
```

Fig. D.10 – T11 : volumes de l’entrepôt au 08/09/2026 (232 254 journaux bruts) et répartition des requêtes normalisées par service émetteur.

D.5 Chaîne de livraison (T8, T9)

Les relevés des portes fermées le 07/09/2026 — Gitleaks pour T8, Semgrep, pytest, jest et Trivy pour T9 — figurent en annexe C et au tableau 5.1.

D.6 Protocole d'évaluation de la qualification assistée

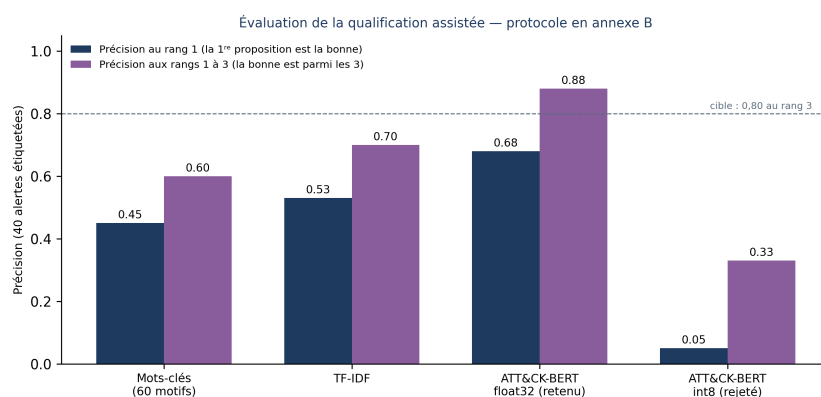


Fig. D.11 – Précision des quatre méthodes de qualification sur 40 alertes, au rang 1 puis aux rangs 1–3 : dictionnaire 0,45/0,60 ; TF-IDF 0,53/0,70 ; ATT&CK-BERT float32 (retenu) 0,68/0,88 ; ATT&CK-BERT int8 (rejeté) 0,05/0,33. Cible : 0,80 aux rangs 1–3.

1. **Jeu d'évaluation** : quarante alertes — vingt issues de la recette (juillet–août), vingt rejouées depuis les scénarios de test ; pour chacune, l'administrateur fixe à l'aveugle la technique ATT&CK de référence *avant* de voir les candidats du modèle.
2. **Méthodes comparées** : dictionnaire de soixante motifs (motif → technique) ; similarité TF-IDF entre le texte de l'alerte et les descriptions des techniques ; ATT&CK-BERT en précision flottante simple (retenu) ; ATT&CK-BERT quantifié sur huit bits.
3. **Mesures** : précision au rang 1 (la première proposition est la bonne) et aux rangs 1 à 3 (la bonne figure parmi les trois propositions) ; latence moyenne par alerte.
4. **Critère fixé à l'avance** : la méthode retenue doit atteindre 0,80 aux rangs 1 à 3 ; toute variante qui descend sous 0,60 au rang 1 est rejetée quel que soit son gain de démarrage.
5. **Résultats** : 0,45/0,60 ; 0,53/0,70 ; 0,68/0,88 (retenu, ≈ 40 ms par alerte) ; 0,05/0,33 (rejeté). Jeu et script d'évaluation versionnés dans `api/eval/`.
6. **Limite** : quarante alertes étiquetées par une seule personne ; le résultat est indicatif, non statistiquement robuste.

Chapitre E

Correspondance MITRE ATT&CK

Tab. E.1 – Règles du socle, alertes de plateforme et techniques MITRE ATT&CK (Enterprise v17.1).

Règle	Tactique	Technique	Ce que le socle observe
R1	TA0006 Credential Access	T1110 Brute Force	Rafale de 401/403 par adresse sur les chemins d'authentification
R2	TA0040 Impact	T1498 Network Denial of Service	Pic de blocages Cloud Armor par adresse
R3	TA0001 Initial Access	T1190 Exploit Public-Facing Application	Traversées de chemin brutes et encodées
R4	TA0007 Discovery	T1046 Network Service Discovery	Agents d'outillage sur les chemins d'API
R5	TA0040 Impact	T1499 Endpoint Denial of Service	Requêtes anormalement longues
R6	TA0001 Initial Access	T1190 Exploit Public-Facing Application	Motifs d'injection SQL, XSS, LFI (requêtes et verdicts WAF)
R7	TA0009 Collection	T1005 Data from Local System	Accès à des fichiers ou chemins sensibles
A1	TA0004 Privilege Escalation	T1078 Valid Accounts	Refus IAM répétés par une identité
A2	TA0005 Defense Evasion	T1562 Impair Defenses	Modification d'une ressource gérée hors chaîne
A3	TA0005 Defense Evasion	T1562.008 Disable or Modify Cloud Logs	Absence de journaux du répartiteur pendant dix minutes

Candidats proposés par la qualification assistée lors du scénario du 07/09/2026 : T1190 et T1498 retenus par la corrélation de l'incident ; T1003.008 *OS Credential Dumping : /etc/passwd* proposé sur les charges d'inclusion de fichier ($\approx 0,70$) ; sur la règle R2 (pic WAF), T1498 est proposé avec un score de 0,44, sous le seuil, et signalé comme incertain.



ECOLE SUPÉRIEURE PRIVÉE D'INGÉNIERIE ET DE TECHNOLOGIES

www.esprit.tn - E-mail : contact@esprit.tn

Siège Social : 18 rue de l'Usine - Charguia II - 2035 - Tél. : +216 71 941 541 - Fax. : +216 71 941 889

Annexe : Z.I. Chotrana II - B.P. 160 - 2083 - Pôle Technologique - El Ghazala - Tél. : +216 70 685 685 - Fax. : +216 70 685 454